

DOCKER

ENGINE • COMPOSE • SWARM

Do contêiner à produção

Fundamentos, Dockerfile, BuildKit, redes, volumes, Compose, segurança, observabilidade, orquestração e CI/CD, com tabelas de decisão, pegadinhas e flashcards. Conferido com o Docker Engine 29 e o Compose 5.

Erik Nathan eriknathan.me

CINCO BLOCOS DO GUIA

01

Fundamentos
Capítulos 1 e 2

02

Imagens e build
Capítulos 3 a 5

03

Rede, dados e Compose
Capítulos 6 a 8

04

Segurança e operação
Capítulos 9 e 10

05

Orquestração e produção
Capítulos 11 e 12

Sumário

COMO USAR ESTE GUIA

Versões, números e atualizações	6	Mapa do guia	7
Método para resolver problemas com Docker	7		

CAPÍTULO 1 – FUNDAMENTOS DE CONTÊINERES E ARQUITETURA DO DOCKER

1.1	Por que contêineres e por que Docker	8	1.4	Arquitetura do Docker	13
1.1.1	Problemas que os contêineres resolvem	8	1.4.1	Componentes	13
1.1.2	Para quem o Docker é bom	8	1.4.2	Padrões OCI	13
1.1.3	A inovação do Docker	8	1.4.3	Objetos do Docker	13
1.2	O que é um contêiner	9	1.5	Instalação e configuração do daemon	14
1.2.1	Definição	9	1.5.1	Requisitos	14
1.2.2	Os três pilares do kernel	9	1.5.2	Formas de instalar	14
1.2.3	Namespaces em detalhe	9	1.5.3	Instalação pelo repositório oficial (Debian e Ubuntu)	15
1.2.4	cgroups e netfilter	10	1.5.4	Pós-instalação no Linux	15
1.2.5	Nativo, máquina virtual e contêiner lado a lado	11	1.5.5	O arquivo daemon.json	16
1.2.6	Contêiner vs. máquina virtual	11	1.5.6	O daemon e suas opções	16
1.3	Uma breve história dos contêineres e do Docker	12	1.5.7	Contextos e acesso remoto	17
1.3.1	Antes do Docker	12	1.5.8	Docker Machine: o que era e o que usar hoje	17
1.3.2	O nascimento do Docker	12	1.5.9	Números de fundamentos	18
1.3.3	Do Docker 1.0 aos dias de hoje	12	1.6	Decisão rápida – Fundamentos	18

CAPÍTULO 2 – CICLO DE VIDA DE CONTÊINERES

2.1	Primeiros passos	20	2.2.3	Políticas de reinício	23
2.1.1	O que acontece em um docker run hello-world	20	2.3	Sinais, PID 1 e parada graciosa	23
2.1.2	Listar imagens e contêineres	20	2.3.1	Como o Docker para um contêiner	23
2.1.3	Modo interativo e modo em segundo plano	21	2.3.2	O problema do PID 1	24
2.1.4	Criar, iniciar, parar, pausar e remover	21	2.3.3	Códigos de saída	24
2.2	Executar contêineres	22	2.4	Inspeção, depuração e limpeza	24
2.2.1	docker run em detalhe	22	2.4.1	Comandos de observação	24
2.2.2	Estados do contêiner	23	2.4.2	Limpeza	25
			2.5	Decisão rápida – Ciclo de vida	25

CAPÍTULO 3 – IMAGENS E REGISTRIES

3.1	Anatomia de uma imagem	27	3.2.5	Enviar imagens ao Docker Hub	29
3.1.1	Camadas, manifest e digest	27	3.2.6	Registry próprio	30
3.1.2	Copy-on-write em detalhe	27	3.3	Gerenciar imagens	31
3.1.3	Referências de imagem	28	3.3.1	Comandos essenciais	31
3.2	Registries	28	3.3.2	Criar uma imagem com docker commit (e por que evitar)	31
3.2.1	Docker Hub	28	3.3.3	Estratégia de tags	32
3.2.2	Outros registries	29	3.3.4	Números de imagens e registries	32
3.2.3	Credenciais da CLI	29	3.4	Decisão rápida – Imagens e registries	32
3.2.4	Avaliar uma imagem antes de usar	29			

CAPÍTULO 4 – DOCKERFILE

4.1	Primeiro Dockerfile e primeiro build	34	4.3.1	Como o cache funciona	38
4.1.1	Do Dockerfile à imagem	34	4.3.2	Ordem que aproveita o cache	38
4.1.2	Processo em primeiro plano	34	4.4	Multi-stage builds	38
4.2	Instruções	35	4.4.1	Por que e como	38
4.2.1	Referência das instruções	35	4.5	Boas práticas	39
4.2.2	Forma shell vs. forma exec	36	4.5.1	Escolha da imagem base	39
4.2.3	CMD e ENTRYPOINT juntos	36	4.5.2	Checklist de um bom Dockerfile	39
4.2.4	COPY vs. ADD e o .dockerignore	37	4.5.3	Por que apagar arquivos em outro RUN não reduz a imagem	40
4.2.5	HEALTHCHECK	37	4.6	Decisão rápida — Dockerfile	40
4.3	Cache de build	38			

CAPÍTULO 5 – BUILDKIT, BUILDX E BUILDS AVANÇADOS

5.1	Recursos do BuildKit no Dockerfile	42	5.3	Imagens multiplataforma	44
5.1.1	Montagens em RUN --mount	42	5.3.1	Estratégias	44
5.1.2	Heredocs e build checks	42	5.4	Bake e atestados	44
5.2	Buildx: builders, cache e saídas	43	5.4.1	Docker Bake	44
5.2.1	Drivers de builder	43	5.4.2	SBOM e provenance	45
5.2.2	Saídas do build	43	5.5	Decisão rápida — BuildKit e Buildx	45
5.2.3	Cache exportável	43			

CAPÍTULO 6 – REDES

6.1	Drivers de rede	47	6.2.3	Como a publicação de portas funciona por dentro	49
6.1.1	Visão geral	47	6.2.4	Acessar o host a partir do contêiner	49
6.1.2	Bridge padrão vs. bridge definida pelo usuário	47	6.3	Redes entre hosts	49
6.1.3	Opções de rede do docker run	48	6.3.1	Overlay e redes de camada 2	49
6.2	Portas, firewall e acesso ao host	48	6.3.2	Números de rede	50
6.2.1	Publicação de portas	48	6.4	Diagnóstico de rede	50
6.2.2	Docker e o firewall do host	49	6.4.1	Ferramentas e sequência	50
			6.5	Decisão rápida — Redes	50

CAPÍTULO 7 – ARMAZENAMENTO E DADOS

7.1	Onde os dados ficam	52	7.2	Comportamento e permissões	54
7.1.1	Camada gravável e storage	52	7.2.1	Pré-população de volumes	54
7.1.2	Storage drivers: passado e presente	52	7.2.2	Permissões e UIDs	54
7.1.3	Tipos de montagem	53	7.2.3	Backup, restauração e drivers	55
7.1.4	-v vs. --mount	53	7.2.4	Comandos de volumes	55
7.1.5	Características dos volumes	53	7.3	Decisão rápida — Armazenamento	55
7.1.6	Padrão antigo: data-only container e --volumes-from	54			

CAPÍTULO 8 – DOCKER COMPOSE

8.1	Modelo e comandos	57	8.2.1	depends_on e healthchecks	59
8.1.1	Projeto, arquivos e nomes	57	8.2.2	Variáveis de ambiente e interpolação	60
8.1.2	Exemplo completo	58	8.2.3	Segredos e configs	60
8.1.3	Comandos do dia a dia	59	8.2.4	Combinar arquivos: override, -f, extends e include	60
8.2	Ordem de inicialização, variáveis e composição de arquivos	59	8.2.5	Perfis	60

8.3	Fluxo de desenvolvimento	61	8.3.2	Outros recursos úteis	61
8.3.1	develop.watch	61	8.4	Decisão rápida — Compose	61
CAPÍTULO 9 – SEGURANÇA					
9.1	Proteger o host e o daemon	63	9.3	Segredos	64
9.1.1	Superfície de ataque do daemon	63	9.3.1	Onde (não) colocar segredos	64
9.1.2	Modo rootless e userns-remap	63	9.4	Cadeia de suprimentos	65
9.2	Endurecer o contêiner em execução	63	9.4.1	Práticas e ferramentas	65
9.2.1	Capabilities	63	9.5	Decisão rápida — Segurança	65
9.2.2	Controles de segurança em execução	64			
CAPÍTULO 10 – RECURSOS, LOGS, OBSERVABILIDADE E TROUBLESHOOTING					
10.1	Limites de recursos	67	10.3.1	Fontes de informação	68
10.1.1	Memória, CPU e outros	67	10.4	Troubleshooting	69
10.1.2	Na prática: dimensionar e conferir limites	67	10.4.1	Sintomas, causas e ações	69
10.2	Logs	68	10.4.2	Números de operação	70
10.2.1	Drivers de log	68	10.5	Decisão rápida — Operação	70
10.3	Métricas e eventos	68			
CAPÍTULO 11 – ORQUESTRAÇÃO: DOCKER SWARM E O CAMINHO PARA KUBERNETES					
11.1	Docker Swarm mode	71	11.3	Stacks: arquivos Compose no Swarm	74
11.1.1	Arquitetura	71	11.3.1	docker stack na prática	74
11.1.2	Criar e administrar o cluster	71	11.3.2	Exemplo de stack de monitoramento	76
11.1.3	Quórum de managers	72	11.4	Do Docker ao Kubernetes	76
11.1.4	Serviços, atualizações e stacks	72	11.4.1	Equivalências	76
11.2	Secrets e configs do Swarm	73	11.4.2	Qual plataforma escolher	77
11.2.1	Comandos e uso	73	11.5	Decisão rápida — Orquestração	77
CAPÍTULO 12 – DOCKER EM PRODUÇÃO E CI/CD					
12.1	Pipeline de imagens	79	12.2.2	Execução	81
12.1.1	Etapas de um pipeline	79	12.2.3	Host e operação	81
12.1.2	Exemplo com GitHub Actions	80	12.3	Ecosistema e alternativas	81
12.1.3	Docker dentro do CI	80	12.3.1	Ferramentas compatíveis	81
12.2	Checklist de produção	81	12.4	Decisão rápida — Produção e CI/CD	82
12.2.1	Imagem	81			
CAPÍTULO 13 – PEGADINHAS E PADRÕES RECORRENTES					
13.1	Afirmções falsas clássicas	83	13.3	Pares que mais se confundem	85
13.2	Informações desatualizadas em materiais antigos	84	13.4	Palavras-chave e respostas	86
CAPÍTULO 14 – REFERÊNCIA RÁPIDA DE COMANDOS					
14.1	Contêineres	87	14.4	Compose	88
14.2	Imagens e build	87	14.5	Sistema, contextos e Swarm	88
14.3	Redes e volumes	88			
CAPÍTULO 15 – AUTOTESTE – FLASHCARDS DE REVISÃO					
15.1	Fundamentos e ciclo de vida	89	15.2	Imagens e Dockerfile	92

15.3	BuildKit e Buildx	95	15.6	Segurança e operação	100
15.4	Redes e armazenamento	96	15.7	Orquestração e produção	101
15.5	Compose	98			

Como usar este guia

- Use os capítulos 1 a 12 para estudar o conteúdo. A ordem segue o caminho de quem usa Docker no dia a dia: fundamentos, ciclo de vida do contêiner, imagens, Dockerfile, build, rede, dados, Compose, segurança, observabilidade, orquestração e produção.
- Use a tabela **Decisão rápida** ao final de cada capítulo como cheat sheet: cada uma reúne situações típicas, a abordagem recomendada e os erros mais comuns naquele tema.
- Use as tabelas **Números de...** para revisar portas, códigos de saída, padrões de tempo e limites que aparecem em entrevistas, provas e incidentes.
- Use as **Pegadinhas e padrões recorrentes** (capítulo 13) para revisar os erros de conceito mais frequentes, como "EXPOSE publica a porta" ou "docker system prune apaga volumes".
- Use a **Referência rápida de comandos** (capítulo 14) como consulta de bancada.
- Use o **Autoteste** (capítulo 15) para revisão ativa: cada flashcard esconde a resposta até você clicar.
- Os blocos de código podem ser copiados e executados em um host com Docker Engine 29 ou Docker Desktop recente. Comandos que exigem Linux (namespaces, iptables, /var/lib/docker) estão indicados.

Versões, números e atualizações

Os comportamentos e padrões deste guia foram conferidos na documentação oficial (docs.docker.com, github.com/docker e github.com/moby) em **setembro de 2026**. Nessa data, as versões correntes eram **Docker Engine 29.8** e **Docker Compose 5.5**. Limites comerciais (como os do Docker Hub) mudam com frequência: confirme na documentação antes de depender de um valor exato.

Mudanças recentes que ainda não aparecem na maior parte dos cursos e tutoriais:

- **Engine 29 (nov/2025):** o **containerd image store** passou a ser o padrão em instalações novas, no lugar dos storage drivers clássicos (overlay2 e afins); o daemon exige **API v1.44 ou mais nova** (clientes do Docker 25 em diante); **cgroup v1** foi marcado como obsoleto, com suporte garantido até pelo menos maio de 2029; há suporte **experimental a nftables** como backend de firewall.
- **Compose v5 (dez/2025):** a numeração pulou da 2.x para a 5.0 para não ser confundida com as antigas versões 2.x / 3.x do formato de arquivo. O builder interno foi removido e os builds do Compose passaram a ser delegados ao **Docker Bake**. O Compose também passou a ser usado como SDK por outras ferramentas.
- O campo `version: do compose.yaml` **é obsoleto**: o Compose sempre valida com o schema mais recente e emite um aviso se o campo existir.
- **Compose v1** (docker-compose, em Python) chegou ao fim da vida em 2023. O comando atual é `docker compose` (com espaço), um plugin da CLI.
- **Docker Hardened Images (DHI):** desde 17/dez/2025, mais de mil imagens base mínimas e endurecidas são gratuitas e open source (Apache 2.0), com SBOM e provenance assinados.
- **Docker Content Trust** (Notary v1) foi aposentado. Para assinar imagens, use Sigstore/cosign ou Notation (capítulo 9).
- **Kubernetes** não usa mais o Docker Engine como runtime desde a versão 1.24 (remoção do dockershim), mas executa normalmente as imagens criadas com Docker, porque elas seguem o padrão OCI.

Se você estuda por um livro ou curso mais antigo, a tabela **Informações desatualizadas em materiais antigos** (capítulo 13) lista o que mudou em instalação, storage drivers, Docker Machine, Compose, registries e Swarm.

Método para resolver problemas com Docker

- 1. Identifique a camada:** o problema é de build (Dockerfile, contexto, cache), de execução (processo, sinais, recursos), de rede (DNS, portas, firewall) ou de dados (volumes, permissões)? Cada camada tem comandos próprios.
 - 2. Leia o estado real, não o esperado:** `docker ps -a`, `docker inspect`, `docker logs` e `docker events` mostram o que aconteceu. O código de saída costuma dizer mais que a mensagem de erro.
 - 3. Reproduza no menor escopo:** `docker run --rm -it <imagem> sh` e um contêiner de diagnóstico na mesma rede (`nicolaka/netshoot`) isolam a causa.
 - 4. Prefira o recurso nativo:** healthcheck, restart policy, volumes nomeados, redes definidas pelo usuário, build secrets e multi-stage costumam resolver o que scripts caseiros tentam resolver.
 - 5. Contêiner é descartável:** se a correção exige entrar no contêiner e mudar algo à mão, a correção certa está no Dockerfile, no Compose ou na configuração.
-

Mapa do guia

Bloco	Capítulos	O que você deve dominar ao final
Fundamentos	1 e 2	Como um contêiner funciona no kernel, a arquitetura do Docker, o ciclo de vida, sinais, códigos de saída e depuração
Imagens e build	3, 4 e 5	Camadas, tags e digests, registries, Dockerfile, cache, multi-stage, BuildKit, Buildx, Bake e multiplataforma
Rede, dados e Compose	6, 7 e 8	Drivers de rede, DNS interno, publicação de portas, volumes e bind mounts, aplicações multi-contêiner com Compose
Segurança e operação	9 e 10	Superfície de ataque, capabilities, rootless, segredos, cadeia de suprimentos, limites, logs e troubleshooting
Orquestração e produção	11 e 12	Swarm, equivalências com Kubernetes, pipeline de CI/CD e checklist de produção
Revisão	13, 14 e 15	Pegadinhas, referência de comandos e flashcards

1. Fundamentos de Contêineres e Arquitetura do Docker

Ideia central: um contêiner **não é uma máquina virtual**. É um **processo comum do Linux** isolado por **namespaces**, limitado por **cgroups** e com um sistema de arquivos próprio montado a partir das **camadas de uma imagem**. Todos os contêineres de um host compartilham o **mesmo kernel**.

1.1 Por que contêineres e por que Docker

1.1.1 Problemas que os contêineres resolvem

Problema clássico	Como os contêineres ajudam	Onde no guia
"Na minha máquina funciona": desenvolvimento, testes e produção com versões diferentes de SO, bibliotecas e runtimes	A imagem carrega a aplicação com suas dependências; o mesmo artefato passa por todos os ambientes	Capítulos 3 e 12
Servidores preparados à mão para muitas aplicações, em linguagens, bibliotecas e versões diferentes e conflitantes	Cada aplicação roda isolada com as próprias dependências; o host só precisa do Docker	Capítulos 1 e 4
Instalação de servidores e deploys demorados, acionamentos de madrugada porque um serviço não sobe em outro servidor	Subir a aplicação é baixar a imagem e iniciar o contêiner, em segundos, em qualquer host compatível	Capítulos 2 e 11
Pico de demanda inesperado (uma campanha de marketing, por exemplo)	Novas réplicas sobem em segundos; um orquestrador distribui e substitui contêineres	Capítulos 8 e 11
Custo alto de datacenter e recursos ociosos	Muito mais aplicações por host do que com VMs, porque não há um sistema operacional por aplicação	Capítulos 1 e 10
Montar ambiente de desenvolvimento com banco, fila e cache	Um <code>docker compose up</code> sobe tudo, igual para todo o time	Capítulo 8
Configurar a mesma aplicação para vários ambientes	Configuração por variáveis de ambiente e segredos, sem mudar a imagem	Capítulos 8 e 9
Integração e entrega contínuas (CI/CD)	Build reproduzível, testes no mesmo ambiente da produção, promoção da mesma imagem	Capítulo 12
Experimentar uma ferramenta sem "sujar" a máquina	<code>docker run --rm</code> executa e descarta, sem instalar nada no host	Capítulo 2

1.1.2 Para quem o Docker é bom

- **Desenvolvedores:** liberdade para escolher linguagem, banco de dados e distribuição por projeto, e reproduzir o ambiente de produção na própria máquina.
- **Administradores de sistemas e SRE:** não é preciso preparar o servidor com as dependências de cada aplicação; o host pode ser físico ou virtual, e toda aplicação é operada da mesma forma (iniciar, parar, logs, limites).
- **A empresa:** transição mais rápida entre QA, homologação e produção (é a **mesma imagem**), menos hardware pelo melhor aproveitamento dos recursos e overhead bem menor que o da virtualização.
- **Arquiteturas de microsserviços:** dividir uma aplicação grande em partes pequenas, cada uma com seu ciclo de vida, fica muito mais viável quando cada parte é um contêiner.

1.1.3 A inovação do Docker

- **O Docker não inventou os contêineres.** Isolamento de processos existe no Unix há décadas (veja a linha do tempo mais adiante neste capítulo).

- A principal inovação foi de **experiência**: tornar essa tecnologia acessível a qualquer desenvolvedor ou administrador, sem anos de prática nem ferramentas caseiras. Isso veio de quatro peças:
 - o **Dockerfile**, uma receita em texto, versionável, para construir o ambiente;
 - a **imagem em camadas**, portátil e reaproveitável;
 - o **registry público** (Docker Hub) para compartilhar imagens;
 - uma **CLI e uma API simples**, resumidas no lema *build, ship, run*.
- O objetivo final não é "usar Docker": é entregar software melhor, mais confiável e mais rápido, com menor custo de desenvolvimento e implantação. O Docker é um meio muito eficiente para isso.

1.2 O que é um contêiner

1.2.1 Definição

- Um **contêiner** é uma aplicação empacotada **junto com suas dependências** (bibliotecas, runtime, arquivos de configuração) e executada como um **processo isolado** que **compartilha o kernel** do sistema operacional do host, seja ele uma máquina física ou virtual.
- Por isso se fala em **virtualização em nível de sistema operacional**: o contêiner não emula hardware nem executa outro kernel; o kernel do host apenas **isola** o que o processo enxerga e **limita** o que ele consome.
- A imagem de um contêiner costuma ser **enxuta**, com só o necessário para a aplicação. Em execução, o **overhead** em relação à mesma aplicação rodando nativamente é **pequeno**: não há hypervisor nem sistema operacional convidado. O custo que existe vem principalmente da rede com NAT e do sistema de arquivos em camadas, e desaparece com `--network host` e volumes.

1.2.2 Os três pilares do kernel

Mecanismo	O que faz	Exemplo prático
Namespaces	Isolam o que o processo enxerga	O contêiner vê só os próprios processos (PID 1 é a aplicação), a própria interface de rede e o próprio hostname
cgroups (control groups)	Limitam e medem o que o processo consome	<code>--memory 512m, --cpus 1.5, --pids-limit 200</code>
Union filesystem / snapshotter	Monta várias camadas somente leitura e uma camada gravável por cima	Dez contêineres da mesma imagem compartilham as camadas em disco; cada um tem só sua camada gravável

- Namespaces usados pelo Docker**: `pid` (processos), `net` (interfaces, rotas, portas), `mnt` (pontos de montagem), `uts` (hostname), `ipc` (memória compartilhada, filas), `user` (mapeamento de UID/GID, usado no modo rootless e no `userns-remap`) e `cgroup`.
- cgroups v2** é a versão atual. O cgroup v1 está obsoleto desde o Engine 29.
- Complementos de segurança do kernel: **capabilities** (dividem o poder do root em partes), **seccomp** (filtra chamadas de sistema) e **AppArmor/SELinux** (controle de acesso obrigatório). Veja o capítulo 9.
- Em **macOS e Windows**, o Docker Desktop roda uma **máquina virtual Linux** leve; os contêineres Linux rodam dentro dela. Contêineres Windows nativos existem, mas exigem host Windows e imagens Windows.

1.2.3 Namespaces em detalhe

Os namespaces foram entrando no kernel Linux aos poucos, entre 2002 e 2020. Não chegaram todos de uma vez.

Namespace	Isola	Kernel	Como se vê no Docker
-----------	-------	--------	----------------------

Namespace	Isola	Kernel	Como se vê no Docker
mnt	Pontos de montagem e o sistema de arquivos raiz. É a evolução do chroot : um processo não enxerga montagens de outro namespace	2.4.19 (2002)	Cada contêiner tem seu próprio <code>/</code> , montado a partir da imagem
uts	Hostname e nome de domínio NIS	2.6.19 (2006)	<code>--hostname</code> ; por padrão, o ID curto do contêiner
ipc	Memória compartilhada e semáforos System V e filas de mensagens POSIX	2.6.19 (2006)	<code>--ipc</code> (<code>private</code> , <code>shareable</code> , <code>container:<nome></code> , <code>host</code>)
pid	Árvore de processos : cada contêiner numera os próprios processos a partir do PID 1	2.6.24 (2008)	O mesmo processo tem um PID dentro do contêiner e outro no host
net	Interfaces, endereços, rotas, portas e regras de firewall	2.6.24–2.6.29 (2008–2009)	Interface <code>eth0</code> própria em cada contêiner
user	Mapeamento de UIDs e GIDs : o root de dentro pode ser um usuário comum fora	3.8 (2013)	Modo <code>rootless</code> e <code>userns-remap</code> (capítulo 9)
cgroup	Visão da hierarquia de cgroups	4.6 (2016)	O contêiner enxerga só o próprio cgroup (padrão com cgroup v2)
time	Relógios de tempo desde o boot	5.6 (2020)	Não usado pelo Docker por padrão

- **O mesmo processo, dois PIDs**: dentro do contêiner, a aplicação aparece como PID 1; no host, com um PID comum. Veja com:

```
docker run -d --name teste alpine sleep 600
docker exec teste ps -o pid,comm          # dentro: sleep é o PID 1
docker inspect -f '{{.State.Pid}}' teste  # no host: PID real, por exemplo 48213
ps -o pid,comm -p "$(docker inspect -f '{{.State.Pid}}' teste)"  # Linux
```

- **Como a rede de um contêiner é montada** (driver bridge): o Docker cria um **par veth**, como um cabo virtual com duas pontas. Uma ponta fica **no namespace de rede do contêiner**, com o nome `eth0` e um IP da rede (ex.: `172.17.0.3/16`). A outra fica **no host**, com o nome `vethXXXXXX`, e é ligada à **bridge** `docker0` (IP `172.17.0.1`, o gateway dos contêineres). No host, `ip addr` mostra `docker0` e as interfaces `veth*`. No contêiner, `ip addr` mostra só `lo` e `eth0`.
- `nsenter` (Linux) entra nos namespaces de um processo: `sudo nsenter -t <PID> -n ip addr` mostra a rede vista pelo contêiner, mesmo em imagens sem ferramentas.

1.2.4 cgroups e netfilter

- **cgroups** controlam e contabilizam **CPU, memória, I/O de disco, número de processos e acesso a dispositivos** (`/dev`) de cada contêiner. As flags `--memory`, `--cpus`, `--pids-limit`, `--device-read-bps` e `--device` usam esse mecanismo (capítulo 10). O trabalho começou no Google em 2006 e entrou no kernel 2.6.24, em 2008.
- **netfilter** é o subsistema de filtragem de pacotes do kernel, configurado por **iptables** ou **nftables**. O Docker cria regras automaticamente:
 - **MASQUERADE** (NAT de origem): o tráfego que sai dos contêineres (ex.: `172.17.0.0/16`) aparece para fora com o IP do host;
 - **DNAT** na chain `DOCKER`: o tráfego que chega a uma porta publicada (`-p 8080:80`) é redirecionado ao IP e à porta do contêiner;
 - chains de filtro (`DOCKER-USER`, `DOCKER-FORWARD` e outras) isolam redes e permitem regras do administrador (capítulo 6).
- Para ver as regras de NAT no Linux: `sudo iptables -t nat -L -n -v` (ou `sudo nft list ruleset` com o backend `nftables`, experimental no Engine 29).

- **Outros recursos do kernel usados pelo Docker:** capabilities, seccomp, AppArmor/SELinux (capítulo 9) e overlays para as camadas (capítulo 7).

1.2.5 Nativo, máquina virtual e contêiner lado a lado

Execução nativa	Máquinas virtuais	Contêineres
+-----+-----+	+-----+-----+	+-----+-----+
App A App B	App A App B	App A App B
+-----+-----+	Bibliotecas Bibliotecas	Bibliot. Bibliot.
Bibliotecas	SO convid. SO convid.	+-----+-----+
(comuns, com	+ kernel + kernel	Docker Engine
conflitos)	+-----+-----+	(containerd + runc)
+-----+-----+	Hypervisor	+-----+-----+
SO do host	+-----+-----+	SO do host (kernel
+ kernel	SO do host / hardware	compartilhado)
+-----+-----+	+-----+-----+	+-----+-----+
Hardware	Hardware	Hardware
+-----+-----+	+-----+-----+	+-----+-----+

- **Nativo:** as aplicações dividem bibliotecas e versões, então atualizar uma pode quebrar a outra.
- **Máquina virtual:** cada aplicação ganha um **sistema operacional completo**, com kernel e hardware virtualizado. O isolamento é forte, mas cada VM consome memória e disco de um SO inteiro e leva tempo para dar boot.
- **Contêiner:** cada aplicação tem **as próprias bibliotecas**, mas todas usam o **kernel do host**. Não há SO convidado, então cabem muito mais aplicações no mesmo hardware.

1.2.6 Contêiner vs. máquina virtual

Aspecto	Contêiner	Máquina virtual
Isolamento	Processo isolado; kernel compartilhado com o host	Sistema operacional completo sobre um hypervisor; kernel próprio
Tamanho típico	Megabytes	Gigabytes
Inicialização	Milissegundos a segundos (é só iniciar um processo)	Dezenas de segundos a minutos (boot do SO)
Densidade	Centenas por host	Dezenas por host
Fronteira de segurança	Mais fina: uma falha no kernel afeta todos	Mais forte: o hypervisor separa os kernels
Portabilidade	Imagem roda em qualquer host com runtime compatível e mesma arquitetura de CPU	Imagem de disco depende do hypervisor
Uso típico	Empacotar e executar aplicações, microserviços, CI	Isolar sistemas inteiros, SOs diferentes, multi-tenant hostil

- As duas tecnologias se combinam: em nuvem, é comum rodar contêineres **dentro** de VMs. Serviços como AWS Fargate usam microVMs (Firecracker) para isolar cada tarefa.
- Um contêiner Linux **não roda em kernel Windows** nem vice-versa, e uma imagem `amd64` não roda nativamente em `arm64` (precisa de emulação ou de uma imagem multiplataforma, capítulo 5).
- **"Roda em qualquer lugar que tenha Docker"** vale com duas condições: kernel do mesmo tipo (Linux, na quase totalidade dos casos) e arquitetura de CPU compatível. No macOS e no Windows, a imagem Linux roda porque o Docker Desktop fornece a VM Linux. O desempenho de referência é o do Linux nativo; no Desktop, o maior custo extra aparece em bind mounts de diretórios do host.

- **O contêiner não "emula" a aplicação nem o sistema:** a aplicação roda direto no kernel do host, só que isolada. É isso que resolve o "na minha máquina funciona": todas as dependências de espaço de usuário vão na imagem.

1.3 Uma breve história dos contêineres e do Docker

1.3.1 Antes do Docker

Época	Marco	O que trouxe
1979	<code>chroot</code> (Unix V7, depois BSD)	Muda o diretório raiz de um processo. Isola só a visão do sistema de arquivos
2000	FreeBSD Jails	Além do sistema de arquivos, isola processos , rede e usuários
Início dos anos 2000	Virtuozzo (SWsoft, depois Parallels)	Contêineres comerciais para Linux, com painel de gerenciamento
2004–2005	Solaris Zones (Sun)	Contêineres no Solaris, com controle de recursos; só para Solaris
2005	OpenVZ	O núcleo do Virtuozzo como open source. Popularizou os VPS e as empresas de hospedagem, mas exigia um kernel Linux com patch
2006–2008	cgroups (engenheiros do Google)	Limite e contabilização de recursos por grupo de processos. Entrou no kernel 2.6.24 (2008); o Google já usava contêineres em seus datacenters
2008	LXC (Linux Containers)	Combina cgroups, namespaces e <code>chroot</code> no kernel padrão, sem patch . Teve contribuições de Virtuozzo, IBM e Google

1.3.2 O nascimento do Docker

- **2008:** Solomon Hykes funda a **dotCloud**, uma PaaS (Platform as a Service) que aceitava várias linguagens, numa época em que muitas plataformas nasciam presas a uma só (o Heroku, por exemplo, começou só com Ruby). Por baixo, a dotCloud já executava as aplicações em contêineres.
- **Março de 2013:** a dotCloud abre o código do núcleo da plataforma na PyCon, com o nome **Docker**. As primeiras versões eram basicamente um **wrapper do LXC** com um sistema de arquivos em camadas (**AUFS**).
- O crescimento foi muito rápido: em poucos meses, milhares de estrelas no GitHub e centenas de contribuidores. Em **outubro de 2013**, a dotCloud passa a se chamar **Docker, Inc.**
- **Março de 2014 (Docker 0.9):** o Docker troca o LXC pela **libcontainer**, biblioteca própria que fala diretamente com o kernel e que depois deu origem ao **runc**.
- **Junho de 2014 (Docker 1.0):** primeira versão considerada **pronta para produção**, lançada junto com o **Docker Hub**. Empresas como o Spotify já o usavam em escala; AWS e Google passaram a suportá-lo em suas nuvens, e a Red Hat o incorporou ao **OpenShift**.
- Ser **open source** acelerou tudo: qualquer pessoa pode ler o código, relatar e corrigir problemas, e a base de testes é a comunidade inteira.

1.3.3 Do Docker 1.0 aos dias de hoje

Ano	Marco
2015	Criação da OCI (Open Container Initiative); o Docker doa o formato de imagem e o runc
2016	Swarm mode embutido no Engine (Docker 1.12); Docker for Mac e Windows (hoje Docker Desktop)
2017	Numeração por data (17.03), edições CE e EE, projeto Moby e doação do containerd à CNCF
2019	O negócio Docker Enterprise é vendido à Mirantis; a Docker, Inc. foca em ferramentas para desenvolvedores

Ano	Marco
2020–2021	Especificação aberta do Compose; Compose v2 reescrito em Go como plugin (<code>docker compose</code>); nova licença do Docker Desktop para empresas maiores
2022	Kubernetes 1.24 remove o dockershim e passa a usar containerd ou CRI-O diretamente; as imagens continuam compatíveis
2023	Fim do Compose v1; o BuildKit já é o builder padrão desde o Engine 23
2025	Engine 29 com containerd image store padrão; Compose v5 ; Docker Hardened Images gratuitas; Docker Machine arquivado

1.4 Arquitetura do Docker

1.4.1 Componentes

Componente	Papel
Docker CLI (<code>docker</code>)	Cliente. Converte comandos em chamadas à Docker Engine API (REST)
Docker daemon (<code>dockerd</code>)	Servidor. Gerencia imagens, contêineres, redes, volumes e builds; expõe a API em um socket Unix (<code>/var/run/docker.sock</code>), named pipe (Windows) ou TCP com TLS
containerd	Runtime de alto nível (projeto CNCF). Gerencia o ciclo de vida dos contêineres, o download e o armazenamento de imagens (no Engine 29, também é o image store padrão)
containerd-shim	Processo que fica entre o containerd e cada contêiner; permite reiniciar o daemon sem matar os contêineres (com <code>live-restore</code>) e guarda o código de saída
runc	Runtime de baixo nível (referência da especificação OCI). Cria os namespaces e cgroups e executa o processo; sai logo depois
BuildKit	Motor de build padrão desde o Engine 23; executa Dockerfiles em paralelo, com cache avançado e segredos (capítulo 5)
Plugins da CLI	<code>docker compose</code> , <code>docker buildx</code> , <code>docker scout</code> , <code>docker init</code> , <code>docker debug</code> etc. são plugins instalados em <code>~/ .docker/cli-plugins</code> ou no diretório do sistema

- **Fluxo de um** `docker run`: CLI → API do `dockerd` → (baixa a imagem se faltar) → containerd → shim → runc → processo do contêiner. Depois que o runc sai, o shim continua como pai do processo.
- **Moby** é o projeto open source de onde vem o Docker Engine. **Docker Desktop** é o produto comercial para estações de trabalho (Engine, CLI, Compose, Buildx, Kubernetes opcional, interface gráfica e VM).

1.4.2 Padrões OCI

A **Open Container Initiative** mantém três especificações. É por causa delas que imagens criadas com Docker rodam em containerd, Podman, CRI-O e Kubernetes.

Especificação	Define
Image spec	Formato da imagem: manifest, config e camadas (tarballs)
Runtime spec	Como um runtime (runc, crun, gVisor, Kata) executa um bundle de sistema de arquivos + configuração
Distribution spec	API HTTP para enviar e baixar imagens e artefatos de um registry

1.4.3 Objetos do Docker

Objeto	O que é	Comando principal
--------	---------	-------------------

Objeto	O que é	Comando principal
Imagem	Modelo somente leitura, em camadas, com o sistema de arquivos e os metadados (comando padrão, variáveis, usuário)	<code>docker image</code>
Contêiner	Instância em execução (ou parada) de uma imagem, com uma camada gravável própria	<code>docker container</code>
Volume	Área de dados persistente gerenciada pelo Docker, fora do ciclo de vida do contêiner	<code>docker volume</code>
Rede	Rede virtual que conecta contêineres entre si e ao mundo externo	<code>docker network</code>
Registry	Serviço que armazena e distribui imagens (Docker Hub, GHCR, ECR, Harbor)	<code>docker login</code> , <code>push</code> , <code>pull</code>
Contexto	Aponta a CLI para um daemon (local, remoto via SSH, Docker Desktop)	<code>docker context</code>

1.5 Instalação e configuração do daemon

1.5.1 Requisitos

- **O daemon roda nativamente só no Linux.** Em outros sistemas, a instalação sobe uma **VM Linux** e roda o daemon nela (é o que o Docker Desktop faz). O **cliente** (`docker`) existe para Linux, macOS e Windows e pode apontar para qualquer daemon.
- **Arquiteturas de 64 bits:** os pacotes oficiais cobrem `x86_64` / `amd64` e `arm64`, e, conforme a distribuição, `armhf` (`arm/v7`), `ppc64le` e `s390x`. Os pacotes oficiais de **32 bits para Raspbian** foram removidos no Engine 29. Os contêineres usam o mesmo kernel do host, então herdam a arquitetura dele.
- **Distribuição suportada:** em vez de um número mínimo de kernel, a documentação lista as **versões de cada distribuição** suportadas (em set/2026, por exemplo, Debian 13 "Trixie" e 12 "Bookworm"). Na prática, qualquer distribuição mantida tem kernel com namespaces, cgroups (de preferência **cgroup v2**) e **overlayfs**.
- **Conferir o host:** `uname -r` (versão do kernel), `uname -m` (arquitetura) e o script `check-config.sh` do repositório moby, que verifica as opções do kernel necessárias. Depois de instalar, `docker info` mostra o driver de cgroup, a versão do cgroup, o storage e avisos.

1.5.2 Formas de instalar

Ambiente	Opção recomendada	Observações
Linux servidor	Pacotes oficiais <code>docker-ce</code> , <code>docker-ce-cli</code> , <code>containerd.io</code> , <code>docker-buildx-plugin</code> , <code>docker-compose-plugin</code> do repositório <code>download.docker.com</code>	Os pacotes <code>docker.io</code> das distribuições costumam estar atrasados. O script <code>get.docker.com</code> é prático, mas não é recomendado para produção
macOS / Windows	Docker Desktop	Roda uma VM Linux; no Windows usa WSL 2. Licença paga para empresas maiores (abaixo)
Linux desktop	Docker Desktop ou Engine	Docker Desktop no Linux também usa uma VM, separada do Engine do host
CI	Engine já instalado no runner, ou serviço <code>docker:dind</code>	Veja "Docker-in-Docker" no capítulo 12

- **Licença do Docker Desktop:** é gratuito para uso pessoal, educação, projetos open source não comerciais e empresas com **menos de 250 funcionários E menos de US\$ 10 milhões de receita anual**. Acima disso, e para órgãos de governo, exige assinatura Pro, Team ou Business. O **Docker Engine** (Moby) é open source (Apache 2.0) e não tem essa restrição.
- **Verificar a instalação:** `docker version` (versões do cliente e do servidor, e da API), `docker info` (storage, cgroup, runtimes, registries, avisos) e `docker run --rm hello-world`.

- **Script de conveniência:** `curl -fsSL https://get.docker.com | sh` detecta a distribuição, configura o repositório oficial e instala a versão mais recente. A própria Docker o recomenda **só para teste e desenvolvimento**: ele não deixa escolher a versão e instala tudo sem confirmação. Em produção, use o repositório (abaixo) com versão controlada.

1.5.3 Instalação pelo repositório oficial (Debian e Ubuntu)

Procedimento atual para Debian (no Ubuntu, troque `debian` por `ubuntu` nas URLs). Remova antes os pacotes não oficiais que conflitam: `docker.io`, `docker-compose`, `docker-doc`, `podman-docker`, `containerd` e `runc`.

```
# 1. Chave GPG do repositório em /etc/apt/keyrings
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# 2. Repositório no formato deb822 (.sources)
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

# 3. Instalação do Engine, da CLI e dos plugins
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

- **Versão específica:** `apt list --all-versions docker-ce` e depois `sudo apt install docker-ce=<versão> docker-ce-cli=<versão> ...`. Em servidores, fixe a versão para atualizar de forma planejada.
- **Fedora, RHEL e derivados:** mesmo princípio, com `dnf` e o repositório `download.docker.com/linux/<distro>/docker-ce.repo`.
- **Materiais antigos** mostram `apt-key adv --keyserver ...` e o repositório `apt.dockerproject.org`. Os dois estão **obsoletos**: o `apt-key` foi descontinuado no Debian e no Ubuntu, e o repositório antigo foi desativado em favor do `download.docker.com`.

1.5.4 Pós-instalação no Linux

- **Serviço:** em distribuições atuais, o Docker é gerenciado pelo **systemd**: `sudo systemctl status docker`, `sudo systemctl start docker`, `sudo systemctl restart docker`. Os comandos antigos `service docker ...` e `/etc/init.d/docker ...` são de sistemas com SysV/Upstart. Logs do daemon: `journalctl -u docker.service`.
- **Por que sudo:** por padrão, o daemon escuta em um **socket Unix** (`/var/run/docker.sock`) que pertence ao **root** e ao grupo `docker`, e não em uma porta TCP.
- **Grupo docker:** adicionar um usuário ao grupo permite usar a CLI sem `sudo`, mas **equivale a dar root no host** (quem controla o daemon pode montar `/` em um contêiner). Em servidores compartilhados, prefira o **modo rootless** (capítulo 9).

```

sudo groupadd docker          # normalmente o pacote já cria o grupo
sudo usermod -aG docker "$USER" # adiciona o seu usuário
newgrp docker                 # ou saia e entre de novo na sessão
docker run --rm hello-world   # agora sem sudo

```

- **Iniciar no boot:** `systemctl enable --now docker.service containerd.service`.
- **Dados do daemon:** ficam em `/var/lib/docker` (e em `/var/lib/containerd` com o containerd image store). Mude com `data-root` no `daemon.json`.

1.5.5 O arquivo `daemon.json`

Configuração do daemon em `/etc/docker/daemon.json` (Linux) ou nas configurações do Docker Desktop. Opções mais usadas:

```

{
  "log-driver": "local",
  "log-opts": { "max-size": "20m", "max-file": "5" },
  "live-restore": true,
  "default-address-pools": [{ "base": "10.200.0.0/16", "size": 24 }],
  "registry-mirrors": ["https://mirror.exemplo.com"],
  "features": { "containerd-snapshotter": true },
  "userns-remap": "default"
}

```

Opção	Para que serve
<code>log-driver / log-opts</code>	Driver de log padrão e rotação (capítulo 10)
<code>live-restore</code>	Mantém os contêineres rodando enquanto o daemon é reiniciado ou atualizado (não funciona com Swarm)
<code>default-address-pools</code>	Faixas de IP usadas nas redes criadas pelo Docker; evita conflito com a rede corporativa ou a VPN
<code>registry-mirrors</code>	Espelho (pull-through cache) para o Docker Hub
<code>insecure-registries</code>	Registries sem TLS válido. Evite fora de laboratório
<code>features.containerd-snapshotter</code>	Liga ou desliga o containerd image store (padrão ligado em instalações novas do Engine 29)
<code>userns-remap</code>	Mapeia o root do contêiner para um usuário sem privilégio no host
<code>data-root</code>	Diretório de dados do daemon
<code>dns</code>	Servidores DNS padrão para os contêineres

- Depois de alterar: `sudo systemctl restart docker` (ou `reload` para algumas opções, como `log-driver` de novos contêineres e `insecure-registries`). Uma opção definida ao mesmo tempo no `daemon.json` e como flag no `dockerd` impede o daemon de subir.

1.5.6 O daemon e suas opções

- `dockerd` é o binário do daemon, separado da CLI desde o Docker 1.12 (antes era `docker -d` e depois `docker daemon`). Ele roda em segundo plano como serviço e é o processo que controla imagens, contêineres, redes e volumes.
- As opções podem ir como **flags** (`dockerd --log-level debug`) ou, de preferência, no `daemon.json`, que é o lugar persistente e versionável. `dockerd --help` lista todas; `dockerd --validate` checa o arquivo de configuração.

Opção (<code>docker --help</code> / flag)	Para que serve
<code>bip</code> / <code>--bip</code> <code>192.168.100.1/24</code>	IP e sub-rede da bridge <code>docker0</code> (padrão <code>172.17.0.1/16</code>)
<code>fixed-cidr</code>	Restringe a faixa de IPs que os contêineres recebem dentro da sub-rede da <code>docker0</code>
<code>default-address-pools</code>	Faixas usadas nas redes criadas pelo usuário e pelo Compose
<code>default-gateway</code>	Gateway padrão dos contêineres na bridge <code>docker0</code>
<code>hosts</code> / <code>-H</code>	Onde a API escuta: <code>unix:///var/run/docker.sock</code> (padrão), <code>tcp://IP:2376</code> (com TLS), <code>ssh</code> (lado do cliente) ou <code>fd://</code> (socket activation do systemd : o <code>systemd</code> abre o socket e o entrega ao daemon)
<code>tls</code> , <code>tlsverify</code> , <code>tlscacert</code> , <code>tlscert</code> , <code>tlskey</code>	TLS mútuo para a API em TCP
<code>dns</code> , <code>dns-search</code>	DNS e domínios de busca padrão dos contêineres
<code>ip-forward</code>	Liga o roteamento IP no host (padrão <code>true</code>); necessário para os contêineres saírem para a rede
<code>iptables</code> / <code>ip6tables</code>	Permite ao Docker gerenciar o firewall (padrão <code>true</code>). Desligar exige recriar à mão todo o NAT
<code>icc</code>	Comunicação entre contêineres na bridge <code>docker0</code> (padrão <code>true</code>). Em redes definidas pelo usuário, o equivalente é a opção <code>com.docker.network.bridge.enable_icc</code>
<code>default-ulimits</code>	<code>ulimit</code> padrão dos contêineres; <code>--ulimit</code> no <code>docker run</code> sobrepõe
<code>log-level</code>	Verbosidade do log do próprio daemon (<code>debug</code> , <code>info</code> , <code>warn</code> , <code>error</code>); <code>debug: true</code> liga o modo depuração
<code>storage-driver</code> , <code>storage-opts</code>	Driver e opções de armazenamento (as antigas <code>dm.*</code> do <code>devicemapper</code> não existem mais; com <code>overlay2</code> sobre XFS com <code>pquota</code> , <code>overlay2.size</code> limita a camada gravável)

- **Conflito com o systemd:** em muitas distribuições, a `unit` do serviço já passa `-H fd://`. Definir `hosts` também no `daemon.json` impede o daemon de subir (a mesma opção em dois lugares). Para mudar os endpoints, crie um `override` (`systemctl edit docker`) que redefine o `ExecStart` sem o `-H`.
- **Depurar o daemon:** `journalctl -u docker -f` mostra os logs; `"debug": true` e `sudo kill -SIGHUP $(pidof dockerd)` recarregam algumas opções sem reiniciar.

1.5.7 Contextos e acesso remoto

- `docker context` guarda endpoints de daemons: `docker context create producao --docker "host=ssh://deploy@servidor"`, depois `docker context use producao` ou `docker --context producao ps`.
- `DOCKER_HOST` (variável de ambiente) sobrepõe o contexto atual, por exemplo `DOCKER_HOST=ssh://deploy@servidor`.
- **SSH** é a forma mais simples e segura de acessar um daemon remoto. A alternativa é TCP com **TLS mútuo** na porta **2376**. Expor a API na porta **2375** sem TLS equivale a dar root no host a qualquer um na rede; o Docker descontinuou esse modo (obsoleto desde o Engine 26).
- **Docker Machine** (`docker-machine`), que criava hosts Docker em VMs locais ou na nuvem, chegou ao fim da vida e teve o repositório arquivado em julho de 2025, assim como o antigo Docker Toolbox. Hoje, crie o host com a ferramenta de infraestrutura do seu ambiente (Terraform, cloud-init, console da nuvem), instale o Engine pelo repositório oficial e acesse com `docker context` via SSH. Nas estações de trabalho, o Docker Desktop substituiu os dois.

1.5.8 Docker Machine: o que era e o que usar hoje

Materiais de 2015 a 2019 dedicam um capítulo ao **Docker Machine**, ferramenta que, com um comando, criava uma VM (VirtualBox, VMware, Hyper-V) ou uma instância de nuvem (AWS, GCP, Azure, DigitalOcean), instalava o Docker (em VMs locais, com a distribuição mínima **boot2docker**), gerava certificados TLS e configurava a CLI local para usar

esse daemon remoto na porta 2376. Ela chegou ao fim da vida e teve o repositório arquivado em julho de 2025. Vale entender o que cada comando fazia para reconhecer o equivalente atual:

Docker Machine (descontinuado)	O que fazia	Equivalente atual
<code>docker-machine create --driver virtualbox dev</code>	Criava uma VM local com Docker	Docker Desktop, Colima, OrbStack ou Rancher Desktop; ou <code>multipass</code> / Lima + instalação do Engine
<code>docker-machine create --driver amazonec2 prod</code>	Criava uma instância na nuvem com Docker	Terraform/OpenTofu, CLI da nuvem ou console, com cloud-init instalando o Engine pelo repositório oficial
<code>docker-machine env prod + eval "\$(...)"</code>	Exportava <code>DOCKER_HOST</code> , <code>DOCKER_TLS_VERIFY</code> , <code>DOCKER_CERT_PATH</code>	<code>docker context create prod --docker host=ssh://usuario@ip</code> e <code>docker context use prod</code>
<code>docker-machine ls</code>	Listava os hosts e a URL do daemon	<code>docker context ls</code> (e o inventário da nuvem)
<code>docker-machine ssh prod</code>	Abria SSH no host	<code>ssh usuario@ip</code>
<code>docker-machine ip prod</code>	Mostrava o IP	CLI da nuvem, saída do Terraform
<code>docker-machine stop/start/rm prod</code>	Controlava a VM	Hypervisor, CLI da nuvem, <code>terraform destroy</code>
<code>docker-machine inspect prod</code>	Mostrava driver, certificados e opções do Engine	<code>docker context inspect prod</code> e <code>docker info</code> no host

- As variáveis `DOCKER_HOST`, `DOCKER_TLS_VERIFY` e `DOCKER_CERT_PATH` continuam valendo na CLI. Contextos são preferíveis porque ficam salvos com nome e não dependem da sessão do shell.
- **boot2docker** e o **Docker Toolbox** (que empacotava Machine + VirtualBox para Mac e Windows) também foram descontinuados.

1.5.9 Números de fundamentos

Item	Valor
Socket padrão da API (Linux)	<code>/var/run/docker.sock</code>
Porta da API sem TLS (não usar)	2375/tcp
Porta da API com TLS	2376/tcp
Diretório de dados padrão	<code>/var/lib/docker</code> (e <code>/var/lib/containerd</code>)
Arquivo de configuração do daemon	<code>/etc/docker/daemon.json</code>
Configuração da CLI (credenciais, contexto atual)	<code>~/.docker/config.json</code>
API mínima aceita pelo Engine 29	v1.44 (Docker 25+)
Suporte a cgroup v1	Obsoleto no Engine 29; mantido até pelo menos maio de 2029
Docker Desktop gratuito	Empresas com < 250 funcionários e < US\$ 10 mi de receita, uso pessoal, educação, OSS não comercial

1.6 Decisão rápida — Fundamentos

Situação	Abordagem recomendada	Erro comum
Isolar aplicações que confiam umas nas outras no mesmo host	Contêineres	Uma VM por aplicação

Situação	Abordagem recomendada	Erro comum
Isolar cargas de clientes diferentes e não confiáveis	VM (ou runtime com sandbox, como gVisor/Kata) por tenant	Confiar só no isolamento de namespaces
Usar Docker em um Mac ou Windows de trabalho	Docker Desktop (verificando a licença) ou alternativa com VM Linux	Esperar contêineres Linux sem VM
Operar um daemon em outro servidor	<code>docker context</code> com SSH	Abrir a porta 2375
Criar um host Docker na nuvem	Terraform/cloud-init + repositório oficial + <code>docker context</code>	Docker Machine (descontinuado)
Redes do Docker precisam de outra faixa de IP	bip (bridge padrão) e <code>default-address-pools</code> (redes novas) no <code>daemon.json</code>	Editar regras de iptables à mão
Permitir que o time rode <code>docker</code> sem <code>sudo</code> em um servidor compartilhado	Modo rootless por usuário	Colocar todos no grupo <code>docker</code>
Atualizar o Engine sem derrubar os contêineres	<code>live-restore: true</code>	Reiniciar no horário de pico e torcer
Redes do Docker conflitando com a VPN	<code>default-address-pools</code> no <code>daemon.json</code>	Apagar e recriar redes até funcionar
Descobrir versão da API, storage e cgroup em uso	<code>docker info</code> e <code>docker version</code>	Ler só o <code>docker --version</code> (mostra apenas o cliente)

2. Ciclo de Vida de Contêineres

Ideia central: um contêiner vive enquanto o seu **processo principal (PID 1)** vive. Quando o PID 1 termina, o contêiner para, e o **código de saída** dele vira o código de saída do contêiner. Quase todo problema de "contêiner que não fica de pé" ou "que demora a parar" se explica pelo PID 1.

2.1 Primeiros passos

2.1.1 O que acontece em um `docker run hello-world`

A imagem `hello-world` existe para testar a instalação. Ela imprime uma mensagem e termina.

```
docker container run hello-world    # forma agrupada
docker run hello-world              # forma curta, equivalente
```

1. A **CLI** envia o pedido à API do **daemon**.
 2. O daemon procura a imagem `hello-world:latest` **no host**. Como ela não existe, baixa do **Docker Hub** (`Unable to find image ... locally` e depois `Pulling from library/hello-world`), escolhendo a variante da arquitetura do host.
 3. O daemon **cria um contêiner** a partir da imagem e o inicia; o processo imprime a mensagem.
 4. O daemon envia a saída à CLI, que a mostra no terminal. O processo termina, e o contêiner fica no estado `exited` com código 0.
- Desde o Docker 1.13, os comandos são agrupados por objeto (`docker container ...`, `docker image ...`, `docker volume ...`, `docker network ...`). As formas curtas antigas (`docker run`, `docker ps`, `docker rmi`) continuam válidas e equivalentes.

2.1.2 Listar imagens e contêineres

Coluna de <code>docker image ls</code>	Significado
REPOSITORY	Nome da imagem (com namespace e registry, se houver)
TAG	Rótulo da versão (<code>latest</code> se nada foi indicado)
IMAGE ID	Início do digest do config da imagem
CREATED	Há quanto tempo a imagem foi construída (não baixada)
SIZE	Tamanho descompactado das camadas

Coluna de <code>docker container ls</code> (Docker)	Significado
CONTAINER ID	Identificador curto (12 caracteres) do contêiner. Qualquer prefixo único serve nos comandos
IMAGE	Imagem usada para criar o contêiner
COMMAND	Comando em execução (<code>ENTRYPOINT</code> + <code>CMD</code>)
CREATED	Quando o contêiner foi criado

Coluna de <code>docker container ls</code> (ícone)	Significado
STATUS	Up 5 minutes, Up ... (healthy), Up ... (Paused), Exited (0) 3 seconds ago, Created, Restarting
PORTS	Portas publicadas (0.0.0.0:8080->80/tcp)
NAMES	Nome dado com <code>--name</code> ou gerado automaticamente

- `docker container ls` mostra só os contêineres **em execução**. Com `-a`, mostra também os parados e os criados e nunca iniciados. `-q` imprime só os IDs (útil em scripts, como `docker rm $(docker ps -aq --filter status=exited)`), e `--format` escolhe as colunas.
- Nos comandos, o contêiner pode ser indicado pelo **nome** ou pelo **ID** (ou por um prefixo único do ID). O nome é mais legível e estável.

2.1.3 Modo interativo e modo em segundo plano

Flag	Efeito
<code>-i / --interactive</code>	Mantém o STDIN aberto , mesmo sem terminal anexado
<code>-t / --tty</code>	Aloca um pseudoterminal (TTY): prompt, cores, edição de linha
<code>-d / --detach</code>	Executa em segundo plano e devolve o terminal, imprimindo o ID do contêiner

- Interativo (`-it`):** para explorar uma imagem, depurar ou executar ferramentas de linha de comando: `docker run -it ubuntu:24.04 bash`. O prompt muda para `root@<id>:/#` porque você está **dentro** do contêiner (`cat /etc/os-release` confirma a distribuição).
- Segundo plano (`-d`):** para serviços que você só vai consumir (um servidor web, um banco): `docker run -d -p 8080:80 nginx`. A aplicação já está configurada na imagem; ninguém precisa entrar no contêiner.
- Sair sem parar:** em uma sessão `-it`, o shell é o **PID 1**. Digitar `exit` encerra o shell e, portanto, **para o contêiner**. Para se desanexar e deixá-lo rodando, use **Ctrl+P seguido de Ctrl+Q** (a sequência pode ser trocada com `--detach-keys`).
- Voltar ao terminal:** `docker container attach <contêiner>` reconecta ao STDIN/STDOUT do **PID 1**. Cuidado: Ctrl+C ou `exit` ali afetam o processo principal. Para abrir um **shell adicional** sem esse risco, prefira `docker exec -it <contêiner> bash` (ou `sh`); ao sair dele, o contêiner continua rodando.

2.1.4 Criar, iniciar, parar, pausar e remover

```

docker container create -it --name meu-ubuntu ubuntu:24.04    # só cria (status Created)
docker container ls -a                                         # aparece só com -a
docker container start -ai meu-ubuntu                          # inicia e anexa (-a) com STDIN (-i)

docker container stop meu-ubuntu                               # SIGTERM, espera 10 s, depois SIGKILL
docker container start meu-ubuntu                              # inicia de novo o mesmo contêiner
docker container restart meu-ubuntu                           # stop + start
docker container pause meu-ubuntu                             # congela os processos: STATUS "Up ... (Paused)"
docker container unpause meu-ubuntu                           # descongela os processos

docker container rm meu-ubuntu                                 # falha se estiver rodando
docker container rm -f meu-ubuntu                             # força: mata (SIGKILL) e remove

```

- `create` + `start` é útil para preparar um contêiner (por exemplo, copiar arquivos com `docker cp`) antes de iniciá-lo.
- `docker container rm` **de um contêiner em execução** retorna erro (`cannot remove container ... : container is running: stop the container before removing or force remove`). Pare antes ou use `-f`, sabendo que `-f` **não** faz parada graciosa.
- **Remover o contêiner não remove a imagem**: ela continua no host para novos contêineres. Para apagá-la, use `docker image rm`.
- `docker rm -v` remove também os **volumes anônimos** do contêiner.

2.2 Executar contêineres

2.2.1 `docker run` em detalhe

`docker run` = `docker create` (cria o contêiner a partir da imagem) + `docker start` (inicia o processo). Se a imagem não existir localmente, é baixada antes.

```
docker run -d --name api \
  -p 8080:3000 \
  -e NODE_ENV=production \
  -v dados-api:/app/data \
  --network backend \
  --restart unless-stopped \
  --memory 512m --cpus 1 \
  minha-org/api:1.4.2
```

Flag	Efeito
<code>-d</code> / <code>--detach</code>	Roda em segundo plano e imprime o ID
<code>-it</code>	<code>-i</code> mantém o STDIN aberto, <code>-t</code> aloca um terminal. Juntas, para sessões interativas (<code>docker run -it ubuntu bash</code>)
<code>--rm</code>	Remove o contêiner (e seus volumes anônimos) quando ele sair
<code>--name</code>	Nome fixo; sem ele, o Docker gera um aleatório (<code>quirky_turing</code>)
<code>-p host:contêiner</code>	Publica uma porta (capítulo 6)
<code>-e</code> / <code>--env-file</code>	Variáveis de ambiente
<code>-v</code> / <code>--mount</code>	Volumes, bind mounts e tmpfs (capítulo 7)
<code>--network</code>	Rede à qual o contêiner se conecta
<code>--restart</code>	Política de reinício (abaixo)
<code>-w</code> / <code>--workdir</code>	Diretório de trabalho
<code>-u</code> / <code>--user</code>	Usuário e grupo do processo (<code>-u 1000:1000</code>)
<code>--entrypoint</code>	Substitui o <code>ENTRYPOINT</code> da imagem
<code>--init</code>	Usa um init mínimo (<code>tini</code>) como PID 1, que repassa sinais e recolhe processos zumbis
<code>--memory</code> , <code>--cpus</code> , <code>--pids-limit</code>	Limites de recursos (capítulo 10)
<code>--read-only</code> , <code>--cap-drop</code> , <code>--security-opt</code>	Endurecimento (capítulo 9)
<code>--platform</code>	Força a plataforma da imagem (<code>linux/amd64</code> , <code>linux/arm64</code>)

Flag	Efeito
<code>--pull always missing never</code>	Quando baixar a imagem (padrão <code>missing</code>)

- **Argumentos depois da imagem** substituem o `CMD`: `docker run alpine echo oi`. Para substituir o `ENTRYPOINT`, use `--entrypoint`.
- `docker run` **sempre cria um contêiner novo**. Para voltar a um contêiner parado, use `docker start`.

2.2.2 Estados do contêiner

Estado	Significado	Como chega lá
<code>created</code>	Contêiner criado, processo nunca iniciado	<code>docker create</code>
<code>running</code>	PID 1 em execução	<code>docker start</code> , <code>docker run</code>
<code>paused</code>	Processos congelados (cgroup freezer); memória preservada	<code>docker pause</code> / <code>unpause</code>
<code>restarting</code>	Saiu e a política de reinício está agindo	Falha com <code>--restart</code>
<code>exited</code>	PID 1 terminou; a camada gravável continua até o <code>docker rm</code>	Fim do processo, <code>docker stop</code> , <code>docker kill</code>
<code>removing</code>	Sendo removido	<code>docker rm</code>
<code>dead</code>	Remoção falhou parcialmente (em geral, recurso ocupado)	Raro; exige investigação

- **Contêiner parado não perde dados** da camada gravável; só o `docker rm` os apaga. Mesmo assim, dados importantes devem ficar em volumes.

2.2.3 Políticas de reinício

Política	Reinicia quando	Depois de <code>docker stop</code> manual	Depois de reiniciar o daemon
<code>no</code> (padrão)	Nunca	—	Não sobe
<code>on-failure[:N]</code>	O código de saída é diferente de 0 (até N tentativas)	Não reinicia	Sobe se tiver saído com erro
<code>always</code>	Sempre que sair	Não reinicia na hora, mas volta a subir quando o daemon reiniciar	Sobe
<code>unless-stopped</code>	Sempre que sair	Não reinicia	Não sobe se tinha sido parado manualmente

- A política só passa a valer depois que o contêiner **fica de pé por pelo menos 10 segundos**, para evitar laços de reinício de contêineres que nunca iniciam.
- O intervalo entre tentativas começa em **100 ms** e **dobra** a cada falha (backoff), sendo zerado quando o contêiner fica estável.
- A política de reinício **não** reage a um healthcheck `unhealthy` no Engine puro. Quem troca contêineres doentes é um orquestrador (Swarm, Kubernetes) ou uma ferramenta externa.

2.3 Sinais, PID 1 e parada graciosa

2.3.1 Como o Docker para um contêiner

1. `docker stop` envia o **sinal de parada** (padrão `SIGTERM`, ou o definido em `STOP_SIGNAL`) ao **PID 1**.

- Espera o **tempo de tolerância: 10 segundos** por padrão (`docker stop -t 30`, `--stop-timeout` no `run`, `stop_grace_period` no Compose).
 - Se o processo não sair, envia `SIGKILL`, que não pode ser tratado. O contêiner sai com código **137**.
- `docker kill` envia `SIGKILL` direto (ou outro sinal com `-s`, como `docker kill -s HUP` para recarregar configuração).

2.3.2 O problema do PID 1

- O kernel trata o PID 1 de um namespace de forma especial: **sinais sem tratador explícito são ignorados**. Uma aplicação que não trata `SIGTERM` como PID 1 não para com `docker stop` e só morre no `SIGKILL`, depois de 10 segundos.
- Forma shell** no Dockerfile (CMD `node server.js`) roda `/bin/sh -c "node server.js"`: o PID 1 é o **shell**, que **não repassa** o `SIGTERM` ao Node. Use a **forma exec** (CMD `["node", "server.js"]`), veja o capítulo 4.
- Scripts de entrada** (`entrypoint.sh`) devem terminar com `exec "$@"`, para que a aplicação substitua o shell e vire o PID 1.
- Processos zumbis**: o PID 1 também precisa recolher filhos que terminam. Aplicações que criam subprocessos devem rodar com `--init` (ou `init: true` no Compose), que usa o `tini` como PID 1.

2.3.3 Códigos de saída

Código	Significado	Causa típica
0	Sucesso	Processo terminou normalmente (esperado em jobs; em serviços, indica que o processo não ficou em primeiro plano)
1	Erro genérico da aplicação	Exceção não tratada, configuração inválida
125	O próprio <code>docker run</code> falhou	Flag inválida, conflito de nome, porta já em uso
126	Comando encontrado mas não executável	Falta permissão de execução no script, arquivo não é binário válido
127	Comando não encontrado	Erro de digitação, binário ausente na imagem, <code>PATH</code> errado, script com final de linha CRLF (<code>/bin/sh^M</code>)
137	Morto por <code>SIGKILL</code> (128 + 9)	OOM kill (confira <code>OOMKilled: true</code> no <code>inspect</code>), <code>docker kill</code> ou <code>docker stop</code> depois do tempo de tolerância
139	Falha de segmentação (128 + 11)	Bug nativo, biblioteca incompatível (ex.: <code>musl</code> vs. <code>glibc</code>), arquitetura errada
143	Terminou por <code>SIGTERM</code> (128 + 15)	Parada normal com <code>docker stop</code> em uma aplicação que não trata o sinal com código 0

- Descubra o código e a causa: `docker inspect -f '{{.State.ExitCode}} {{.State.OOMKilled}} {{.State.Error}}' <contêiner>`.

2.4 Inspeção, depuração e limpeza

2.4.1 Comandos de observação

Comando	Para que serve
<code>docker ps</code> / <code>docker ps -a</code>	Contêineres em execução / todos. <code>--filter status=exited</code> , <code>--format</code>
<code>docker logs -f --tail 100 --since 10m <c></code>	STDOUT e STDERR do PID 1. Só funciona com drivers de log que guardam localmente

Comando	Para que serve
<code>docker inspect <c></code>	JSON completo: estado, IP, montagens, variáveis, política de reinício. Use <code>-f</code> com template Go ou <code>--format json jq</code>
<code>docker exec -it <c> sh</code>	Abre um processo novo dentro de um contêiner em execução
<code>docker top <c></code>	Processos do contêiner vistos do host
<code>docker stats</code>	CPU, memória, rede e disco em tempo real
<code>docker events</code>	Fluxo de eventos do daemon (create, start, die, oom, health_status)
<code>docker diff <c></code>	Arquivos adicionados (A), alterados (C) ou apagados (D) na camada gravável
<code>docker cp <c>:/caminho ./local</code>	Copia arquivos entre o contêiner e o host (funciona com o contêiner parado)
<code>docker port <c></code>	Mapeamentos de porta publicados
<code>docker debug <c></code>	Abre um shell com ferramentas de depuração mesmo em imagens mínimas, sem alterar a imagem (recurso do Docker Desktop)

- **Imagem mínima sem shell** (distroless, `scratch`): `docker exec` não tem o que executar. Alternativas: `docker debug`, ou um contêiner auxiliar que compartilha os namespaces do alvo: `docker run -it --rm --pid container:api --network container:api nicolaka/netshoot`.
- **Contêiner que sai logo ao iniciar**: `docker logs` mostra a última saída; para investigar, sobrescreva o comando: `docker run -it --entrypoint sh <imagem>`.

2.4.2 Limpeza

Comando	Remove
<code>docker container prune</code>	Contêineres parados
<code>docker image prune</code>	Imagens dangling (sem tag, <code><none>:<none></code>)
<code>docker image prune -a</code>	Todas as imagens não usadas por nenhum contêiner
<code>docker volume prune</code>	Volumes anônimos não usados (desde o Engine 23). Use <code>-a</code> para incluir volumes nomeados
<code>docker network prune</code>	Redes personalizadas sem contêineres
<code>docker builder prune</code>	Cache de build
<code>docker system prune</code>	Contêineres parados, redes não usadas, imagens dangling e cache de build. Não remove volumes
<code>docker system prune -a --volumes</code>	Tudo acima + imagens não usadas + volumes anônimos não usados
<code>docker system df / df -v</code>	Mostra quanto espaço cada tipo de objeto usa e quanto pode ser liberado

- Todos aceitam `--filter "until=24h"` e `--filter label=...`, úteis em jobs de limpeza de runners de CI.

2.5 Decisão rápida — Ciclo de vida

Situação	Abordagem recomendada	Erro comum
Contêiner leva 10 s para parar	Forma exec no <code>CMD / ENTRYPOINT</code> , exec <code>"\$@"</code> no script, tratar <code>SIGTERM</code> ou <code>--init</code>	Aumentar o timeout de parada
Contêiner sai com 137	Verificar <code>OOMKilled</code> no <code>inspect</code> ; ajustar limite de memória ou vazamento	Tratar como erro da aplicação
Contêiner sai com 127	Conferir binário, <code>PATH</code> e finais de linha (CRLF) do script	Reinstalar o Docker

Situação	Abordagem recomendada	Erro comum
Contêiner de serviço sai com 0 logo após iniciar	O processo foi para segundo plano (daemonizou); rodar em primeiro plano (<code>nginx -g 'daemon off;'</code>)	<code>tail -f /dev/null</code> no fim do script
Serviço deve voltar após reboot, mas respeitar uma parada manual	<code>--restart unless-stopped</code>	<code>--restart always</code>
Job que deve tentar de novo só se falhar	<code>--restart on-failure:3</code>	<code>always</code>
Investigar imagem distroless	<code>docker debug</code> ou contêiner auxiliar com <code>--pid / --network container:</code>	Trocar a base para ter shell
Ver o que o contêiner mudou no disco	<code>docker diff</code>	<code>docker exec + find</code>
Liberar espaço no runner de CI	<code>docker system prune -af --filter until=24h</code>	<code>rm -rf /var/lib/docker</code> com o daemon rodando
Sair de uma sessão <code>-it</code> sem parar o contêiner	Ctrl+P Ctrl+Q; depois <code>docker exec -it</code> para voltar	<code>exit</code> (para o PID 1)
Rodar comando único e descartar	<code>docker run --rm</code>	Acumular contêineres <code>exited</code>

3. Imagens e Registries

Ideia central: uma imagem é uma **lista ordenada de camadas** somente leitura mais um **arquivo de configuração**, descritos por um **manifest**. Tudo é endereçado por conteúdo (hash **SHA-256**): a **tag** é um rótulo móvel, o **digest** é imutável.

3.1 Anatomia de uma imagem

3.1.1 Camadas, manifest e digest

Peça	O que é
Camada (layer)	Tarball com as mudanças de sistema de arquivos de uma instrução do build (RUN, COPY, ADD). Camadas iguais são armazenadas e baixadas uma vez só
Config	JSON com metadados: Cmd, Entrypoint, Env, User, WorkingDir, ExposedPorts, Labels, histórico e a lista de diff_ids
Manifest	JSON que lista o config e as camadas (com seus digests e tamanhos) de uma plataforma
Index (manifest list)	JSON que aponta para vários manifests, um por plataforma (linux/amd64, linux/arm64 ...). O cliente escolhe o da sua plataforma
Digest	Hash SHA-256 do manifest (ou do index): sha256:3f5a... Qualquer mudança gera outro digest

- **Copy-on-write:** quando um contêiner altera um arquivo de uma camada inferior, o arquivo é copiado para a camada gravável do contêiner antes da mudança. Apagar um arquivo em uma camada superior **não reduz** o tamanho da imagem: ele continua na camada onde foi criado (capítulo 4).
- **Metadados sem camada:** ENV, CMD, LABEL, EXPOSE e similares só alteram o config; não criam camada de sistema de arquivos.

3.1.2 Copy-on-write em detalhe

- **Ideia:** um recurso (bloco de disco, página de memória, arquivo) só é **copiado quando alguém vai modificá-lo**. Até lá, todos compartilham o original. Uma analogia comum: é como fazer anotações em um livro emprestado, em que, no instante em que a caneta toca a página, alguém tira uma cópia daquela página e você escreve na cópia. O livro original continua intacto.
- **No Docker,** um contêiner é uma pilha de **N camadas somente leitura** (as da imagem) com **uma camada de leitura e escrita** por cima (a do contêiner). O union filesystem apresenta tudo como um único diretório.

Operação no contêiner	O que acontece nas camadas
Ler um arquivo	A busca começa na camada superior e desce até encontrar o arquivo. Nada é copiado
Criar um arquivo	É gravado direto na camada gravável
Alterar um arquivo da imagem	Na primeira escrita, o arquivo inteiro é copiado para a camada gravável (copy-up) e alterado lá. Com overlays, a cópia acontece uma vez só; as escritas seguintes vão direto para a cópia
Apagar um arquivo da imagem	As camadas inferiores são somente leitura, então é criado um marcador whiteout na camada superior, que esconde o arquivo. O espaço não é liberado na imagem

- **Consequências práticas:** alterar pela primeira vez um arquivo grande da imagem (um banco de dados embutido, um log grande) é caro, porque ele é copiado inteiro. Muitas camadas deixam buscas de arquivo um pouco mais lentas. E dados que mudam muito devem ficar em **volumes**, que ficam fora do union filesystem (capítulo 7).
- **Economia:** dez contêineres da mesma imagem ocupam o espaço da imagem **uma vez** mais dez camadas graváveis, geralmente pequenas. Com overlayfs, os contêineres também **compartilham o cache de páginas** dos arquivos comuns, o que economiza memória.
- `docker container ls -s` mostra o tamanho da camada gravável de cada contêiner (`size`) e o tamanho virtual (camada gravável + imagem).
- `docker image history <imagem>` mostra cada camada, a instrução que a criou e o tamanho. `docker image inspect` mostra config e digests.

3.1.3 Referências de imagem

Formato completo: `[registry[:porta]/][namespace/]repositório[:tag] [@digest]`

Referência escrita	Referência completa
<code>nginx</code>	<code>docker.io/library/nginx:latest</code>
<code>bitnami/redis:7.4</code>	<code>docker.io/bitnami/redis:7.4</code>
<code>ghcr.io/org/app:1.2.0</code>	Registry GitHub Container Registry
<code>123456789012.dkr.ecr.sa-east-1.amazonaws.com/app:abc123</code>	Amazon ECR
<code>localhost:5000/app</code>	Registry local na porta 5000
<code>nginx:1.27@sha256:...</code>	O digest manda; a tag vira só documentação

- **Sem registry**, assume `docker.io`. **Sem namespace** no Docker Hub, assume `library` (imagens oficiais). **Sem tag**, assume `latest`.
- `latest` **não significa "mais recente"**: é só a tag padrão. Ela aponta para o que foi enviado por último com esse nome, ou para nada.
- **Tags são mutáveis**: `node:22` hoje e daqui a um mês podem ser imagens diferentes. Para builds reproduzíveis, **fixe pelo digest** (`FROM node:22-slim@sha256:...`) e use uma ferramenta (Renovate, Dependabot) para atualizar.

3.2 Registries

3.2.1 Docker Hub

- Registry padrão. Tipos de conteúdo confiável: **Docker Official Images** (namespace `library`, mantidas pela Docker com os upstreams), **Verified Publisher** (empresas parceiras) e **Docker-Sponsored Open Source**.
- **Docker Hardened Images (DHI)**: imagens mínimas, sem shell e sem gerenciador de pacotes na variante de runtime, que rodam como usuário não root, com SBOM e provenance assinados. Gratuitas e Apache 2.0 desde dez/2025; o plano Enterprise acrescenta SLA de correção de CVEs críticos, imagens FIPS e STIG.
- **Limites de pull** (janela de 6 horas, conferidos em set/2026):

Tipo de usuário	Pulls por 6 horas
Não autenticado	100 por endereço IPv4 ou sub-rede IPv6 /64
Personal (conta gratuita autenticada)	200
Pro, Team e Business	Ilimitado (sujeito a uso justo)

- Em CI e clusters, **sempre autentique** os pulls, use um **mirror/pull-through cache** (Harbor, ECR pull-through cache, `registry-mirrors`) e evite baixar a mesma imagem em cada job. Um NAT compartilhado faz vários servidores contarem como um único IP.

3.2.2 Outros registries

Registry	Autenticação típica
GitHub Container Registry (<code>ghcr.io</code>)	<code>GITHUB_TOKEN</code> no Actions, ou personal access token com <code>write:packages</code>
Amazon ECR	<code>aws ecr get-login-password docker login --username AWS --password-stdin <conta>.dkr.ecr.<região>.amazonaws.com</code> (token vale 12 h), ou o credential helper <code>docker-credential-ecr-login</code>
Azure Container Registry	<code>az acr login --name <registry></code>
Google Artifact Registry	<code>gcloud auth configure-docker <região>-docker.pkg.dev</code>
GitLab Container Registry	<code>CI_REGISTRY_USER</code> / <code>CI_REGISTRY_PASSWORD</code> no pipeline
Harbor (self-hosted, CNCF)	Usuários, robot accounts, replicação, scan e cache de proxy
Distribution (imagem oficial <code>registry</code>)	Registry mínimo open source do projeto CNCF <code>distribution/distribution</code> ; versão atual <code>registry:3</code> (veja "Registry próprio" abaixo)

3.2.3 Credenciais da CLI

- `docker login <registry>` grava a credencial em `~/.docker/config.json`. Sem **credential helper**, ela fica **codificada em base64, não criptografada**.
- Configure `credsStore` (`osxkeychain`, `wincred`, `secretservice`, `pass`, `desktop`) ou `credHelpers` por registry (ex.: `ecr-login`) para guardar no cofre do sistema.
- Em scripts, use `--password-stdin` para a senha não aparecer no histórico nem na lista de processos.

3.2.4 Avaliar uma imagem antes de usar

Imagens prontas economizam muito tempo, mas **não coloque em produção algo que você não sabe o que é**. Antes de adotar uma imagem de terceiros:

- `docker image inspect <imagem>` mostra o config completo: `Cmd`, `Entrypoint`, `Env`, `User`, `ExposedPorts`, `Volumes`, `Labels`, arquitetura, SO e digests. Filtre com `-f '{{.Config.User}}'` ou `--format json | jq`.
- `docker image history <imagem>` lista as camadas com a instrução que criou cada uma e o tamanho. Instruções só de metadados aparecem com tamanho 0; camadas de imagens baixadas aparecem com `<missing>` na coluna `IMAGE`, porque os IDs intermediários não são distribuídos. `--no-trunc` mostra o comando inteiro.
- Dockerfile e repositório de origem:** imagens oficiais e boas imagens da comunidade publicam o Dockerfile. Confira o rótulo `org.opencontainers.image.source`.
- Sem baixar a imagem:** o Docker Hub mostra as camadas de cada tag na página da imagem, e `docker buildx imagetools inspect` mostra plataformas e digests. O antigo site ImageLayers deixou de existir. Para explorar o conteúdo de cada camada, a ferramenta **dive** é bastante usada.
- Vulnerabilidades e origem:** `docker scout quickview / cves`, SBOM e provenance (capítulo 9).

3.2.5 Enviar imagens ao Docker Hub

1. Crie a conta em **hub.docker.com** (a criação pela CLI não existe mais). Contas gratuitas têm repositórios públicos ilimitados e um número limitado de repositórios privados; builds automáticos a partir do GitHub/Bitbucket são recurso de planos pagos.

2. Autentique: `docker login` (Docker Hub) ou `docker login registry.exemplo.com` (outro registry). Prefira um **personal access token** no lugar da senha e um credential helper.
3. Nomeie a imagem no formato `usuário-ou-organização/repositório:tag` (ex.: `minhaconta/apache:1.0`). Sem o namespace, o push tenta `docker.io/library/...`, que é reservado às imagens oficiais, e falha com `denied`.
4. Envie: `docker push minhaconta/apache:1.0`. Camadas que o registry já tem aparecem como `Layer already exists` e não são reenviadas; no fim, o push mostra o **digest** da imagem.

```
docker login
docker image tag apache:1.0 minhaconta/apache:1.0
docker push minhaconta/apache:1.0

# Testar o caminho completo: remover a cópia local e baixar de novo
docker image rm minhaconta/apache:1.0
docker pull minhaconta/apache:1.0
docker run -d -p 8080:80 minhaconta/apache:1.0
```

- O repositório é criado **público** por padrão no primeiro push (configurável na conta).
- `docker search <termo>` procura repositórios no Docker Hub (colunas `NAME`, `DESCRIPTION`, `STARS`, `OFFICIAL`; a coluna `AUTOMATED` foi removida). Filtros: `--filter is-official=true`, `--filter stars=100`.
- **Remover imagem em uso:** `docker image rm` falha se algum contêiner (mesmo parado) usa a imagem. Com `-f`, a tag é removida (`Untagged: ...`), mas as camadas continuam enquanto houver contêineres usando-as. O caminho limpo é remover os contêineres antes.

3.2.6 Registry próprio

Para guardar imagens dentro da empresa sem depender de um serviço externo, rode a imagem oficial `registry` (projeto **Distribution**, hoje na CNCF, que substituiu o antigo Docker Registry em Python):

```
docker run -d -p 5000:5000 --restart=always --name registry -v registry-dados:/var/lib/registry registry:3

docker image tag minhaconta/apache:1.0 localhost:5000/apache:1.0
docker push localhost:5000/apache:1.0
docker pull localhost:5000/apache:1.0
curl http://localhost:5000/v2/_catalog # lista os repositórios
```

- O **nome do registry faz parte da referência**: para enviar a outro registry, basta dar à imagem uma tag com o endereço dele (`host:porta/repositório:tag`).
- `localhost` é aceito sem TLS. Para outros hosts, o Docker exige **HTTPS**; sem certificado válido, cada cliente precisa listar o registry em `insecure-registries` no `daemon.json`, o que só é aceitável em laboratório.
- Para uso sério, configure **TLS, autenticação** (htpasswd ou token), armazenamento persistente (volume ou S3/Azure/GCS) e **garbage collection** para liberar espaço de camadas sem referência.
- Precisa de interface web, usuários e projetos, replicação, scan de vulnerabilidades e cache de proxy? Use o **Harbor** (CNCF) ou o registry gerenciado da sua nuvem (ECR, Artifact Registry, ACR, GHCR).

3.3 Gerenciar imagens

3.3.1 Comandos essenciais

Comando	O que faz
<code>docker pull <ref></code>	Baixa a imagem (a variante da plataforma local, se houver index)
<code>docker push <ref></code>	Envia a imagem; camadas que o registry já tem não são reenviadas
<code>docker tag origem destino</code>	Cria outro nome para a mesma imagem (não copia dados)
<code>docker image ls / ls --digests</code>	Lista imagens locais
<code>docker image rm / rmi</code>	Remove tags; os dados só saem quando nenhuma tag ou contêiner os usa
<code>docker save -o app.tar app:1.0 / docker load -i app.tar</code>	Exporta e importa imagens completas (camadas, tags e metadados). Serve para ambientes sem acesso a registry
<code>docker export <c> > fs.tar / docker import fs.tar</code>	Exporta o sistema de arquivos achatado de um contêiner ; o import gera uma imagem de uma camada, sem histórico, CMD ou ENV
<code>docker commit <c> img:tag</code>	Cria uma imagem a partir do estado de um contêiner. Útil para investigação; não para produção (não é reproduzível)
<code>docker buildx imagetools inspect <ref></code>	Mostra o index e as plataformas de uma imagem no registry , sem baixá-la
<code>docker manifest inspect <ref></code>	Alternativa mais antiga ao <code>imagetools inspect</code>

- **Imagem dangling:** aparece como `<none>:<none>` quando uma tag é movida para um build novo e a imagem antiga fica sem nome. É removida por `docker image prune`.

3.3.2 Criar uma imagem com `docker commit` (e por que evitar)

Há duas formas de criar uma imagem: **declarativa**, com um Dockerfile (capítulo 4), e **imperativa**, alterando um contêiner à mão e salvando o resultado.

```
docker run -it --name base debian:13 bash
# dentro do contêiner:
apt-get update && apt-get install -y apache2 && apt-get clean
# Ctrl+P Ctrl+Q para sair sem parar

docker container commit -m "Debian com Apache" \
  --change 'CMD ["apachectl", "-D", "FOREGROUND"]' base # imagem <none>:<none>
docker image tag <IMAGE ID> minha-org/apache:1.0 # dá nome e tag
# ou, direto: docker container commit base minha-org/apache:1.0
```

- `-m` registra uma mensagem, `-a` o autor e `--change` aplica instruções de Dockerfile (`CMD`, `ENV`, `EXPOSE`, `WORKDIR` ...) à nova imagem. Sem `--change`, a imagem herda o `CMD` do contêiner (no exemplo, `bash`), e o serviço precisaria ser iniciado à mão.
- **Por que evitar em produção:** ninguém sabe exatamente o que foi feito (não há receita), a imagem carrega sujeira da sessão (históricos, caches, arquivos temporários), não há cache de build nem revisão de código, e refazer a imagem com uma base atualizada exige repetir tudo à mão. Use `commit` para **investigação** (congelar um contêiner com problema) ou para descobrir os passos que depois vão para o Dockerfile.

- **Imagens prontas de terceiros** agilizam muito provas de conceito (POC): em minutos é possível testar várias ferramentas. Para uso contínuo, prefira imagens oficiais ou a sua própria imagem construída a partir de um Dockerfile.

3.3.3 Estratégia de tags

- **Tag imutável por build:** SHA do commit (`app:3f9c2ab`) ou versão semântica (`app:1.4.2`). É o que o deploy deve referenciar.
- **Tags móveis de conveniência:** `app:1.4`, `app:1`, `app:latest`, apontando para a última versão compatível.
- **Não reutilize** uma tag de versão para outro conteúdo; alguns registries (ECR, Harbor, GHCR com regras) permitem **tornar tags imutáveis**.
- **Promoção entre ambientes:** a mesma imagem (mesmo digest) passa por dev, homologação e produção; muda só a configuração. Rebuildar para cada ambiente quebra a garantia de que o que foi testado é o que vai para produção.

3.3.4 Números de imagens e registries

Item	Valor
Registry padrão	<code>docker.io</code> (Docker Hub)
Namespace das imagens oficiais	<code>library</code>
Tag padrão	<code>latest</code>
Algoritmo do digest	SHA-256
Janela dos limites de pull do Docker Hub	6 horas
Pulls não autenticados no Docker Hub	100 por IPv4 ou /64 IPv6
Pulls autenticados (Personal)	200
Validade do token do <code>aws ecr get-login-password</code>	12 horas
Porta convencional de um registry local	5000

3.4 Decisão rápida — Imagens e registries

Situação	Abordagem recomendada	Erro comum
Garantir que o deploy use exatamente a imagem testada	Referenciar pelo digest ou por tag imutável por commit	Usar <code>latest</code>
Build reproduzível da imagem base	<code>FROM imagem:tag@sha256:... + atualização automática</code>	Tag flutuante sem digest
Levar imagens para um ambiente sem internet	<code>docker save</code> / <code>docker load</code> (ou registry interno)	<code>docker export</code> / <code>import</code> (perde metadados)
CI estourando o limite de pulls do Docker Hub	Autenticar, usar mirror/pull-through cache	Repetir o job até passar
Descobrir se uma imagem tem variante <code>arm64</code> sem baixá-la	<code>docker buildx imagetools inspect</code>	<code>docker pull</code> em cada máquina
Imagem ocupando espaço sem nome	<code>docker image prune</code> (dangling)	Apagar diretórios em <code>/var/lib/docker</code>
Guardar credenciais de registry com segurança	Credential helper (<code>credsStore</code>) e <code>--password-stdin</code>	Deixar o <code>config.json</code> com base64

Situação	Abordagem recomendada	Erro comum
Escolher uma base mínima e mantida	Docker Official Image -slim, distroless ou Docker Hardened Image	Imagem aleatória do Docker Hub sem mantenedor
Criar uma imagem a partir de alterações feitas à mão em um contêiner	Levar as alterações para o Dockerfile	<code>docker commit</code> em produção

4. Dockerfile

Ideia central: o Dockerfile é a **receita reproduzível** da imagem. Cada instrução que altera arquivos gera uma camada, e o cache é reaproveitado **em ordem, de cima para baixo, até a primeira instrução que mudar**. Ordenar do que muda menos para o que muda mais é a otimização mais barata que existe.

4.1 Primeiro Dockerfile e primeiro build

4.1.1 Do Dockerfile à imagem

- O **Dockerfile** é um arquivo de texto com a receita da imagem: a base, os pacotes a instalar, os arquivos a copiar, as variáveis, o usuário, a porta e o comando de início. Funciona como um **Makefile** para imagens, e a vantagem é a automação: a mesma receita gera a mesma imagem em qualquer máquina ou pipeline.

```
# primeiro/Dockerfile
FROM debian:13
RUN echo "HELLO DOCKER"
```

```
cd primeiro
docker build -t primeiro:1.0 .
docker image ls primeiro
```

- O **último argumento** do `docker build` é o **contexto de build**, um **diretório** (ou URL de repositório Git), e não o arquivo. O `.` significa "o diretório atual". Por padrão, o builder procura um arquivo `Dockerfile` na raiz do contexto. Para usar outro nome ou lugar: `docker build -f docker/api.Dockerfile -t api:1.0 ..`
- Sem** `-t`, a imagem é criada **sem nome**: aparece como `<none>:<none>` em `docker image ls` e só pode ser referenciada pelo ID. Dê o nome no build (`-t repositório:tag`, podendo repetir `-t`) ou depois, com `docker image tag`.
- A saída atual é a do **BuildKit** (`[+] Building`, passos numerados como `[2/2] RUN ...`, `CACHED` quando reaproveita o cache). Materiais antigos mostram a saída do builder legado (`Sending build context to Docker daemon`, `Step 1/2, Removing intermediate container`), que só aparece hoje com `DOCKER_BUILDKIT=0` em imagens Windows. Use `--progress=plain` para ver a saída completa de cada `RUN` (por exemplo, o `HELLO DOCKER` do exemplo).

4.1.2 Processo em primeiro plano

Um erro comum ao começar: construir uma imagem com o serviço instalado, entrar no contêiner com `-it` e iniciar o serviço à mão (`/etc/init.d/apache2 start`). O serviço até responde, mas **ninguém deveria precisar entrar no contêiner** para isso, e o PID 1 continua sendo o `bash`.

- O contêiner deve iniciar a aplicação **sozinho**, com o processo em **primeiro plano** como PID 1. Serviços que se "daemonizam" (vão para segundo plano) fazem o contêiner terminar logo depois de iniciar.

```
# syntax=docker/dockerfile:1
FROM debian:13
RUN apt-get update \
  && apt-get install -y --no-install-recommends apache2 \
  && rm -rf /var/lib/apt/lists/*
ENV APACHE_RUN_USER=www-data \
  APACHE_RUN_GROUP=www-data \
  APACHE_RUN_DIR=/var/run/apache2 \
  APACHE_PID_FILE=/var/run/apache2/apache2.pid \
  APACHE_LOCK_DIR=/var/lock/apache2 \
  APACHE_LOG_DIR=/var/log/apache2
LABEL org.opencontainers.image.description="Servidor web Apache"
EXPOSE 80
ENTRYPOINT ["apachectl"]
CMD ["-D", "FOREGROUND"]
```

- `ENTRYPOINT ["apachectl"]` + `CMD ["-D", "FOREGROUND"]` resulta em `apachectl -D FOREGROUND`: o `ENTRYPOINT` é o executável e o `CMD` são os **argumentos padrão**, que podem ser trocados no `docker run`.
- Equivalentes em outros servidores: `nginx -g 'daemon off;'`, `httpd -DFOREGROUND` (a imagem oficial `httpd` já faz isso), `php-fpm -F`.
- Várias instruções `ENV` podem ser agrupadas em uma só (`ENV A=1 B=2`), o que deixa o Dockerfile mais legível. Como `ENV` não cria camada de sistema de arquivos, o ganho é de organização, não de tamanho.
- Na prática, prefira a **imagem oficial** do serviço (`httpd`, `nginx`) a instalar o servidor sobre uma distribuição: ela já vem configurada para rodar em primeiro plano e é mantida pelos upstreams.

4.2 Instruções

4.2.1 Referência das instruções

Instrução	O que faz	Observação
<code># syntax=docker/dockerfile:1</code>	Diretiva de parser: usa a versão mais recente estável do frontend do Dockerfile	Deve ser a primeira linha. Habilita recursos novos sem atualizar o Engine
<code>FROM imagem [AS nome]</code>	Define a imagem base (ou <code>scratch</code> , vazia) e inicia um estágio	Pode haver vários <code>FROM</code> (multi-stage). Antes do primeiro <code>FROM</code> só podem vir diretivas de parser, comentários e <code>ARG</code>
<code>ARG nome[=padrão]</code>	Variável só de build (<code>--build-arg</code>)	Antes do primeiro <code>FROM</code> , só vale para os <code>FROM</code> . Aparece no histórico: não use para segredos
<code>ENV nome=valor</code>	Variável de ambiente no build e na execução	Persiste na imagem e em todos os estágios filhos
<code>WORKDIR /app</code>	Diretório de trabalho (criado se não existir)	Prefira a <code>RUN cd ...</code>
<code>COPY [--chown] [--chmod] [--link] [--from] origem destino</code>	Cópia do contexto de build ou de outro estágio/imagem	Preferido para arquivos locais
<code>ADD origem destino</code>	Como <code>COPY</code> , mas também baixa URLs, clona repositórios Git e extraí tarballs locais	Use só quando precisar desses extras
<code>RUN comando</code>	Executa um comando no build e grava o resultado em uma camada	Suporta <code>--mount</code> (<code>cache</code> , <code>secret</code> , <code>ssh</code> , <code>bind</code>) e heredocs
<code>USER nome uid[:gid]</code>	Usuário das instruções seguintes e do processo em execução	O padrão é root , a menos que a base defina outro. Rode a aplicação como não root
<code>EXPOSE 8080[/tcp]</code>	Documenta a porta que a aplicação escuta	Não publica a porta; <code>-P</code> usa essa lista

Instrução	O que faz	Observação
VOLUME /dados	Declara um ponto de montagem; cria volume anônimo se nada for montado	Mudanças no diretório depois do VOLUME são descartadas no build
CMD	Comando (ou argumentos) padrão, executado quando o contêiner inicia (o RUN executa no build)	Substituído por argumentos no docker run. Só o último CMD vale
ENTRYPOINT	Executável fixo do contêiner; quando ele termina, o contêiner termina	Substituído só com --entrypoint
HEALTHCHECK	Comando que o Docker executa para saber se o contêiner está saudável	HEALTHCHECK NONE desliga o herdado
STOPSIGNAL SIGQUIT	Sinal enviado pelo docker stop	Padrão SIGTERM
SHELL ["pwsh", "-c"]	Troca o shell da forma shell	Útil em imagens Windows
LABEL chave=valor	Metadados (versão, fonte, licença)	Padrão org.opencontainers.image.*
ONBUILD instrução	Instrução executada quando outra imagem usar esta como base	Pouco usado; esconde comportamento
MAINTAINER	Obsoleto	Use LABEL org.opencontainers.image.authors

4.2.2 Forma shell vs. forma exec

Forma	Exemplo	Executa	Consequência
Exec (array JSON)	CMD ["node", "server.js"]	O binário diretamente, como PID 1	Recebe os sinais; sem expansão de variáveis de shell
Shell	CMD node server.js	/bin/sh -c "node server.js"	Expansão de variáveis funciona, mas o PID 1 é o shell e os sinais não chegam à aplicação

- O array JSON exige **aspas duplas**: CMD ['node', 'server.js'] (aspas simples) é tratado como forma shell e falha.
- Precisa de variáveis e sinais corretos ao mesmo tempo? Use um script com **exec** no final, ou CMD ["sh", "-c", "exec node server.js --port \$PORT"].

4.2.3 CMD e ENTRYPOINT juntos

	Sem ENTRYPOINT	ENTRYPOINT "app" (exec)	ENTRYPOINT app (shell)
Sem CMD	Erro: nada a executar (a menos que a base defina)	app	/bin/sh -c app
CMD ["--port", "80"]	--port 80 (inválido)	app --port 80	/bin/sh -c app (CMD ignorado)
CMD ["sh", "-c", "x"]	sh -c x	app sh -c x	/bin/sh -c app (CMD ignorado)

- Padrão recomendado:** ENTRYPOINT em forma exec com o executável e CMD com os argumentos padrão, que o usuário troca no docker run imagem --outro-arg.
- ENTRYPOINT em forma shell **ignora o CMD e os argumentos do docker run**.
- Definir ENTRYPOINT em um estágio **zero** o CMD herdado da imagem base.

4.2.4 COPY vs. ADD e o .dockerignore

- `COPY` é o padrão. `ADD` só para: baixar um arquivo com verificação de checksum (`ADD --checksum=sha256:... https://...`), clonar um repositório Git (`ADD git@github.com:org/repo.git /src`) ou extrair um tarball local automaticamente.
- `COPY --link` cria a camada de forma independente das anteriores, permitindo reaproveitá-la mesmo quando a base muda (útil com `--from` em multi-stage).
- `COPY --chown=app:app` evita um `RUN chown -R` posterior, que duplicaria os arquivos em uma nova camada.
- **Contexto de build:** o diretório (ou URL) enviado ao builder. Tudo o que não está nele não pode ser copiado; tudo o que está nele é enviado, a menos que o `.dockerignore` exclua.
- `.dockerignore` deve excluir `.git`, `node_modules`, artefatos de build, arquivos `.env` e chaves. Sem ele, o build fica lento, o cache é invalidado à toa e segredos podem acabar na imagem com um `COPY . . .`.

```
# .dockerignore
.git
node_modules
dist
*.log
.env*
**/*.pem
Dockerfile*
compose*.yaml
```

4.2.5 HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --start-interval=2s --retries=3 \
  CMD ["wget", "-q0-", "http://localhost:8080/health"]
```

Opção	Padrão	Significado
<code>--interval</code>	30 s	Intervalo entre verificações
<code>--timeout</code>	30 s	Tempo máximo de uma verificação
<code>--start-period</code>	0 s	Período inicial em que falhas não contam para <code>unhealthy</code>
<code>--start-interval</code>	5 s	Intervalo entre verificações durante o <code>start-period</code> (Engine 25+)
<code>--retries</code>	3	Falhas seguidas até marcar <code>unhealthy</code>

- Estados: `starting` → `healthy` ou `unhealthy`. Código 0 = saudável, 1 = doente.
- O comando roda **dentro do contêiner**: a ferramenta (`curl`, `wget`) precisa existir na imagem. Em imagens mínimas, use um binário próprio (`/app/healthcheck`) ou a própria aplicação com uma flag.
- Quem usa o estado: `docker ps`, `depends_on: condition: service_healthy` no Compose, Swarm (substitui tarefas doentes) e ferramentas de deploy. Kubernetes **ignora** o `HEALTHCHECK` e usa probes próprias.

4.3 Cache de build

4.3.1 Como o cache funciona

- Para cada instrução, o builder verifica se já existe uma camada gerada **a partir da mesma camada pai com a mesma instrução**.
- Para `COPY` e `ADD`, o que conta é o **conteúdo** (checksum) dos arquivos copiados, não a data.
- Para `RUN`, conta só o **texto do comando**: `RUN apt-get update` fica em cache para sempre, mesmo que os repositórios mudem. Por isso, **junte** `apt-get update` e `apt-get install` no mesmo `RUN`.
- **Quando uma instrução invalida o cache, todas as seguintes são refeitas.**

4.3.2 Ordem que aproveita o cache

```
# syntax=docker/dockerfile:1
FROM node:22-slim
WORKDIR /app

# 1. Manifestos de dependências mudam pouco: copiados primeiro
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci --omit=dev

# 2. Código-fonte muda a cada commit: copiado depois
COPY . .

USER node
CMD ["node", "server.js"]
```

- Com essa ordem, mudar só o código-fonte **não** reinstala as dependências.
- **Cache mount** (`--mount=type=cache`) guarda o cache do gerenciador de pacotes (npm, pip, Maven, Go, apt) entre builds, **sem colocá-lo na imagem**.
- **Forçar um build limpo**: `docker build --no-cache`, ou `--no-cache-filter <estágio>` para um estágio específico. `--pull` força baixar a versão mais nova da imagem base.

4.4 Multi-stage builds

4.4.1 Por que e como

Vários `FROM` no mesmo Dockerfile. Os estágios iniciais têm compiladores e ferramentas; o estágio final copia **só o artefato**. A imagem final não carrega o SDK, o código-fonte nem o cache.

```
# syntax=docker/dockerfile:1
FROM golang:1.25 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download
COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 go build -o /out/app ./cmd/app

FROM gcr.io/distroless/static-debian12:nonroot AS runtime
COPY --from=build /out/app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

- `--target <estágio>` faz o build parar em um estágio: útil para ter um estágio `dev` (com ferramentas) e um `test` no mesmo arquivo.
- O BuildKit **só executa os estágios necessários** para o alvo e roda estágios independentes **em paralelo**.
- `COPY --from=<imagem>` também copia de uma imagem externa: `COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/`.
- Uma imagem Go estática pode partir de `scratch` (nada além do binário). Nesse caso, lembre de copiar certificados CA e dados de fuso horário se a aplicação precisar.
- **Ordem de grandeza:** uma imagem de estágio único baseada em `golang` passa de 800 MB, porque carrega o compilador e o SDK inteiros; o mesmo binário em um estágio final `alpine`, `distroless` ou `scratch` fica com poucos MB. Com o multi-stage, o estágio de build é descartado e só o binário vai para a imagem final.
- Se o estágio final for **Alpine** (musl libc), compile com `CGO_ENABLED=0` para gerar um binário estático, e use uma versão **mantida** do Alpine. Exemplos antigos com `alpine:3.1` ou `golang` sem tag usam versões fora de suporte.
- Prefira `COPY` a `ADD` para o código-fonte e a forma exec em `ENTRYPOINT ["/app"]`. A forma shell (`ENTRYPOINT /app`) exige um shell na imagem final e deixa o `sh` como PID 1.

4.5 Boas práticas

4.5.1 Escolha da imagem base

Base	Tamanho típico	Tem shell e gerenciador de pacotes?	Quando usar
<code>debian / ubuntu</code> completa	75–120 MB	Sim	Desenvolvimento, quando precisa de muitas ferramentas
<code>-slim</code> (Debian)	25–80 MB	Sim (apt)	Bom equilíbrio para a maioria das linguagens interpretadas
<code>alpine</code>	5–8 MB	Sim (apk)	Imagens pequenas. Usa musl libc: binários nativos compilados para glibc podem falhar ou ficar mais lentos
Distroless	2–25 MB	Não	Produção; só a runtime da linguagem e suas bibliotecas
Docker Hardened Images	Varia	Não na variante de runtime; sim na variante <code>-dev</code>	Produção com SBOM, provenance e correção contínua de CVEs
<code>scratch</code>	0	Não	Binários estáticos (Go, Rust)

4.5.2 Checklist de um bom Dockerfile

- Primeira linha `# syntax=docker/dockerfile:1`.

- Base oficial ou endurecida, com **tag específica e digest**.
- **Multi-stage**: build separado do runtime.
- Dependências antes do código, com **cache mounts**.
- `.dockerignore` completo.
- `RUN apt-get update && apt-get install -y --no-install-recommends ... && rm -rf /var/lib/apt/lists/*` no mesmo `RUN`.
- **Usuário não root** (`USER 10001` ou o usuário da base, como `node` ou `nonroot`).
- `CMD / ENTRYPOINT` em **forma exec**; scripts de entrada com `exec "$@"`.
- **Um processo principal por contêiner**; processos auxiliares em outros contêineres.
- **Nenhum segredo** em `ARG`, `ENV` ou arquivos copiados: use build secrets (capítulo 5).
- Configuração por **variáveis de ambiente**, não embutida por ambiente.
- `HEALTHCHECK` quando quem executa usa o estado (Compose, Swarm).
- `LABEL org.opencontainers.image.source`, `.version`, `.revision`.
- Validar com `docker build --check` (build checks do BuildKit) e um linter como o **Hadolint**.

4.5.3 Por que apagar arquivos em outro `RUN` não reduz a imagem

```
# Ruim: o arquivo de 500 MB continua na camada do primeiro RUN
RUN curl -o /tmp/pacote.tgz https://exemplo.com/pacote.tgz
RUN tar xzf /tmp/pacote.tgz -C /opt && rm /tmp/pacote.tgz

# Bom: baixa, extrai e apaga na mesma camada
RUN curl -fsSL https://exemplo.com/pacote.tgz | tar xz -C /opt
```

- Alternativas: fazer o download em um estágio de build e copiar só o resultado, ou usar `RUN --mount=type=bind / type=cache` para arquivos que não devem entrar na imagem.

4.6 Decisão rápida — Dockerfile

Situação	Abordagem recomendada	Erro comum
Qualquer mudança no código reinstala todas as dependências	Copiar manifestos de dependências antes do código	<code>COPY . .</code> no início
Imagem de produção com compilador e código-fonte	Multi-stage com estágio final mínimo	Apagar ferramentas com <code>RUN rm</code>
Aplicação não recebe <code>SIGTERM</code>	<code>CMD / ENTRYPOINT</code> em forma <code>exec</code>	Forma shell com timeout maior
Imagem aceitar argumentos extras no <code>docker run</code>	<code>ENTRYPOINT ["app"] + CMD ["--padrao"]</code>	<code>ENTRYPOINT</code> em forma shell
Copiar arquivos locais	<code>COPY</code>	<code>ADD</code> sem precisar de URL, Git ou extração
Baixar arquivo remoto com integridade	<code>ADD --checksum=sha256:...</code> ou <code>curl + verificação</code>	<code>ADD</code> sem checksum
<code>apt-get install</code> instala versões antigas	<code>update</code> e <code>install</code> no mesmo <code>RUN</code>	<code>RUN apt-get update</code> isolado (fica em cache)
Build lento por enviar gigabytes de contexto	<code>.dockerignore</code>	Mover o Dockerfile para outro lugar
Estágios de dev e produção no mesmo arquivo	Multi-stage com <code>--target</code>	Dois Dockerfiles quase iguais
Porta aparecer em <code>docker ps</code> sem <code>-p</code>	Não existe: <code>EXPOSE</code> só documenta	Esperar que <code>EXPOSE</code> publique

Situação	Abordagem recomendada	Erro comum
Binário nativo falha no Alpine	Base <code>-slim</code> (glibc) ou compilar para musl	Adicionar pacotes de compatibilidade até funcionar
Tamanho mínimo para Go/Rust estático	<code>scratch</code> ou <code>distroless static</code>	Base Debian completa
Serviço iniciado à mão dentro do contêiner	<code>ENTRYPOINT</code> / <code>CMD</code> com o processo em primeiro plano	<code>docker exec + /etc/init.d/<serviço> start</code>
Imagem sem nome (<code><none></code>) depois do build	<code>docker build -t nome:tag</code> ou <code>docker image tag</code>	Referenciar pelo ID em scripts

5. BuildKit, Buildx e Builds Avançados

Ideia central: o **BuildKit** é o motor que executa o build (padrão desde o Engine 23), o **Buildx** é a CLI que conversa com ele (`docker build` já é um alias de `docker buildx build`) e o **Bake** descreve vários builds em um arquivo. Juntos, entregam paralelismo, cache exportável, segredos que não vazam, imagens multiplataforma e atestados de cadeia de suprimentos.

5.1 Recursos do BuildKit no Dockerfile

5.1.1 Montagens em `RUN --mount`

Tipo	Para que serve	Exemplo
cache	Cache persistente entre builds que não entra na imagem	<code>RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt</code>
secret	Disponibiliza um segredo só durante aquele <code>RUN</code> , sem gravá-lo em camada nem no histórico	<code>RUN --mount=type=secret,id=npnrc,target=/root/.npnrc npn ci</code>
ssh	Encaminha o agente SSH do host para clonar repositórios privados	<code>RUN --mount=type=ssh git clone git@github.com:org/privado.git</code>
bind	Monta arquivos do contexto ou de outro estágio sem copiá-los para a imagem	<code>RUN --mount=type=bind,source=go.sum,target=go.sum go mod download</code>
tmpfs	Diretório temporário em memória durante o <code>RUN</code>	<code>RUN --mount=type=tmpfs,target=/tmp ...</code>

- **Passar os segredos no build:** `docker build --secret id=npnrc,src=$HOME/.npnrc .` e `docker build --ssh default .` Por padrão, o segredo aparece em `/run/secrets/<id>`; com `env=NOME` ele vira uma variável de ambiente só naquele `RUN`.
- **Por que não** `ARG TOKEN`: valores de `ARG` e `ENV` ficam gravados no histórico e no config da imagem (`docker history`, `docker inspect`), e qualquer pessoa com a imagem os lê.

5.1.2 Heredocs e build checks

- **Heredocs** deixam scripts de várias linhas legíveis, em um único `RUN` (uma camada):

```
RUN <<EOF
set -eux
apt-get update
apt-get install -y --no-install-recommends ca-certificates curl
rm -rf /var/lib/apt/lists/*
EOF
```

- **Build checks:** `docker build --check .` avalia o Dockerfile sem construir e aponta problemas como forma shell no `CMD`, `FROM` com maiúsculas e minúsculas inconsistentes, `ENV` em sintaxe antiga ou `ARG` com nome de segredo. A diretiva `# check=error=true` transforma os avisos em erro, útil no CI.
- **Saída do build:** `--progress=plain` mostra o log completo de cada passo (o padrão, `auto`, resume). Útil para depurar um `RUN` que falha.

5.2 Buildx: builders, cache e saídas

5.2.1 Drivers de builder

Driver	Onde roda o BuildKit	Uso
<code>docker</code> (padrão)	Embutido no daemon	Build local simples; com o containerd image store, já faz multiplataforma e exporta cache
<code>docker-container</code>	Contêiner <code>moby/buildkit</code> criado pelo Buildx	Recursos completos de cache e saída, versão do BuildKit independente do Engine
<code>kubernetes</code>	Pods em um cluster	Builders compartilhados e escaláveis
<code>remote</code>	Um BuildKit já em execução, acessado por TCP/socket	Infraestrutura de build própria
Docker Build Cloud	Builders gerenciados pela Docker (amd64 e arm64 nativos)	Cache compartilhado entre o time e o CI sem manter infraestrutura

- `docker buildx create --name multi --driver docker-container --use`, `docker buildx ls`, `docker buildx inspect --bootstrap`, `docker buildx rm`.

5.2.2 Saídas do build

Opção	Resultado
<code>--load</code>	Carrega a imagem no image store local (padrão com o driver <code>docker</code>)
<code>--push</code>	Envia ao registry (atalho para <code>--output type=registry</code>)
<code>--output type=local,dest=./out</code>	Grava o sistema de arquivos final em um diretório (ex.: extrair binários compilados)
<code>--output type=tar,dest=out.tar</code>	Tarball do sistema de arquivos
<code>--output type=oci,dest=img.tar / type=docker</code>	Tarball da imagem em formato OCI / Docker

- Com o driver `docker-container`, a imagem **não aparece** em `docker image ls` a menos que você use `--load` ou `--push`.

5.2.3 Cache exportável

O cache local do builder se perde em runners efêmeros de CI. Exporte e importe:

Backend	Exemplo	Quando usar
<code>inline</code>	<code>--cache-to type=inline</code>	Cache embutido na própria imagem; simples, só modo <code>min</code>
<code>registry</code>	<code>--cache-to type=registry,ref=org/app:buildcache,mode=max --cache-from type=registry,ref=org/app:buildcache</code>	Qualquer CI com acesso ao registry
<code>gha</code>	<code>--cache-to type=gha,mode=max --cache-from type=gha</code>	GitHub Actions
<code>local</code>	<code>--cache-to type=local,dest=/tmp/cache</code>	Cache em diretório persistido pelo CI
<code>s3 / azblob</code>	Buckets de nuvem	Pipelines na AWS ou Azure

- `mode=min` (padrão) exporta só as camadas da imagem final; `mode=max` exporta também as camadas dos estágios intermediários, que é o que acelera builds multi-stage.

5.3 Imagens multiplataforma

5.3.1 Estratégias

Estratégia	Como funciona	Prós e contras
Emulação QEMU	O builder executa instruções de outra arquitetura via <code>binfmt_misc</code>	Zero configuração no Docker Desktop; lenta em compilação pesada
Nós nativos	Um builder com um nó <code>amd64</code> e outro <code>arm64</code> (ou Docker Build Cloud)	Rápida; exige infraestrutura
Compilação cruzada	O estágio de build roda na plataforma nativa (<code>--platform=\$BUILDPLATFORM</code>) e compila para <code>\$TARGETOS/\$TARGETARCH</code>	Rápida e barata; depende do suporte da linguagem (Go, Rust, .NET)

```
# syntax=docker/dockerfile:1
FROM --platform=$BUILDPLATFORM golang:1.25 AS build
ARG TARGETOS TARGETARCH
WORKDIR /src
COPY . .
RUN GOOS=$TARGETOS GOARCH=$TARGETARCH CGO_ENABLED=0 go build -o /out/app .

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/app /app
ENTRYPOINT ["/app"]
```

```
docker buildx build --platform linux/amd64,linux/arm64 -t org/app:1.0 --push .
```

- O resultado é um **index** com um manifest por plataforma, sob a mesma tag. Quem faz o pull recebe automaticamente a variante da sua arquitetura.
- **Requisito:** o image store clássico não guarda imagens multiplataforma. Com o **containerd image store** (padrão no Engine 29 e no Docker Desktop) funciona direto; antes disso, era preciso um builder `docker-container` e `--push`.
- **Argumentos automáticos:** `BUILDPLATFORM`, `TARGETPLATFORM`, `TARGETOS`, `TARGETARCH`, `TARGETVARIANT` (declare com `ARG` para usar).
- Sintoma clássico de arquitetura errada: `exec format error` ao iniciar o contêiner (por exemplo, imagem `amd64` construída em um Mac Apple Silicon sem `--platform`).

5.4 Bake e atestados

5.4.1 Docker Bake

Arquivo declarativo (`docker-bake.hcl`, JSON ou o próprio `compose.yaml`) que descreve vários builds, executados em paralelo com `docker buildx bake`.

```

variable "TAG" { default = "dev" }

group "default" { targets = ["api", "worker"] }

target "_comum" {
  platforms = ["linux/amd64", "linux/arm64"]
  cache-from = ["type=gha"]
  cache-to   = ["type=gha,mode=max"]
}

target "api" {
  inherits = ["_comum"]
  context  = "./api"
  tags     = ["ghcr.io/org/api:${TAG}"]
}

target "worker" {
  inherits = ["_comum"]
  context  = "./worker"
  tags     = ["ghcr.io/org/worker:${TAG}"]
}

```

- `docker buildx bake --print` mostra a configuração resolvida; `TAG=1.4.2 docker buildx bake --push` constrói e envia tudo.
- Desde o **Compose v5**, o `docker compose build` delega os builds ao Bake.

5.4.2 SBOM e provenance

- **SBOM** (Software Bill of Materials): inventário dos pacotes da imagem. `docker buildx build --sbom=true ...`
- **Provenance** (SLSA): como, onde e a partir de qual código a imagem foi construída. `--provenance=mode=max` inclui detalhes completos (em repositórios públicos, cuidado com o que o build expõe).
- Os atestados são anexados ao index da imagem no registry e podem ser lidos com `docker buildx imagetools inspect <ref> --format '{{ json .SBOM }}'` e analisados pelo Docker Scout (capítulo 9).

5.5 Decisão rápida — BuildKit e Buildx

Situação	Abordagem recomendada	Erro comum
Build precisa de token para baixar dependências privadas	<code>RUN --mount=type=secret + --secret</code>	<code>ARG TOKEN</code> ou <code>COPY .npmrc</code>
Clonar repositório privado no build	<code>RUN --mount=type=ssh + --ssh default</code>	Copiar a chave SSH para a imagem
Build lento no CI porque o cache se perde	<code>--cache-to/--cache-from (registry ou gha, mode=max)</code>	Desligar o cache
Instalação de pacotes lenta a cada build	<code>--mount=type=cache</code> no diretório do gerenciador	Aceitar a lentidão
Imagem para servidores amd64 e Macs/Graviton arm64	<code>buildx build --platform linux/amd64,linux/arm64 --push</code>	Duas tags diferentes por arquitetura
Compilação sob QEMU muito lenta	Compilação cruzada com <code>\$BUILDPLATFORM</code> ou nós nativos	Aumentar o timeout do CI
<code>exec format error</code>	Construir ou baixar para a plataforma certa (<code>--platform</code>)	Reinstalar dependências

Situação	Abordagem recomendada	Erro comum
Extrair só o binário compilado, sem gerar imagem	<code>--output type=local,dest=./out</code>	<code>docker create</code> + <code>docker cp</code>
Vários serviços construídos juntos, com configuração comum	Docker Bake	Script shell com vários <code>docker build</code>
Auditar de onde veio uma imagem	Provenance e SBOM no build	Confiar só na tag
Ver o log completo de um passo que falha	<code>--progress=plain</code>	Rodar de novo esperando outro resultado

6. Redes

Ideia central: coloque os contêineres que conversam entre si em uma **rede bridge definida pelo usuário**. Nela, eles se encontram **pelo nome** (DNS embutido) e ficam isolados de outras redes. **Publique portas** (`-p`) só para o que precisa ser acessado de fora do host.

6.1 Drivers de rede

6.1.1 Visão geral

Driver	O que faz	Quando usar
bridge (padrão)	Rede virtual privada no host, com NAT para a saída e portas publicadas para a entrada	Contêineres de um mesmo host (caso mais comum)
host	O contêiner usa a pilha de rede do host diretamente; não há isolamento de rede nem mapeamento de portas	Desempenho máximo de rede, muitas portas, ferramentas de rede. Linux (no Docker Desktop, suporte recente e limitado)
none	Só a interface de loopback	Jobs que não devem ter rede
overlay	Rede distribuída entre vários hosts (VXLAN)	Swarm; contêineres em hosts diferentes
macvlan	Cada contêiner recebe um MAC próprio e aparece como um dispositivo físico na LAN	Aplicações legadas que precisam estar na rede física
ipvlan	IP próprio na LAN compartilhando o MAC do host (modos L2 e L3)	Quando a rede limita MACs por porta de switch
<code>container: <nome></code>	Compartilha o namespace de rede de outro contêiner	Sidecars e depuração (<code>netshoot</code>)
Plugins	Drivers de terceiros	Integração com SDN específica

6.1.2 Bridge padrão vs. bridge definida pelo usuário

Aspecto	Bridge padrão (<code>bridge</code> , interface <code>eth0</code>)	Bridge definida pelo usuário (<code>docker network create api</code>)
Resolução de nomes	Não há DNS por nome (só IP ou o obsoleto <code>--link</code>)	DNS embutido: nome do contêiner, <code>--network-alias</code> e, no Compose, o nome do serviço
Isolamento	Todos os contêineres sem <code>--network</code> caem nela e se enxergam	Só os contêineres conectados àquela rede se enxergam
Conectar e desconectar em execução	Não	<code>docker network connect/disconnect</code>
Configuração	Global, no <code>daemon.json</code>	Por rede (sub-rede, gateway, MTU, <code>--internal</code>)
Recomendação	Evitar em aplicações	Usar sempre

- O **DNS embutido** responde em `127.0.0.11` dentro do contêiner e encaminha nomes externos para os servidores do host.
- Um contêiner pode estar em **várias redes** ao mesmo tempo (ex.: `frontend` e `backend`), o que permite segmentar: o proxy fala com a API, a API fala com o banco, o proxy não enxerga o banco.
- `--internal` cria uma rede sem rota para fora: bom para bancos e serviços que não devem acessar a internet.

6.1.3 Opções de rede do `docker run`

Opção	Efeito
<code>--network</code> <code><rede host none container:</code> <code><nome>></code>	Rede ou modo de rede do contêiner. <code>--net</code> é o nome antigo, ainda aceito
<code>--network-alias <nome></code>	Nome adicional no DNS da rede
<code>-p</code> / <code>--publish</code>	Publica uma porta do contêiner em uma porta do host
<code>--expose <porta></code>	Equivala ao <code>EXPOSE</code> na execução: só documenta, não publica
<code>--hostname</code> / <code>-h</code>	Hostname do contêiner (namespace UTS)
<code>--dns</code> , <code>--dns-search</code> , <code>--dns-option</code>	Servidores DNS, domínios de busca e opções do <code>resolv.conf</code> do contêiner
<code>--add-host nome:ip</code>	Entrada extra no <code>/etc/hosts</code> do contêiner
<code>--ip</code> , <code>--ip6</code>	IP fixo (só em redes definidas pelo usuário com sub-rede declarada)
<code>--mac-address</code>	Endereço MAC da interface
<code>--link <contêiner>:<alias></code>	Legado: ligação entre contêineres na bridge padrão, com variáveis de ambiente e entrada no <code>/etc/hosts</code> . Substituído pelas redes definidas pelo usuário

- Não existe `--default-gateway` no `docker run`: o gateway é o da rede (definido com `docker network create --gateway`), e quem define o gateway padrão da bridge `docker0` é o daemon (capítulo 1).
- **Etapas ao conectar um contêiner a uma rede bridge:** o Docker cria um **par veth**, liga a ponta do host (`vethXXXX`) à bridge, move a outra ponta para o namespace de rede do contêiner com o nome `eth0`, define o MAC, aloca um **IP da sub-rede** da bridge e configura a rota padrão e o DNS.
- No **Linux**, o host alcança o IP interno do contêiner diretamente (`curl http://172.17.0.3`). No **Docker Desktop** não, porque os contêineres estão dentro da VM: use as portas publicadas.

6.2 Portas, firewall e acesso ao host

6.2.1 Publicação de portas

Sintaxe	Efeito
<code>-p 8080:80</code>	Porta 8080 de todas as interfaces do host (IPv4 e IPv6) → porta 80 do contêiner
<code>-p 127.0.0.1:8080:80</code>	Só no loopback do host: acessível apenas localmente
<code>-p 80</code>	Porta 80 do contêiner em uma porta aleatória do host
<code>-P</code>	Publica todas as portas do <code>EXPOSE</code> em portas aleatórias
<code>-p 5353:53/udp</code>	Porta UDP
<code>-p 8000-8010:8000-8010</code>	Faixa de portas

- `EXPOSE` **não publica nada**; ele só documenta e alimenta o `-P`.
- Contêineres na **mesma rede** se comunicam pela porta interna, **sem** `-p`. Publicar a porta do banco "para a API acessar" é desnecessário e expõe o banco.
- Porta já em uso no host → o `docker run` falha com código **125** (`bind: address already in use`).

6.2.2 Docker e o firewall do host

- No Linux, o Docker cria regras de **iptables** (ou nftables, experimental no Engine 29) para NAT e portas publicadas. Essas regras são avaliadas **antes** das regras de ferramentas como **UFW** e **firewalld**: uma porta publicada com `-p 5432:5432` fica **acessível da internet mesmo com o UFW bloqueando a porta**.
- Formas corretas de restringir: publicar só no `127.0.0.1` (atrás de um proxy reverso), não publicar portas que não precisam, ou adicionar regras na chain `DOCKER-USER`, que o Docker reserva para regras do administrador.
- Desde o Engine 28, portas **não publicadas** de contêineres em redes bridge deixaram de ser alcançáveis diretamente por outros hosts da LAN.

6.2.3 Como a publicação de portas funciona por dentro

Não há mágica: o Docker programa o **netfilter** do kernel. Para `docker run -d -p 8080:80 --name web nginx`, com o contêiner em `172.17.0.3`:

```
tabela nat
PREROUTING / OUTPUT → chain DOCKER
  DNAT  tcp dpt:8080  to:172.17.0.3:80      # entrada: porta do host → contêiner
POSTROUTING
  MASQUERADE  172.17.0.0/16 → fora      # saída: contêineres usam o IP do host
tabela filter
FORWARD → chains do Docker (DOCKER, DOCKER-USER, isolamento entre redes)
ACCEPT  tcp → 172.17.0.3 dpt:80      # libera o tráfego encaminhado
```

- DNAT** (NAT de destino) troca `IP-do-host:8080` por `172.17.0.3:80`; **MASQUERADE** (NAT de origem) faz o tráfego de saída dos contêineres usar o IP do host.
- As chains de **isolamento** impedem que redes bridge diferentes se falem; a `DOCKER-USER` é avaliada antes das regras do Docker e é o lugar das regras do administrador. Os nomes exatos das chains mudaram ao longo das versões (o Engine 28 reorganizou a tabela filter), então consulte a sua versão com `sudo iptables -S` e `sudo iptables -t nat -S`.
- Além das regras, o Docker pode iniciar o `docker-proxy` (userland proxy) para cada porta publicada, que atende conexões vindas do próprio host via `localhost`. Pode ser desligado com `"userland-proxy": false` no `daemon.json`.

6.2.4 Acessar o host a partir do contêiner

- Docker Desktop**: use o nome `host.docker.internal`.
- Linux**: `docker run --add-host=host.docker.internal:host-gateway ...` (ou `extra_hosts` no Compose). `localhost` dentro do contêiner é o **próprio contêiner**, não o host.
- O serviço no host precisa escutar em uma interface alcançável pela bridge (não só em `127.0.0.1`).

6.3 Redes entre hosts

6.3.1 Overlay e redes de camada 2

- Overlay** conecta contêineres em vários daemons de um Swarm, encapsulando o tráfego em **VXLAN**. Redes overlay criadas com `--attachable` aceitam contêineres avulsos (`docker run`), além de serviços.
- `--opt encrypted` cifra o tráfego de dados da overlay com IPsec (o tráfego de controle do Swarm já é cifrado). Tem custo de desempenho.

- **Portas necessárias entre os nós do Swarm:** **2377/tcp** (gerenciamento do cluster), **7946/tcp e udp** (descoberta e gossip entre nós) e **4789/udp** (dados da overlay, VXLAN).
- **macvlan:** por padrão, o **host não consegue falar** com os próprios contêineres macvlan (limitação do kernel); é preciso criar uma interface macvlan auxiliar no host. Muitas redes Wi-Fi e provedores de nuvem não aceitam vários MACs por interface.

6.3.2 Números de rede

Item	Valor
Sub-rede padrão da bridge <code>docker0</code>	<code>172.17.0.0/16</code>
Endereço do DNS embutido	<code>127.0.0.11</code>
Gerenciamento do Swarm	2377/tcp
Descoberta entre nós do Swarm	7946/tcp e 7946/udp
Tráfego da overlay (VXLAN)	4789/udp
Chain de iptables para regras do administrador	<code>DOCKER-USER</code>
Nome do host visto do contêiner (Docker Desktop)	<code>host.docker.internal</code>

6.4 Diagnóstico de rede

6.4.1 Ferramentas e sequência

1. **O contêiner está na rede certa?** `docker network inspect <rede>` lista os contêineres e IPs; `docker inspect -f '{{json .NetworkSettings.Networks}}' <c>`.
2. **O nome resolve?** De um contêiner na mesma rede: `docker run --rm --network app nicolaka/netshoot nslookup api`.
3. **A porta responde internamente?** `curl http://api:3000/health` a partir do `netshoot`.
4. **A aplicação escuta em 0.0.0.0?** Uma aplicação que escuta só em `127.0.0.1` **dentro do contêiner** não recebe conexões vindas de fora dele, nem por porta publicada. Esse é um dos erros mais comuns.
5. **A porta está publicada?** `docker port <c>`; confira se o mapeamento é `host:contêiner` e não o contrário.
6. **Há firewall ou conflito de sub-rede?** Redes do Docker sobrepostas à VPN ou à rede corporativa quebram rotas; ajuste `default-address-pools`.

6.5 Decisão rápida — Redes

Situação	Abordagem recomendada	Erro comum
API precisa falar com o banco no mesmo host	Rede bridge definida pelo usuário e o nome do contêiner/serviço	IP fixo do contêiner ou <code>--link</code>
Banco não deve ser acessível de fora	Não publicar a porta; rede <code>--internal</code> se não precisar de saída	<code>-p 5432:5432</code>
Painel administrativo só para acesso local	<code>-p 127.0.0.1:8080:80</code>	Confiar no UFW
Porta publicada ignora o UFW	Regras na chain <code>DOCKER-USER</code> ou bind em <code>127.0.0.1</code>	Desligar o iptables do Docker
Contêiner precisa acessar serviço do host	<code>host.docker.internal</code> (+ <code>host-gateway</code> no Linux)	<code>localhost</code>

Situação	Abordagem recomendada	Erro comum
Conexão recusada mesmo com -p correto	Fazer a aplicação escutar em 0.0.0.0	Trocar a porta do host
Máximo desempenho de rede no Linux	<code>--network host</code>	Bridge com muitas portas publicadas
Contêineres em hosts diferentes se falando	Overlay (Swarm) ou orquestrador	Publicar portas e usar o IP dos hosts
Aplicação legada precisa de IP na LAN física	macvlan ou ipvlan	<code>--network host</code>
Depurar rede de imagem sem ferramentas	nicolaka/netshoot com <code>--network container:<c></code>	Instalar pacotes no contêiner em produção
Redes do Docker conflitam com a VPN	<code>default-address-pools</code>	Desligar a VPN

7. Armazenamento e Dados

Ideia central: a camada gravável do contêiner é **efêmera** (some com o `docker rm`) e **lenta** para escrita intensa. Dados que precisam sobreviver ao contêiner vão para **volumes** (gerenciados pelo Docker) ou **bind mounts** (diretório do host). Dados temporários sensíveis podem ir para **tmpfs**.

7.1 Onde os dados ficam

7.1.1 Camada gravável e storage

- Cada contêiner recebe uma **camada gravável** sobre as camadas da imagem. No Engine 29, instalações novas usam o **containerd image store** com **snapshotters** (em geral `overlayfs`); instalações antigas podem continuar com o storage driver `overlay2`. `docker info` mostra qual está em uso.
- Trocar entre o image store clássico e o do containerd **não migra** imagens e contêineres: eles ficam invisíveis até voltar a configuração (e precisam ser baixados ou reconstruídos no novo store).
- Escrever muito na camada gravável (logs, bancos, uploads) é lento por causa do copy-on-write e faz o contêiner crescer sem controle. Use volumes.

7.1.2 Storage drivers: passado e presente

O **storage driver** define como o Engine monta as camadas e grava na camada do contêiner. Materiais antigos dedicam bastante espaço a drivers que já não existem.

Driver	Como funciona	Situação em 2026
AUFS	Union filesystem em nível de arquivo ; foi o primeiro driver do Docker. Copia o arquivo inteiro na primeira escrita e procura cada diretório do caminho em todas as camadas (lento com muitas camadas e muitos <code>import</code>)	Obsoleto no 19.03 e removido no Engine 24
devicemapper	Thin provisioning em nível de bloco (framework do kernel usado por LVM). Criado pela Red Hat para distribuições sem AUFS. Em modo <code>loop-lvm</code> era lento; <code>direct-lvm</code> exigia configurar um dispositivo dedicado	Obsoleto no 18.09 e removido no Engine 25
overlay (v1)	OverlayFS com só duas camadas por montagem; consumia muitos inodes	Obsoleto no 18.09 e removido no Engine 24
overlay2	OverlayFS com suporte nativo a várias camadas , compartilhamento de cache de páginas e cópia do arquivo só uma vez	Driver clássico recomendado ; continua em instalações existentes
btrfs / zfs	Snapshots copy-on-write em nível de bloco do próprio sistema de arquivos	Suportados apenas quando <code>/var/lib/docker</code> está em Btrfs ou ZFS; casos específicos
fuse-overlayfs	Overlay em espaço de usuário	Usado no modo rootless em kernels antigos
vfs	Sem copy-on-write: cópia completa por camada	Só para testes
containerd snapshotters (<code>overlayfs</code> e outros)	O containerd gerencia imagens e camadas	Padrão em instalações novas do Engine 29

- Trocar o storage driver** (ou o image store) torna **inacessíveis** as imagens, os contêineres e os volumes anônimos criados com o anterior: eles ficam no disco, mas o daemon não os enxerga. Faça backup, exporte o que precisa (`docker save`, backup de volumes) e recrie depois da troca.

- `docker info --format '{{.Driver}}'` mostra o driver em uso.

7.1.3 Tipos de montagem

Tipo	Origem dos dados	Gerenciado pelo Docker	Uso típico
Volume nomeado	Área do Docker (<code>/var/lib/docker/volumes/<nome>/_data</code> no Linux)	Sim	Bancos de dados, dados de aplicação em produção
Volume anônimo	Igual, com nome aleatório	Sim	Criado por <code>VOLUME</code> no Dockerfile ou <code>-v /caminho</code> ; difícil de rastrear
Bind mount	Qualquer caminho do host	Não	Código-fonte em desenvolvimento, arquivos de configuração, sockets
tmpfs	Memória RAM do host	—	Dados temporários ou sensíveis que não devem tocar o disco (Linux)
Image mount	Conteúdo de outra imagem, somente leitura	Sim	Montar dados ou ferramentas empacotados como imagem (Engine 28+)

7.1.4 `-v` VS. `--mount`

```
# Volume nomeado
docker run -v dados-pg:/var/lib/postgresql/data postgres:17
docker run --mount type=volume,src=dados-pg,dst=/var/lib/postgresql/data postgres:17

# Bind mount somente leitura
docker run -v "$(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro" nginx
docker run --mount type=bind,src="$(pwd)/nginx.conf",dst=/etc/nginx/nginx.conf,readonly nginx

# tmpfs
docker run --mount type=tmpfs,dst=/tmp,tmpfs-size=64m app
```

- `--mount` é mais explícito e recomendado em scripts. Diferença importante: com `-v`, um bind mount de caminho **inexistente cria um diretório vazio** no host (o que causa erros confusos, como um diretório no lugar de um arquivo de configuração); com `--mount`, o comando **falha** com erro claro.
- Sufixos do `-v`: `:ro` (somente leitura), `:z` / `:Z` (rótulo SELinux compartilhado/privado).
- **Montar um único arquivo** também funciona: `--mount type=bind,src=/etc/app/config.yaml,dst=/app/config.yaml,readonly`. Uma escrita em um ponto de montagem somente leitura falha com `Read-only file system`.

7.1.5 Características dos volumes

- Um volume **não passa pelo union filesystem**: leituras e escritas vão direto ao diretório do volume, sem copy-on-write.
- É criado (ou reaproveitado) quando o contêiner é criado e **continua existindo** depois que o contêiner é removido.
- Pode ser **reutilizado e compartilhado** entre contêineres.
- **Não entra na imagem**: `docker commit`, `docker export` e `docker save` ignoram o conteúdo dos volumes montados.

- **Onde fica no host:** `docker volume inspect --format '{{.Mountpoint}}' meu-volume` retorna `/var/lib/docker/volumes/meu-volume/_data` no Linux. No Docker Desktop, esse caminho fica **dentro da VM**, não no macOS ou no Windows.
- **Bind mounts dependem do host:** o diretório precisa existir, com as permissões certas, em **cada** host onde o contêiner rodar. Em um cluster isso vira um problema (o contêiner pode ser agendado em outro nó). Volumes nomeados, com driver compartilhado quando necessário, deixam a aplicação portátil.

7.1.6 Padrão antigo: data-only container e `--volumes-from`

- Antes do comando `docker volume` (Docker 1.9, 2015), o jeito de ter um volume "com nome" era criar um **contêiner só de dados**, que nunca rodava, e montar os volumes dele em outros contêineres com `--volumes-from`:

```
# Padrão antigo (ainda funciona, mas não é mais necessário)
docker container create -v /data --name dbdados alpine
docker run -d --volumes-from dbdados app
```

```
# Equivalente atual: volume nomeado
docker volume create dbdados
docker run -d --mount type=volume,src=dbdados,dst=/data app
```

- `--volumes-from <contêiner>` monta **todos** os volumes de outro contêiner, nos mesmos caminhos. Ainda é útil para **backup** rápido dos volumes de um contêiner existente: `docker run --rm --volumes-from app -v "$(pwd)":/backup alpine tar czf /backup/app.tgz /data`.
- **Nunca rode duas instâncias de banco de dados sobre o mesmo diretório de dados** (dois PostgreSQL ou dois MySQL montando o mesmo volume). Cada instância supõe ser a dona exclusiva dos arquivos: o PostgreSQL recusa iniciar ao encontrar o `postmaster.pid` da outra ou, se forçado, **corrompe os dados**. Para ter duas instâncias, use volumes separados e replicação do próprio banco.

7.2 Comportamento e permissões

7.2.1 Pré-população de volumes

- Ao montar um **volume vazio** em um diretório que já tem arquivos na imagem, o Docker **copia** esses arquivos para o volume na primeira vez (a menos que se use `volume-nocopy`).
- **Bind mounts não pré-populam:** montar um diretório do host **esconde** o conteúdo da imagem naquele caminho. Esse é o motivo clássico de "sumiu o `node_modules`" ao montar o código-fonte em `/app` no desenvolvimento. Solução comum: um volume (anônimo ou nomeado) em `/app/node_modules` por cima do bind mount, ou o `develop.watch` do Compose (capítulo 8).

7.2.2 Permissões e UIDs

- O contêiner vê **UIDs e GIDs numéricos**, não nomes. Se a aplicação roda como UID 10001 e o bind mount pertence ao UID 1000 do host, a escrita falha com `permission denied`.
- Soluções: rodar o contêiner com o UID do dono (`-u $(id -u):$(id -g)` em desenvolvimento), ajustar a posse dos arquivos no build (`COPY --chown`) e, para volumes, deixar a imagem criar o diretório com o dono certo antes da montagem (a pré-população copia as permissões).
- Com **SELinux** (Fedora, RHEL), bind mounts podem exigir `:z` ou `:Z`.

- No **modo rootless** e com `usersns-remap`, o root do contêiner é mapeado para um UID alto do host; arquivos criados em bind mounts aparecem com esse UID.

7.2.3 Backup, restauração e drivers

```
# Backup de um volume para um tarball no diretório atual
docker run --rm -v dados-pg:/origem:ro -v "$(pwd)":/backup alpine \
  tar czf /backup/dados-pg.tgz -C /origem .

# Restauração em um volume novo
docker volume create dados-pg-restaurado
docker run --rm -v dados-pg-restaurado:/destino -v "$(pwd)":/backup alpine \
  tar xzf /backup/dados-pg.tgz -C /destino
```

- Para bancos de dados, prefira a ferramenta de backup do próprio banco (`pg_dump`, `mysqldump`, `snapshots`) em vez de copiar arquivos com o banco rodando.
- Copiar `/var/lib/docker/volumes` inteiro em rotinas de backup do host funciona só com os contêineres **parados** (ou com snapshot do sistema de arquivos). Com aplicações gravando, a cópia pode ficar inconsistente.
- Drivers de volume:** o driver `local` aceita opções de montagem, inclusive **NFS** e **CIFS** (`docker volume create --driver local --opt type=nfs --opt o=addr=10.0.0.5,rw --opt device=:/export nfs-dados`). Plugins de terceiros integram storage de nuvem ou distribuído.
- Volume compartilhado entre contêineres:** vários contêineres podem montar o mesmo volume, mas o Docker **não coordena** escrita concorrente; isso é responsabilidade da aplicação.

7.2.4 Comandos de volumes

Comando	O que faz
<code>docker volume create <nome></code>	Cria um volume
<code>docker volume ls / ls -f dangling=true</code>	Lista volumes / os que não estão em uso
<code>docker volume inspect <nome></code>	Mostra driver, opções e <code>Mountpoint</code>
<code>docker volume rm <nome></code>	Remove (falha se estiver em uso)
<code>docker volume prune / prune -a</code>	Remove anônimos não usados / todos os não usados
<code>docker run --rm</code>	Remove também os volumes anônimos do contêiner (não os nomeados)

7.3 Decisão rápida — Armazenamento

Situação	Abordagem recomendada	Erro comum
Dados de banco em produção	Volume nomeado (ou storage gerenciado) + backup pela ferramenta do banco	Camada gravável do contêiner
Código-fonte com recarga automática em desenvolvimento	Bind mount ou <code>develop.watch</code> do Compose	Rebuild a cada alteração
Arquivo de configuração do host	Bind mount somente leitura (<code>:ro</code>)	Copiar para dentro com <code>docker cp</code>
Segredo temporário que não pode ir para o disco	<code>tmpfs</code> (ou <code>secrets</code> do Compose/Swarm)	Variável de ambiente logada

Situação	Abordagem recomendada	Erro comum
Bind mount criou um diretório no lugar do arquivo	Usar <code>--mount</code> (falha se a origem não existe) e corrigir o caminho	Criar o arquivo dentro do diretório
<code>node_modules</code> sumiu ao montar o código	Volume em <code>/app/node_modules</code> por cima do bind mount	Instalar dependências no host
<code>permission denied</code> em bind mount	Alinhar UID/GID (<code>-u</code> , <code>--chown</code>) ou <code>:z</code> no SELinux	<code>chmod 777</code>
Compartilhar dados entre hosts	Volume com driver NFS/plugin ou storage de objetos	Volume <code>local</code> achando que replica
Backup de um volume	Contêiner temporário com <code>tar</code> montando o volume	Copiar <code>/var/lib/docker/volumes</code> com o daemon rodando
Limpeza sem perder dados importantes	<code>docker volume prune</code> (só anônimos) e revisar antes de <code>-a</code>	<code>docker system prune -a --volumes</code> em produção

8. Docker Compose

Ideia central: o Compose descreve uma **aplicação multi-contêiner** em um arquivo YAML (`compose.yaml`) e a gerencia como um **projeto**: serviços, redes, volumes, segredos e configurações sobem, são atualizados e descem juntos. É a ferramenta padrão para desenvolvimento local, testes de integração e implantações simples em um único host.

8.1 Modelo e comandos

8.1.1 Projeto, arquivos e nomes

- **Comando:** `docker compose` (plugin da CLI, versão 5.x em 2026). O antigo `docker-compose` (v1, Python) está fora de suporte desde 2023.
- **Arquivo padrão:** `compose.yaml` (também aceita `compose.yml`, `docker-compose.yaml` e `docker-compose.yml`). Se existir `compose.override.yaml`, ele é mesclado automaticamente por cima.
- **Nome do projeto** (prefixo de contêineres, redes e volumes), por ordem de precedência: flag `-p`, variável `COMPOSE_PROJECT_NAME`, atributo `name:` no topo do arquivo e, por fim, o nome do diretório.
- **Nomes gerados:** contêineres `<projeto>-<serviço>-<n>`, rede padrão `<projeto>_default`, volumes `<projeto>_<volume>`.
- **Rede padrão:** todos os serviços entram em uma rede bridge do projeto e se encontram pelo **nome do serviço** (`db`, `redis`).
- O **campo** `version:` é obsoleto e só gera aviso.

8.1.2 Exemplo completo

```
name: loja

services:
  web:
    build:
      context: ./web
      target: runtime
    image: ghcr.io/org/loja-web:${TAG:-dev}
    ports:
      - "127.0.0.1:8080:3000"
    environment:
      DATABASE_URL: postgres://loja@db:5432/loja
      NODE_ENV: ${NODE_ENV:-production}
    env_file: .env.web
    depends_on:
      db:
        condition: service_healthy
        restart: true
      migracao:
        condition: service_completed_successfully
    secrets:
      - db_senha
    networks: [frontend, backend]
    restart: unless-stopped
    deploy:
      resources:
        limits: { cpus: "1.0", memory: 512M }

  migracao:
    image: ghcr.io/org/loja-web:${TAG:-dev}
    command: ["npm", "run", "migrate"]
    depends_on:
      db:
        condition: service_healthy
    networks: [backend]

  db:
    image: postgres:17
    environment:
      POSTGRES_USER: loja
      POSTGRES_PASSWORD_FILE: /run/secrets/db_senha
    secrets:
      - db_senha
    volumes:
      - dados-db:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U loja"]
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 20s
    networks: [backend]

  adminer:
    image: adminer
    profiles: [debug]
    ports: ["127.0.0.1:8081:8080"]
    networks: [backend]
```

```

networks:
  frontend:
  backend:
    internal: true

volumes:
  dados-db:

secrets:
  db_senha:
    file: ./secrets/db_senha.txt

```

8.1.3 Comandos do dia a dia

Comando	O que faz
<code>docker compose up -d</code>	Cria e inicia tudo em segundo plano; recria só os serviços cuja configuração ou imagem mudou
<code>docker compose up -d --build</code>	Reconstrói as imagens antes de subir
<code>docker compose up -d --wait</code>	Espera os serviços ficarem <code>running</code> / <code>healthy</code> antes de retornar (útil em CI)
<code>docker compose down</code>	Para e remove contêineres e redes do projeto. Preserva volumes nomeados
<code>docker compose down -v</code>	Remove também os volumes nomeados do projeto e os anônimos. Apaga dados
<code>docker compose stop / start</code>	Para e inicia sem remover
<code>docker compose ps / ls</code>	Serviços do projeto / projetos em execução
<code>docker compose logs -f web</code>	Logs de um ou mais serviços
<code>docker compose exec web sh</code>	Processo novo no contêiner em execução
<code>docker compose run --rm web npm test</code>	Contêiner novo e avulso do serviço, para tarefas pontuais
<code>docker compose build / pull / push</code>	Constrói (via Bake no v5), baixa ou envia as imagens
<code>docker compose config</code>	Mostra o modelo resolvido (interpolação, merge de arquivos, perfis). Primeiro passo para depurar
<code>docker compose up -d --scale worker=3</code>	Várias réplicas de um serviço (sem <code>container_name</code> nem porta fixa no host)
<code>docker compose watch</code>	Sincroniza e reconstrói automaticamente em desenvolvimento
<code>docker compose --profile debug up</code>	Inclui serviços do perfil <code>debug</code>

8.2 Ordem de inicialização, variáveis e composição de arquivos

8.2.1 `depends_on` e `healthchecks`

Condição	Espera até
<code>service_started</code> (padrão na sintaxe curta)	O contêiner da dependência iniciar (não significa pronto)
<code>service_healthy</code>	O healthcheck da dependência ficar <code>healthy</code>
<code>service_completed_successfully</code>	A dependência terminar com código 0 (migrações, seeds)

- `restart: true` dentro do `depends_on` reinicia o serviço dependente quando a dependência for atualizada pelo Compose.

- `required: false` torna a dependência opcional.
- `depends_on` só controla a **ordem no up**. Se o banco cair depois, a aplicação precisa tratar reconexão; nenhum orquestrador substitui isso.

8.2.2 Variáveis de ambiente e interpolação

Há **dois mecanismos diferentes**, que costumam ser confundidos:

Mecanismo	Para que serve	Onde
Interpolação <code>\${VAR}</code>	Substituir valores no arquivo Compose antes de aplicá-lo	Lê do shell e do arquivo <code>.env</code> do diretório do projeto (ou <code>--env-file</code>)
<code>environment / env_file</code>	Definir variáveis dentro do contêiner	No serviço

- Sintaxes de interpolação: `${VAR:-padrão}` (padrão se vazia ou ausente), `${VAR-padrão}` (só se ausente), `${VAR:?mensagem}` (erro se vazia ou ausente), `$$` para um `$` literal.
- **Precedência dentro do contêiner** (da maior para a menor): `docker compose run -e`, atributo `environment`, atributo `env_file` e, por último, o `ENV` da imagem.
- O `.env` **não é injetado** automaticamente nos contêineres; ele só alimenta a interpolação. Para levar variáveis ao contêiner, referencie-as em `environment` ou use `env_file`.

8.2.3 Segredos e configs

- `secrets`: montados como arquivo em `/run/secrets/<nome>`, a partir de um arquivo (`file:`) ou de uma variável de ambiente do host (`environment:`). Muitas imagens oficiais aceitam variáveis `*_FILE` (como `POSTGRES_PASSWORD_FILE`) para ler o segredo do arquivo.
- `configs`: mesma ideia para arquivos de configuração não sensíveis.
- Vantagem sobre variáveis de ambiente: segredos em arquivo não aparecem em `docker inspect`, em dumps de ambiente nem em logs de erro que imprimem o ambiente.

8.2.4 Combinar arquivos: `override`, `-f`, `extends` e `include`

Recurso	Como funciona	Uso
<code>compose.override.yaml</code>	Mesclado automaticamente por cima do <code>compose.yaml</code>	Ajustes de desenvolvimento (bind mounts, portas de debug)
<code>-f base.yaml -f prod.yaml</code>	Mescla na ordem; o último vence em valores simples, listas como <code>ports</code> são concatenadas	Variações por ambiente
<code>extends</code>	Um serviço herda a definição de outro (mesmo arquivo ou outro)	Reaproveitar configuração comum
<code>include</code>	Importa outro projeto Compose inteiro, com caminhos relativos ao arquivo incluído	Aplicações com partes mantidas por times diferentes
Fragmentos YAML e <code>x-</code>	Âncoras (<code>&comum</code> , <code><<: *comum</code>) em chaves de extensão <code>x-...</code>	Evitar repetição no mesmo arquivo

8.2.5 Perfis

- Serviços com `profiles: [debug]` **só sobem** quando o perfil é ativado (`--profile debug` ou `COMPOSE_PROFILES=debug`). Serviços sem `profiles` sobem sempre.
- Uso típico: ferramentas de administração, mocks, serviços de observabilidade opcionais.

8.3 Fluxo de desenvolvimento

8.3.1 develop.watch

```
services:
  web:
    build: .
    develop:
      watch:
        - action: sync
          path: ./src
          target: /app/src
        - action: rebuild
          path: package.json
        - action: sync+restart
          path: ./config
          target: /app/config
```

Ação	Efeito ao detectar mudança
sync	Copia os arquivos para o contêiner em execução (ideal com recarga automática do framework)
rebuild	Reconstrói a imagem e recria o contêiner (ex.: dependências mudaram)
sync+restart	Copia os arquivos e reinicia o contêiner (ex.: configuração lida só na inicialização)
sync+exec	Copia e executa um comando no contêiner

- Rode com `docker compose watch` ou `docker compose up --watch`. Diferente do bind mount, o watch respeita o `.dockerignore`, funciona bem em sistemas de arquivos de VMs (Docker Desktop) e não esconde `node_modules`.

8.3.2 Outros recursos úteis

- `docker init`: gera `Dockerfile`, `compose.yaml` e `.dockerignore` iniciais com boas práticas para a linguagem detectada (Node, Python, Go, Java, Rust, .NET, PHP).
- **Hooks de ciclo de vida**: `post_start` e `pre_stop` executam comandos no contêiner depois de iniciar e antes de parar (ex.: ajustar permissões em um volume).
- `init: true`: usa o `tiny` como PID 1 (o mesmo que `--init`).
- `stop_grace_period: 30s`: tempo entre o sinal de parada e o `SIGKILL`.
- `deploy.resources.limits` e `deploy.replicas` são respeitados também fora do Swarm.
- **Compose em produção** faz sentido em **um único host** (pequenas aplicações, VMs de borda). Para vários hosts, alta disponibilidade e rolling updates, use Swarm ou Kubernetes (capítulo 11).

8.4 Decisão rápida — Compose

Situação	Abordagem recomendada	Erro comum
API sobe antes do banco aceitar conexões	<code>depends_on</code> com <code>condition: service_healthy</code> + healthcheck no banco	<code>sleep 30</code> no entrypoint
Rodar migrações antes da aplicação	Serviço de migração + <code>service_completed_successfully</code>	Migração no entrypoint de cada réplica

Situação	Abordagem recomendada	Erro comum
Ver o arquivo final depois de variáveis e overrides	<code>docker compose config</code>	Adivinhar a mesclagem
Variável do <code>.env</code> não aparece no contêiner	Referenciar em <code>environment</code> ou usar <code>env_file</code>	Esperar que o <code>.env</code> seja injetado
Senha do banco no Compose	<code>secrets</code> + variáveis <code>*_FILE</code>	Senha em <code>environment</code> versionada
Diferenças entre desenvolvimento e produção	<code>compose.override.yaml</code> ou <code>-f</code> por ambiente	Dois arquivos inteiros duplicados
Ferramenta de administração opcional	<code>profiles</code>	Comentar e descomentar serviços
Derrubar o ambiente sem perder o banco	<code>docker compose down</code>	<code>docker compose down -v</code>
Recarga rápida de código em desenvolvimento	<code>docker compose watch</code>	Rebuild manual a cada mudança
Tarefa pontual (testes, shell) com a configuração do serviço	<code>docker compose run --rm <serviço> <comando></code>	<code>docker run</code> repetindo todas as flags
Aplicação em vários hosts com alta disponibilidade	Swarm ou Kubernetes	Compose em cada host sincronizado à mão
Começar a containerizar um projeto existente	<code>docker init</code> e revisão do resultado	Copiar um Dockerfile genérico da internet

9. Segurança

Ideia central: segurança em Docker tem três frentes. **O host e o daemon:** quem controla o daemon controla o host. **O contêiner em execução:** menos privilégios, menos capabilities, sistema de arquivos somente leitura, sem root. **A cadeia de suprimentos:** imagens mínimas, conhecidas, escaneadas, assinadas e fixadas por digest. Nenhuma camada substitui as outras.

9.1 Proteger o host e o daemon

9.1.1 Superfície de ataque do daemon

- O `dockerd` tradicional roda como **root**. Acesso ao socket `/var/run/docker.sock` (grupo `docker`, bind mount do socket, API por TCP) permite `docker run -v /:/host --privileged ...` e, portanto, **root no host**.
- Montar o `docker.sock` em um contêiner** (Traefik, Portainer, runners de CI) dá a esse contêiner controle total do host, **mesmo com `:ro`** (o somente leitura vale para o arquivo, não para a API). Se for inevitável, use um **proxy de socket** que libere só os endpoints necessários, ou restrinja o que roda ali.
- API remota:** só por SSH ou TLS mútuo (2376). Nunca a porta 2375 exposta.
- Mantenha o Engine atualizado: correções de `runc` e `containerd` (fugas de contêiner) chegam por atualização.

9.1.2 Modo rootless e `userns-remap`

Recurso	Como funciona	Limitações
Rootless	O daemon e os contêineres rodam como um usuário comum, dentro de um user namespace (<code>dockerd-rootless-setup tool.sh install</code>)	Portas abaixo de 1024 exigem ajuste de <code>sysctl</code> ; rede com desempenho um pouco menor; alguns drivers e opções indisponíveis; um daemon por usuário
<code>userns-remap</code>	O daemon continua root, mas o root do contêiner é mapeado para um UID alto sem privilégio no host	Afeta permissões de bind mounts; incompatível com alguns recursos (<code>--privileged</code> , <code>--network host</code> com certas opções)
Enhanced Container Isolation	Recurso do Docker Desktop Business que roda os contêineres com user namespace e bloqueia acesso ao socket do Docker	Só Docker Desktop, assinatura Business

- Com qualquer um deles, uma fuga do contêiner cai em um usuário sem privilégios no host, em vez de root.

9.2 Endurecer o contêiner em execução

9.2.1 Capabilities

O root do Linux é dividido em **capabilities**. Por padrão, o Docker concede um conjunto reduzido de **14**: `CHOWN`, `DAC_OVERRIDE`, `FOWNER`, `FSETID`, `KILL`, `SETGID`, `SETUID`, `SETPCAP`, `SETFCAP`, `NET_BIND_SERVICE`, `NET_RAW`, `SYS_CHROOT`, `MKNOD` e `AUDIT_WRITE`.

- Remova tudo e adicione só o necessário:** `--cap-drop ALL --cap-add NET_BIND_SERVICE`.
- Capabilities perigosas que costumam ser pedidas sem necessidade: `SYS_ADMIN` (quase root), `NET_ADMIN`, `SYS_PTRACE`, `SYS_MODULE`.

- `--privileged` concede **todas** as capabilities, acesso a todos os dispositivos do host e desliga seccomp e AppArmor. Equivale a rodar sem isolamento. Use apenas em casos muito específicos (Docker-in-Docker, drivers) e nunca para "resolver" um erro de permissão.

9.2.2 Controles de segurança em execução

Controle	Flag	Efeito
Usuário não root	<code>-u 10001:10001</code> ou <code>USER</code> no Dockerfile	Limita o impacto de uma falha na aplicação
Sem escalada de privilégio	<code>--security-opt no-new-privileges</code>	Impede que binários <code>setuid</code> ganhem privilégios
Sistema de arquivos somente leitura	<code>--read-only + --tmpfs /tmp</code>	Impede que um invasor grave binários ou altere a aplicação
seccomp	Perfil padrão ativo; <code>--security-opt seccomp=perfil.json</code>	Bloqueia chamadas de sistema perigosas (o padrão bloqueia cerca de 44, como <code>mount</code> , <code>reboot</code> , <code>kexec_load</code>)
AppArmor / SELinux	Perfil <code>docker-default</code> / rótulos SELinux	Controle de acesso obrigatório a arquivos e recursos
Limite de processos	<code>--pids-limit 200</code>	Protege contra fork bombs
Limites de memória e CPU	<code>--memory</code> , <code>--cpus</code>	Evita que um contêiner derrube o host (capítulo 10)
Sem compartilhar namespaces do host	Evitar <code>--pid host</code> , <code>--ipc host</code> , <code>--network host</code> , <code>--userns host</code>	Mantém o isolamento
Runtime com sandbox	<code>--runtime runsc</code> (gVisor) ou Kata Containers	Kernel intermediário ou microVM para código não confiável

```
docker run -d --name api \
  --user 10001:10001 \
  --read-only --tmpfs /tmp:size=64m \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --pids-limit 200 --memory 512m --cpus 1 \
  ghcr.io/org/api@sha256:...
```

9.3 Segredos

9.3.1 Onde (não) colocar segredos

Local	Seguro?	Por quê
<code>ENV</code> ou <code>ARG</code> no Dockerfile	Não	Fica gravado na imagem e no histórico
Arquivo copiado para a imagem e apagado depois	Não	Continua na camada em que foi copiado
<code>-e SENHA=...</code> na execução	Fraco	Visível em <code>docker inspect</code> , no <code>/proc/<pid>/environ</code> , em dumps e logs de erro, e herdado por processos filhos
Build secret (<code>RUN --mount=type=secret</code>)	Sim , para o build	Não entra em camada nem no histórico
Compose/Swarm secrets em <code>/run/secrets</code>	Sim , para execução	Arquivo montado só no contêiner que precisa (no Swarm, cifrado no Raft e montado em <code>tmpfs</code>)
Cofre externo (Vault, AWS Secrets Manager, Azure Key Vault)	Sim	Rotação, auditoria e credenciais temporárias

- Imagens enviadas por engano com segredos devem ser consideradas **comprometidas**: rotacione o segredo; apagar a tag não basta.
- Ferramentas como Docker Scout, Trivy e GitGuardian detectam segredos em camadas de imagens.

9.4 Cadeia de suprimentos

9.4.1 Práticas e ferramentas

Prática	Ferramentas	Observação
Imagens base mínimas e mantidas	Docker Official Images, Docker Hardened Images , distroless, Chainguard	Menos pacotes = menos CVEs e menos superfície
Fixar por digest e atualizar automaticamente	FROM ...@sha256:..., Renovate, Dependabot	Reprodutível sem ficar parado no tempo
Escanear vulnerabilidades	Docker Scout (docker scout quickview, docker scout cves, docker scout recommendations), Trivy, Gype	Rode no CI e bloqueie severidades críticas com correção disponível
SBOM	docker buildx build --sbom=true, docker scout sbom, Syft	Saber o que existe em cada imagem quando surgir um novo CVE
Provenance (SLSA)	--provenance=mode=max	Liga a imagem ao commit e ao pipeline que a gerou
Assinatura	Sigstore/cosign (inclusive keyless, com OIDC do CI), Notation	O Docker Content Trust (Notary v1) foi aposentado
Verificar antes de executar	Políticas de admissão no Kubernetes (Kyverno, Sigstore policy-controller), políticas do Scout	Só roda imagem assinada, do registry permitido
Restringir registries	Registry Access Management (Docker Business), políticas do cluster	Evita imagens de origem desconhecida
Auditoria de configuração	CIS Docker Benchmark, docker-bench-security	Verifica daemon, host e contêineres

- **Imagens de terceiros**: prefira oficiais ou de publicadores verificados; confira o repositório de origem (org.opencontainers.image.source), a frequência de atualização e o Dockerfile.
- **Atualização contínua**: reconstrua as imagens periodicamente (mesmo sem mudança no código) para incorporar correções da base.

9.5 Decisão rápida — Segurança

Situação	Abordagem recomendada	Erro comum
Aplicação precisa escutar na porta 80 sem root	--cap-drop ALL --cap-add NET_BIND_SERVICE (ou escutar em porta alta)	Rodar como root
Contêiner "precisa de permissão" para algo	Identificar a capability ou o dispositivo exato	--privileged
Ferramenta pede o docker.sock	Avaliar o risco; proxy de socket com endpoints mínimos	Montar o socket :x0 achando que é seguro
Servidor compartilhado com vários usuários de Docker	Modo rootless	Grupo docker para todos
Impedir que um invasor altere binários	--read-only + tmpfs para diretórios temporários	Confiar só no usuário não root
Senha do banco na imagem	Secrets em tempo de execução ou cofre externo	ENV DB_PASSWORD
Token para baixar dependências no build	Build secret	ARG com o token

Situação	Abordagem recomendada	Erro comum
Segredo foi enviado em uma imagem pública	Rotacionar o segredo e reconstruir	Apagar a tag e seguir
Saber se as imagens em produção têm um CVE novo	SBOM + Docker Scout/Trivy no registry	Escanear só no build
Garantir que só imagens do pipeline rodem	Assinatura com cosign + verificação na admissão	Confiar na tag
Executar código não confiável de usuários	gVisor, Kata ou uma VM por carga	Contêiner padrão com <code>--cap-drop</code>

10. Recursos, Logs, Observabilidade e Troubleshooting

Ideia central: sem limites, um contêiner pode consumir **toda** a memória e CPU do host. Sem rotação, os logs enchem o disco. E sem logs em **STDOUT/STDERR**, `docker logs` e as ferramentas de coleta ficam cegas. Limites, rotação e saída padrão são a base da operação.

10.1 Limites de recursos

10.1.1 Memória, CPU e outros

Flag	Efeito	Observação
<code>--memory 512m</code>	Limite rígido de memória	Ao estourar, o kernel mata o processo (OOM , código 137 , <code>OOMKilled: true</code>)
<code>--memory-reservation 256m</code>	Limite flexível, aplicado quando o host está sob pressão	Deve ser menor que <code>--memory</code>
<code>--memory-swap</code>	Memória + swap. Igual a <code>--memory</code> = sem swap ; <code>-1</code> = swap ilimitado	Sem definir, o contêiner pode usar swap igual ao limite de memória (se o host tiver swap)
<code>--oom-kill-disable</code>	Impede o OOM kill	Perigoso sem <code>--memory</code> : o host pode matar outros processos
<code>--cpus 1.5</code>	Até 1,5 CPU de tempo (cota do CFS)	Forma mais simples de limitar CPU
<code>--cpu-shares 512</code>	Peso relativo (padrão 1024), só vale quando há disputa	Não é um limite
<code>--cpuset-cpus 0,1</code>	Fixa o contêiner em núcleos específicos	Cargas sensíveis a latência
<code>--pids-limit 200</code>	Máximo de processos	Proteção contra fork bomb
<code>--shm-size 256m</code>	Tamanho do <code>/dev/shm</code> (padrão 64 MB)	Navegadores headless e alguns bancos precisam de mais
<code>--ulimit nofile=65536:65536</code>	Limites do processo	Servidores com muitas conexões
<code>--gpus all</code>	Expõe GPUs NVIDIA	Exige o NVIDIA Container Toolkit no host

- **Alterar sem recriar:** `docker update --memory 1g --cpus 2 <contêiner>` (também `docker container update`, disponível desde o Docker 1.10). Aceita CPU, memória, `--pids-limit`, `--blkio-weight` e `--restart`; `docker update --help` lista as opções.

10.1.2 Na prática: dimensionar e conferir limites

- **Sem limites, um contêiner pode usar todos os recursos do host** e prejudicar os vizinhos. Defina limites por tipo de serviço. Exemplo: servidores web com `--cpus 0.5 -m 128m` e bancos com `--cpus 1 -m 256m` (valores ilustrativos; meça o consumo real com `docker stats` antes de fixar).

```
docker run -d --name web --cpus 0.5 -m 128m nginx
docker inspect -f 'Memória: {{.HostConfig.Memory}} bytes | NanoCpus: {{.HostConfig.NanoCpus}}' web
# Memória: 134217728 bytes | NanoCpus: 500000000

docker update -m 256m --memory-swap 256m --cpus 1 web
docker stats --no-stream web
```

- Os valores aparecem no `inspect` em **bytes** (`Memory`) e em **bilionésimos de CPU** (`NanoCpus`: `0.5 CPU = 5000000000`). `Memory: 0` e `NanoCpus: 0` significam **sem limite**.
- `-m` é a forma curta de `--memory` e aceita sufixos `b`, `k`, `m` e `g`. O mínimo é 6 MB.
- Ao **aumentar** `--memory` com `docker update` em um contêiner que já tem `--memory-swap`, ajuste os dois juntos: `--memory-swap` não pode ser menor que `--memory`.
- O campo `KernelMemory` (`--kernel-memory`) de saídas antigas foi descontinuado e não tem efeito com cgroup v2.
- Runtimes precisam enxergar os limites:** a JVM (10+) respeita cgroups e dimensiona o heap com `-XX:MaxRAMPercentage`; o Node.js tem heap próprio (`--max-old-space-size`); o Go ajusta o `GOMAXPROCS` ao limite de CPU desde a versão 1.25. Aplicações antigas podem enxergar a memória do host inteiro e ser mortas por OOM.

10.2 Logs

10.2.1 Drivers de log

Driver	Destino	<code>docker logs</code> funciona?	Observação
<code>json-file</code> (padrão)	Arquivo JSON no host	Sim	Sem rotação por padrão: configure <code>max-size</code> e <code>max-file</code>
<code>local</code>	Arquivo em formato otimizado e comprimido	Sim	Rotação por padrão (20 MB × 5 arquivos). Recomendado para hosts de uso geral
<code>journald</code>	systemd journal	Sim	Integra com <code>journalctl</code>
<code>syslog</code> , <code>gelf</code> , <code>fluentd</code> , <code>splunk</code>	Servidor de logs remoto	Sim, pelo cache local (dual logging)	Envio direto a plataformas de log
<code>awslogs</code> , <code>gcplogs</code>	CloudWatch Logs, Cloud Logging	Sim, pelo cache local	Usados em ECS e VMs de nuvem
<code>none</code>	Nenhum	Não	Descarta a saída

```
# Por contêiner
docker run --log-driver local --log-opt max-size=10m --log-opt max-file=3 app
# Global: log-driver e log-opts no daemon.json (vale para contêineres novos)
```

- Aplicação deve escrever em STDOUT/STDERR**, não em arquivos dentro do contêiner. Imagens oficiais como a do nginx ligam `/var/log/nginx/access.log` a `/dev/stdout`.
- Modo de entrega:** o padrão `mode=blocking` faz a aplicação **esperar** se o destino do log travar; `mode=non-blocking` com `max-buffer-size` evita isso, mas pode descartar mensagens quando o buffer enche.
- Logs estruturados (JSON) facilitam consultas no destino.

10.3 Métricas e eventos

10.3.1 Fontes de informação

- `docker stats`: CPU, memória (uso e limite), rede, disco e PIDs em tempo real. `--no-stream` para uma leitura única (útil em scripts).

- **Métricas do daemon:** `"metrics-addr": "127.0.0.1:9323"` no `daemon.json` expõe métricas Prometheus do Engine.
- **cAdvisor:** exporta métricas por contêiner para Prometheus.
- `docker events --filter event=die --filter event=oom`: acompanha mortes, OOMs, mudanças de health e reinícios.
- **Healthcheck:** `docker inspect -f '{{json .State.Health}}' <c>` mostra as últimas execuções e suas saídas.
- **Rastreamento:** a aplicação instrumentada com OpenTelemetry envia traces a um coletor que roda como outro contêiner na mesma rede.

10.4 Troubleshooting

10.4.1 Sintomas, causas e ações

Sintoma / mensagem	Causa provável	O que fazer
Cannot connect to the Docker daemon at unix:///var/run/docker.sock	Daemon parado, contexto errado ou DOCKER_HOST apontando para outro lugar	<code>systemctl status docker</code> , <code>docker context ls</code> , <code>echo \$DOCKER_HOST</code>
permission denied while trying to connect to the Docker daemon socket	Usuário sem acesso ao socket	Usar <code>sudo</code> , <code>rootless</code> ou (conscientemente) o grupo <code>docker</code> e novo login
no space left on device	Imagens, cache de build, volumes ou logs sem rotação	<code>docker system df</code> , prune seletivo, rotação de logs, <code>data-root</code> maior
Código 137 / <code>OOMKilled: true</code>	Memória acima do limite	Aumentar o limite, corrigir vazamento, ajustar heap do runtime
<code>exec format error</code>	Imagem de outra arquitetura	<code>--platform</code> correto ou imagem multiplataforma
<code>exec /entrypoint.sh: no such file or directory</code> com o arquivo presente	Finais de linha CRLF ou shebang para shell inexistente (<code>#!/bin/bash</code> no Alpine)	Converter para LF (<code>.gitattributes</code>), usar <code>#!/bin/sh</code>
<code>bind: address already in use / port is already allocated</code>	Porta do host ocupada	Outra porta no host ou parar o processo que a usa (<code>ss -ltnp</code>)
<code>toomanyrequests: You have reached your pull rate limit</code>	Limite do Docker Hub	Autenticar, mirror, cache de imagens
<code>manifest unknown / no matching manifest for linux/arm64</code>	Tag inexistente ou sem a plataforma pedida	Conferir a tag e as plataformas com <code>imagetools inspect</code>
<code>unauthorized / denied: requested access to the resource is denied</code>	Sem login, token expirado ou repositório errado	<code>docker login</code> , conferir o nome completo e as permissões
Contêiner em <code>Restarting</code> sem parar	Processo falha ao iniciar e a política reinicia	<code>docker logs</code> , <code>docker inspect</code> , rodar com <code>--entrypoint sh</code>
Contêiner saudável mas inacessível pela porta publicada	Aplicação escuta em <code>127.0.0.1</code> dentro do contêiner	Escutar em <code>0.0.0.0</code>
Resolução de nomes falha dentro do contêiner	DNS corporativo, VPN ou <code>resolv.conf</code> do host	<code>dns</code> no <code>daemon.json</code> ou <code>--dns</code>
Build não reflete a alteração	Cache de <code>RUN</code> com comando idêntico ou arquivo ignorado pelo <code>.dockerignore</code>	<code>--no-cache</code> pontual, revisar a ordem e o <code>.dockerignore</code>
Bind mounts lentos no macOS/Windows	Sincronização de arquivos entre o host e a VM	<code>develop.watch</code> , <code>synchronized file shares</code> , código dentro do WSL 2

10.4.2 Números de operação

Item	Valor
Tempo de tolerância do <code>docker stop</code>	10 s
Sinal de parada padrão	SIGTERM
Tamanho padrão do <code>/dev/shm</code>	64 MB
Peso padrão de <code>--cpu-shares</code>	1024
Rotação padrão do driver <code>local</code>	20 MB × 5 arquivos
Rotação padrão do <code>json-file</code>	Nenhuma
Healthcheck: intervalo / timeout / retries padrão	30 s / 30 s / 3
Atraso inicial da política de reinício	100 ms , dobrando a cada falha
Tempo mínimo em execução para a política de reinício valer	10 s
Porta padrão das métricas do daemon (convenção)	9323

10.5 Decisão rápida — Operação

Situação	Abordagem recomendada	Erro comum
Disco do host cheio de logs	Driver <code>local</code> ou <code>max-size / max-file</code> no <code>daemon.json</code>	Apagar arquivos de log com o contêiner rodando
Um contêiner derruba os outros por memória	<code>--memory</code> em todos os serviços	<code>--oom-kill-disable</code>
Java morto por OOM com heap aparentemente pequeno	<code>-XX:MaxRAMPercentage</code> e limite coerente	Fixar <code>-Xmx</code> maior que o limite do contêiner
Coletar logs em uma plataforma central	Driver de log ou agente coletor lendo STDOUT	Logar em arquivo dentro do contêiner
Aplicação trava quando o servidor de logs cai	<code>mode=non-blocking</code> com buffer	Remover o driver de log
Saber quando contêineres morrem por OOM	<code>docker events --filter event=oom</code> ou métricas	Olhar <code>docker ps</code> de vez em quando
Navegador headless travando	<code>--shm-size</code> maior	Rodar com <code>--privileged</code>
Ajustar limites sem recriar o contêiner	<code>docker update</code>	Parar, remover e recriar

11. Orquestração: Docker Swarm e o Caminho para Kubernetes

Ideia central: o Docker Engine sozinho gerencia contêineres **em um host**. Quando é preciso **vários hosts**, alta disponibilidade, rolling updates e substituição automática de contêineres doentes, entra um **orquestrador**. O **Swarm mode** vem embutido no Engine e usa arquivos Compose; o **Kubernetes** é o padrão de mercado para ambientes maiores.

11.1 Docker Swarm mode

11.1.1 Arquitetura

- **Managers:** mantêm o estado do cluster com o algoritmo de consenso **Raft**, agendam tarefas e atendem a API. Por padrão, também executam tarefas.
- **Workers:** executam as tarefas designadas pelos managers.
- **Serviço:** definição desejada (imagem, réplicas, portas, redes). **Tarefa:** cada contêiner que o Swarm cria para cumprir o serviço.
- Criar e entrar no cluster: `docker swarm init --advertise-addr <ip>` no primeiro manager; `docker swarm join-token worker` (ou `manager`) mostra o comando de entrada para os outros nós.
- O **Swarm clássico** (projeto separado, anterior a 2016) foi descontinuado; o que existe hoje é o **Swarm mode**, parte do Engine.

11.1.2 Criar e administrar o cluster

Cenário: três hosts Linux com Docker (`no1`, `no2`, `no3`), com as portas 2377/tcp, 7946/tcp e udp e 4789/udp liberadas entre eles.

```
# no1: cria o cluster e vira o primeiro manager (Leader)
docker swarm init --advertise-addr 10.0.0.11

# comandos de entrada (o token define o papel: worker ou manager)
docker swarm join-token worker
docker swarm join-token manager

# no2 e no3: entram no cluster com o comando mostrado
docker swarm join --token SWMTKN-1-... 10.0.0.11:2377

# em qualquer manager
docker node ls
docker node inspect no2 --pretty
docker node promote no2 no3      # workers viram managers (3 managers = tolera 1 falha)
docker node demote no3           # manager volta a ser worker
docker node update --availability drain no3
docker node update --label-add zona=a no2
```

Coluna	em	Significado
Leader		O manager que lidera o Raft no momento

Coluna <code>MANAGER STATUS</code> em <code>docker node ls</code>	Significado
Reachable	Manager saudável, participando do quórum
Unreachable	Manager que os outros não alcançam (conta como falha)
(vazio)	Worker

- **Qualquer manager** aceita comandos de administração e encaminha o que for preciso ao líder. Não é preciso estar no `Leader`, e comandos de cluster **não funcionam em workers**.
- **Sair e remover um nó:** no próprio nó, `docker swarm leave` (manager precisa de `--force` e deve ser **rebaixado antes**, para não quebrar o quórum); depois, em um manager, `docker node rm <nó>` apaga o registro, que fica como `Down`.
- **Tokens:** quem tem o token de manager pode entrar como manager. Guarde-os como segredo e troque se vazarem: `docker swarm join-token --rotate worker`.

11.1.3 Quórum de managers

Managers	Falhas toleradas	Quórum (maioria)
1	0	1
3	1	2
5	2	3
7	3	4

- Fórmula: um cluster com **N** managers tolera **(N – 1) / 2** falhas. Use **número ímpar**: 4 managers toleram só 1 falha, como 3. Mais de 7 managers deixa o consenso lento sem ganho real.
- **Sem quórum**, as tarefas existentes continuam rodando, mas **não é possível alterar** o cluster (criar, atualizar ou reagendar serviços). Recuperação: `docker swarm init --force-new-cluster` em um manager sobrevivente.
- Distribua os managers em zonas de falha diferentes.
- **Dois managers não dão alta disponibilidade.** O quórum de 2 é 2: se um cair, o outro não consegue alterar o cluster. É pior que ter só um manager, porque dobra a chance de falha sem tolerar nenhuma. Um exemplo com "2 managers e 1 worker" deve ser lido como laboratório; em produção, use 3 managers.

11.1.4 Serviços, atualizações e stacks

```
docker service create --name web --replicas 3 -p 80:8080 \
  --update-parallelism 1 --update-delay 10s \
  --update-failure-action rollback --update-order start-first \
  ghcr.io/org/web:1.4.2

docker service scale web=5
docker service update --image ghcr.io/org/web:1.4.3 web
docker service rollback web
docker service ps web          # tarefas e histórico
docker service logs -f web
```

- **O que é um serviço:** a unidade que o Swarm mantém no estado desejado. Por padrão, recebe um **IP virtual (VIP)** na rede do serviço: conexões ao nome ou ao VIP são **balanceadas** entre as tarefas pelo IPVS do kernel. Com `--endpoint-mode dnsrr`, o DNS devolve os IPs das tarefas no lugar do VIP.

- `docker service ls` mostra as réplicas atuais/desejadas (5/5); `docker service ps web` mostra em que nó está cada tarefa e o histórico de falhas; `docker service inspect --pretty web` resume a configuração (inclusive `UpdateConfig`, `RestartPolicy` e o VIP em `Endpoint.VirtualIPs`); `docker service rm web` remove o serviço e as tarefas.
- **Agendamento:** o scheduler distribui as tarefas pela estratégia **spread**, considerando os nós elegíveis (constraints, recursos **reservados**, plataforma) e quantas tarefas cada um já tem. Ele **não mede a carga real** (CPU em uso) dos nós. Para reservar capacidade, use `--reserve-cpu` e `--reserve-memory`.
- **Volumes em serviços:** `--mount type=volume,src=dados,dst=/app` cria um volume **local em cada nó** onde uma tarefa roda. **Não é o mesmo volume compartilhado entre os nós:** réplicas em nós diferentes enxergam dados diferentes, e uma tarefa reagendada em outro nó começa com um volume vazio. Para dados compartilhados, use um driver de volume de rede (NFS, plugin de storage) ou um serviço de dados externo (banco, storage de objetos), ou fixe o serviço em um nó com constraint.

Conceito	Detalhe
Modo replicated	Número fixo de réplicas distribuídas entre os nós
Modo global	Exatamente uma tarefa por nó elegível (agentes de monitoramento, coletores de log)
Rolling update	<code>parallelism</code> (quantas por vez), <code>delay</code> , <code>failure_action</code> (<code>pause</code> é o padrão; <code>continue</code> ou <code>rollback</code>), <code>order</code> (<code>stop-first</code> padrão ou <code>start-first</code>)
Placement	<code>--constraint node.role==worker</code> , <code>node.labels.zona==a</code> ; <code>--placement-pref spread=node.labels.zona</code>
Disponibilidade do nó	<code>active</code> , <code>pause</code> (não recebe tarefas novas), <code>drain</code> (tarefas são movidas para outros nós; usado em manutenção)
Stack	<code>docker stack deploy -c compose.yaml loja</code> implanta um arquivo Compose no Swarm usando a seção <code>deploy</code>

- Em stacks, `build:` é **ignorado**: as imagens precisam existir em um registry acessível por todos os nós.
- **Routing mesh:** uma porta publicada por um serviço fica disponível em **todos os nós** do cluster (rede `ingress`), que encaminham para uma tarefa saudável. Com `mode: host`, a porta é publicada só no nó onde a tarefa roda.
- **Secrets e configs do Swarm:** guardados **cifrados no log do Raft**, entregues só aos serviços autorizados e montados em **tmpfs** em `/run/secrets/<nome>`. São imutáveis: para rotacionar, crie um novo segredo e atualize o serviço.
- **Autolock** (`docker swarm update --autolock=true`): as chaves do Raft passam a ser protegidas por uma chave que precisa ser informada (`docker swarm unlock`) quando um manager reinicia.
- **Healthcheck no Swarm:** tarefas `unhealthy` são **substituídas** automaticamente.

11.2 Secrets e configs do Swarm

11.2.1 Comandos e uso

- **Docker secrets** (desde o Docker 1.13) guardam senhas, chaves e certificados de forma segura em clusters Swarm. Cada secret tem até **500 KB**.

Comando	O que faz
<code>printf 'minha-senha' docker secret create db_pass -</code>	Cria a partir do STDIN (o <code>-</code> final). <code>printf</code> evita a quebra de linha que o <code>echo</code> acrescenta
<code>docker secret create tls_key ./chave.pem</code>	Cria a partir de um arquivo
<code>docker secret ls</code>	Lista (nome, datas)

Comando	O que faz
<code>docker secret inspect db_pass</code>	Mostra metadados, nunca o conteúdo
<code>docker secret rm db_pass</code>	Remove (falha se algum serviço ainda usa)

```
docker service create --name app \
  --secret source=db_pass,target=password,uid=10001,gid=10001,mode=0400 \
  minha-org/app:1.0
# dentro de cada tarefa: arquivo /run/secrets/password (tmpfs), dono 10001, modo 0400
```

- A aplicação **lê o arquivo** (`open('/run/secrets/password').read().strip()` em Python). Muitas imagens oficiais aceitam variáveis `*_FILE` apontando para ele.
- **Rotação** (secrets são imutáveis): crie uma nova versão, troque no serviço e remova a antiga.

```
printf 'nova-senha' | docker secret create db_pass_v2 -
docker service update --secret-rm db_pass \
  --secret-add source=db_pass_v2,target=password app
docker secret rm db_pass
```

- O `update` recria as tarefas (rolling update). Mantenha o mesmo `target` para a aplicação não precisar mudar o caminho. Trocar a senha também no banco é outra etapa, que precisa ser coordenada.
- **Configs** (`docker config create/ls/inspect/rm`, `--config source=...,target=...`) funcionam igual, para arquivos de configuração **não sensíveis** (o conteúdo aparece no `inspect`).
- **Fora do Swarm**: o Docker Compose também tem `secrets` (capítulo 8), mas lá são arquivos do host montados no contêiner, **sem** a criptografia do Raft.

11.3 Stacks: arquivos Compose no Swarm

11.3.1 `docker stack` na prática

- `docker compose` executa um arquivo Compose em **um host**. `docker stack deploy` implanta o mesmo formato de arquivo em um **cluster Swarm**, transformando cada serviço em um serviço do Swarm. São ferramentas diferentes para o mesmo formato. O `docker stack` já vem com o Docker, e o Compose também, como plugin (`docker compose`).

```
# stack.yaml (sem "version:", que é obsoleto)
services:
  web:
    image: nginx:1.29
    ports:
      - "8080:80"
    networks: [webnet]
    deploy:
      replicas: 5
      resources:
        limits: { cpus: "0.10", memory: 50M }
      restart_policy:
        condition: on-failure
        delay: 10s
        max_attempts: 3
        window: 120s
      update_config:
        parallelism: 2
        delay: 10s
        failure_action: rollback
        order: start-first
      placement:
        constraints: [node.role == worker]

networks:
  webnet:
    driver: overlay
```

```
docker stack deploy -c stack.yaml primeiro    # cria (e, rodando de novo, atualiza)
docker stack ls                               # stacks e número de serviços
docker stack services primeiro               # serviços do stack
docker stack ps primeiro                     # tarefas e nós
docker service logs -f primeiro_web
docker stack rm primeiro                     # remove serviços e redes (volumes ficam)
```

Chave em <code>deploy</code>	Significado
<code>mode</code>	<code>replicated</code> (com <code>replicas</code>) ou <code>global</code> (uma tarefa por nó)
<code>replicas</code>	Número de tarefas
<code>resources.limits</code> / <code>reservations</code>	Limite e reserva de CPU e memória
<code>restart_policy</code>	<code>condition</code> (<code>none</code> , <code>on-failure</code> , <code>any</code>), <code>delay</code> , <code>max_attempts</code> , <code>window</code> (tempo para considerar que o reinício deu certo)
<code>update_config</code> / <code>rollback_config</code>	<code>parallelism</code> , <code>delay</code> , <code>failure_action</code> , <code>monitor</code> , <code>order</code>
<code>placement</code>	<code>constraints</code> (ex.: <code>node.role == manager</code> , <code>node.labels.zona == a</code>) e <code>preferences</code>
<code>labels</code>	Rótulos do serviço (os do contêiner ficam fora de <code>deploy</code>)

- **Atualizar um stack** é rodar o mesmo `docker stack deploy`: o Swarm compara o estado e faz rolling update só no que mudou; serviços novos são criados. Serviços removidos do arquivo só saem com `--prune`.
- **Nomes**: serviços e redes ganham o prefixo do stack (`primeiro_web`, `primeiro_webnet`); sem `networks`, é criada a rede `<stack>_default`.

- **Ignorado ou diferente no stack:** `build` (use imagens de registry), `depends_on` (não há ordem de início: a aplicação deve tolerar dependências indisponíveis), `container_name`, `restart` (use `deploy.restart_policy`) e `bind mounts` relativos (o caminho precisa existir em **cada** nó).
- **YAML é sensível à indentação:** chaves de `deploy` fora do nível certo (`resources` alinhado com `deploy`, por exemplo) são ignoradas ou geram erro. Valide com `docker compose -f stack.yaml config` antes do deploy.
- **Senhas:** exemplos didáticos colocam senhas em `environment`. Em stacks, use `secrets` externos (`docker secret create`) com `external: true` e as variáveis `*_FILE` das imagens.
- **Cuidado com o `docker.sock`:** serviços de visualização e monitoramento (visualizer, cAdvisor, Portainer) costumam montar o socket e são restritos a managers com `constraints`. Isso dá a eles controle total do cluster; publique essas portas só em redes administrativas.
- **Imagens dos exemplos:** tutoriais antigos usam `mysql:5.7`, `postgres:9.4`, `mongo:3.2` e `google/cadvisor`, todas fora de suporte ou movidas (o cAdvisor hoje é `gcr.io/cadvisor/cadvisor`). Use versões mantidas e fixadas.

11.3.2 Exemplo de stack de monitoramento

Um stack clássico para observar o cluster e os hosts combina:

Serviço	Papel	Modo típico
Prometheus	Coleta e guarda as métricas	<code>replicated</code> , 1 réplica, fixado em um nó com volume
node-exporter	Métricas do host (CPU, memória, disco, rede)	<code>global</code> : um por nó
cAdvisor	Métricas por contêiner	<code>global</code> : um por nó (monta <code>/</code> , <code>/sys</code> , <code>/var/lib/docker</code> e o socket, somente leitura)
Alertmanager	Agrega e envia alertas (e-mail, Slack, Rocket.Chat, Teams)	<code>replicated</code>
Grafana	Dashboards sobre o Prometheus	<code>replicated</code> , com volume para dados

- O modo `global` garante coletores em **todos os nós**, inclusive nos que entrarem depois. Nesses serviços, `hostname: "{{.Node.ID}}"` usa **templates do Swarm** para identificar o nó nas métricas.

11.4 Do Docker ao Kubernetes

11.4.1 Equivalências

Docker / Compose	Kubernetes
Contêiner	Contêiner dentro de um Pod
Serviço do Compose com réplicas	Deployment (ou <code>StatefulSet</code> para dados, <code>DaemonSet</code> para um por nó)
Nome do serviço na rede do Compose	Service (DNS interno, IP virtual)
ports publicadas	<code>Service NodePort</code> / <code>LoadBalancer</code> , Ingress ou <code>Gateway API</code>
Volume nomeado	PersistentVolumeClaim
secrets / configs	Secret / ConfigMap
<code>HEALTHCHECK</code> / <code>healthcheck</code>	livenessProbe , readinessProbe e startupProbe (o <code>HEALTHCHECK</code> da imagem é ignorado)
<code>deploy.resources.limits</code>	<code>resources.requests</code> e <code>resources.limits</code>

Docker / Compose	Kubernetes
<code>depends_on</code>	Sem equivalente direto: initContainers , readiness probes e tolerância a falhas na aplicação
<code>restart: always</code>	Deployment + <code>restartPolicy: Always</code>
Rede definida pelo usuário	Namespace + NetworkPolicy
<code>docker stack deploy</code>	<code>kubectl apply</code> , Helm, Kustomize, GitOps (Argo CD, Flux)

- **Kubernetes não usa o Docker Engine** como runtime desde a versão 1.24 (usa containerd ou CRI-O diretamente), mas **roda as mesmas imagens OCI**. Nada muda no Dockerfile.
- **Ferramentas de transição:** `kompose` converte arquivos Compose em manifests (ponto de partida, não resultado final); Kubernetes do Docker Desktop, **kind**, **k3d** e **minikube** para clusters locais.

11.4.2 Qual plataforma escolher

Cenário	Opção adequada
Uma aplicação pequena em um servidor	Docker Engine + Compose
Poucos hosts, time pequeno, sem experiência em Kubernetes	Swarm mode
Muitas equipes, ecossistema, autoescalonamento, políticas	Kubernetes gerenciado (EKS, GKE, AKS)
Não quer operar servidores nem cluster	Contêineres serverless: AWS ECS com Fargate, Google Cloud Run, Azure Container Apps
Jobs de build e teste	Runners de CI com Docker ou builders remotos

11.5 Decisão rápida — Orquestração

Situação	Abordagem recomendada	Erro comum
Swarm que sobreviva à perda de um manager	3 managers (ou 5 para tolerar 2 falhas)	2 managers (não toleram nenhuma falha)
Senha de banco em um stack do Swarm	<code>docker secret create + secrets no stack + variáveis *_FILE</code>	Senha em <code>environment</code>
Trocar a senha usada por um serviço	Novo secret + <code>docker service update --secret-rm/--secret-add</code>	Tentar editar o secret existente
Dados de uma réplica precisam sobreviver à troca de nó	Volume com driver de rede ou serviço de dados externo	Volume <code>local</code> achando que é compartilhado
Coletor de métricas em todos os nós	Serviço <code>global</code> (node-exporter, cAdvisor)	Uma réplica por nó contada à mão
Manutenção de um nó sem derrubar serviços	<code>docker node update --availability drain</code>	Desligar o nó direto
Um agente de monitoramento em cada nó	Serviço em modo <code>global</code>	Réplicas iguais ao número de nós
Atualização sem indisponibilidade e com volta automática	<code>update_config</code> com <code>order: start-first</code> e <code>failure_action: rollback</code>	Remover e recriar o serviço
Segredo compartilhado no cluster	<code>docker secret create + referência no serviço</code>	Variável de ambiente no stack
Stack não encontra a imagem nos workers	Enviar a imagem a um registry acessível	Construir em cada nó
Migrar um Compose para Kubernetes	<code>kompose</code> como ponto de partida + revisão (probes, recursos, Ingress)	Esperar que <code>depends_on</code> e o <code>HEALTHCHECK</code> funcionem iguais

Situação	Abordagem recomendada	Erro comum
Aplicação com tráfego variável sem operar cluster	Cloud Run, ECS Fargate ou Container Apps	Swarm em VMs superdimensionadas

12. Docker em Produção e CI/CD

Ideia central: em produção, a imagem é um **artefato imutável** construído **uma vez** pelo pipeline, testado, escaneado, assinado e promovido pelo **digest** entre ambientes. O que muda de um ambiente para outro é a **configuração**, injetada em tempo de execução.

12.1 Pipeline de imagens

12.1.1 Etapas de um pipeline

1. **Lint:** `hadolint Dockerfile` e `docker build --check`.
2. **Build** com Buildx e cache exportado (`type=gha` ou `type=registry`), multiplataforma se necessário.
3. **Testes** dentro do contêiner: unitários no estágio `test` (`--target test`) e de integração com `docker compose up --wait` ou **Testcontainers**.
4. **Scan** de vulnerabilidades e segredos (Docker Scout, Trivy), com **política de bloqueio** (ex.: falhar com CVE crítico que tenha correção).
5. **SBOM e provenance** anexados à imagem.
6. **Push** com tags imutáveis (SHA do commit, versão semântica) e tags de conveniência.
7. **Assinatura** (cosign keyless com a identidade OIDC do CI).
8. **Deploy pelo digest** em homologação; depois, **promoção** do mesmo digest para produção.

12.1.2 Exemplo com GitHub Actions

```
name: imagem
on:
  push:
    branches: [main]
    tags: ["v*"]

permissions:
  contents: read
  packages: write
  id-token: write      # assinatura keyless com cosign

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: docker/setup-qemu-action@v4
      - uses: docker/setup-buildx-action@v4
      - uses: docker/login-action@v4
        with:
          registry: ghcr.io
          username: ${github.actor}
          password: ${secrets.GITHUB_TOKEN}
      - id: meta
        uses: docker/metadata-action@v6
        with:
          images: ghcr.io/${github.repository}
          tags: |
            type=sha
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}
            type=ref,event=branch
      - id: build
        uses: docker/build-push-action@v7
        with:
          platforms: linux/amd64,linux/arm64
          push: true
          tags: ${steps.meta.outputs.tags}
          labels: ${steps.meta.outputs.labels}
          cache-from: type=gha
          cache-to: type=gha,mode=max
          sbom: true
          provenance: mode=max
```

- A saída `steps.build.outputs.digest` traz o digest da imagem, que deve ser usado nos passos de assinatura e de deploy.
- Versões das actions conferidas em setembro de 2026.

12.1.3 Docker dentro do CI

Abordagem	Como funciona	Riscos e cuidados
Runner com Docker nativo	O job usa o daemon da VM do runner (ex.: GitHub-hosted runners)	Simples; a VM é descartada ao fim do job

Abordagem	Como funciona	Riscos e cuidados
Docker-in-Docker (<code>docker:dind</code>)	Um daemon separado roda em um contêiner privilegiado	Exige <code>--privileged</code> ; sem cache persistente a menos que configurado
Socket do host (Docker-out-of-Docker)	O job monta o <code>docker.sock</code> do host	O job ganha root no host e vê os contêineres de outros jobs
BuildKit rootless ou builder remoto	Build sem daemon privilegiado (<code>moby/buildkit:rootless</code> , driver <code>kubernetes</code> , Docker Build Cloud)	Opção mais segura em clusters compartilhados

- O **Kaniko**, alternativa popular para builds sem daemon no Kubernetes, foi **arquivado pelo Google em 2025**; prefira BuildKit rootless ou builders remotos em projetos novos.

12.2 Checklist de produção

12.2.1 Imagem

- Base mínima, mantida e fixada por digest; multi-stage; sem ferramentas de build no runtime.
- Usuário não root; sem segredos; `.dockerignore` completo.
- Scan sem vulnerabilidades críticas conhecidas com correção disponível; SBOM e provenance; assinatura.
- Rótulos OCI (`source`, `revision`, `version`).

12.2.2 Execução

- Configuração por variáveis de ambiente e segredos por arquivo ou cofre (princípios **12-factor**).
- **Limites** de memória, CPU e PIDs em todos os contêineres.
- **Healthcheck** (ou probes no Kubernetes) e política de reinício.
- **Parada graciosa**: tratar `SIGTERM`, forma `exec`, `stop_grace_period` coerente com o tempo de encerramento.
- Sistema de arquivos **somente leitura**, `--cap-drop ALL`, `no-new-privileges`.
- Portas publicadas apenas onde necessário, atrás de proxy reverso com TLS.

12.2.3 Host e operação

- Engine atualizado; `live-restore`; rotação de logs; monitoramento de disco.
- Daemon acessível só por SSH ou TLS mútuo; ninguém no grupo `docker` sem necessidade.
- Coleta centralizada de logs e métricas; alertas para OOM, reinícios e healthchecks.
- Backups de volumes testados (restauração, não só cópia).
- Limpeza periódica de imagens e cache de build.

12.3 Ecossistema e alternativas

12.3.1 Ferramentas compatíveis

Ferramenta	O que é
Podman	Motor de contêineres sem daemon, rootless por padrão, com CLI compatível com a do Docker e suporte a pods
containerd + nerdctl	Runtime do Kubernetes com uma CLI compatível com o Docker

Ferramenta	O que é
Colima, Rancher Desktop, OrbStack	Alternativas ao Docker Desktop que rodam uma VM Linux com um Engine ou containerd
Testcontainers	Bibliotecas que sobem dependências reais (bancos, filas) em contêineres durante os testes
Dev Containers	Ambiente de desenvolvimento completo definido em <code>devcontainer.json</code> , usado pelo VS Code e pelo GitHub Codespaces

- Como todos seguem as especificações **OCI**, imagens e Dockerfiles funcionam entre essas ferramentas com poucas ou nenhuma mudança.

12.4 Decisão rápida — Produção e CI/CD

Situação	Abordagem recomendada	Erro comum
Mesma imagem em homologação e produção	Construir uma vez e promover o digest	Rebuildar por ambiente
Configuração diferente por ambiente	Variáveis de ambiente, arquivos de configuração e segredos na execução	Uma imagem por ambiente
Build de imagens em Kubernetes compartilhado	BuildKit rootless ou builder remoto	Montar o socket do host no pod
Testes de integração com banco real	Testcontainers ou <code>docker compose up --wait</code> no CI	Mocks de tudo ou banco compartilhado
Bloquear imagens vulneráveis	Scan no pipeline com política de falha	Relatório que ninguém lê
Rastrear qual commit gerou a imagem em produção	Tag por SHA + provenance + rótulos OCI	Tag <code>latest</code>
Empresa grande sem licença do Docker Desktop	Assinatura paga ou alternativa (Colima, Rancher Desktop, Podman)	Ignorar a licença
Parada sem perder requisições em andamento	Tratar <code>SIGTERM</code> , drenar conexões, <code>stop_grace_period</code>	<code>SIGKILL</code> depois de 10 s

13. Pegadinhas e Padrões Recorrentes

Como usar: revise esta seção na véspera de uma entrevista, prova ou revisão de arquitetura. Cada linha é um erro de conceito que aparece com frequência em código real, em fóruns e em questões de certificação.

13.1 Afirmações falsas clássicas

Afirmção falsa	O que é verdade
"EXPOSE publica a porta"	EXPOSE só documenta. Quem publica é -p (ou -P, que usa a lista do EXPOSE)
"Contêiner é uma VM leve"	É um processo isolado por namespaces e cgroups, que compartilha o kernel do host
"latest é a versão mais nova"	É só a tag padrão; aponta para o que foi enviado por último com esse nome
"Apagar um arquivo em um RUN posterior diminui a imagem"	O arquivo continua na camada em que foi criado
"docker system prune apaga meus volumes"	Só com --volumes, e mesmo assim só os anônimos (use docker volume prune -a para os nomeados)
"docker compose down apaga o banco"	Só com -v. Sem a flag, volumes nomeados são preservados
"depends_on espera o banco ficar pronto"	Na forma curta, só espera o contêiner iniciar . Para esperar ficar pronto: condition: service_healthy
"O .env vai para dentro dos contêineres"	Ele só alimenta a interpolação do arquivo Compose
"--restart always reinicia contêineres unhealthy"	Políticas de reinício reagem à saída do processo, não ao healthcheck (no Engine sem orquestrador)
"Montar o docker.sock com :ro é seguro"	O somente leitura vale para o arquivo; a API continua com poder total
"Estar no grupo docker é mais seguro que usar sudo"	É equivalente a root no host
"UFW bloqueia portas publicadas pelo Docker"	As regras do Docker são avaliadas antes; use DOCKER-USER ou bind em 127.0.0.1
"ARG serve para passar senhas ao build"	Valores de ARG ficam no histórico da imagem; use build secrets
"docker export faz backup da imagem"	export achata o sistema de arquivos de um contêiner e perde metadados; para imagens use docker save
"Kubernetes não roda imagens Docker"	Kubernetes não usa o Docker Engine, mas roda imagens OCI construídas com Docker normalmente
"version: '3.8' é obrigatório no Compose"	O campo é obsoleto e só gera aviso
"CMD e ENTRYPOINT são a mesma coisa"	ENTRYPOINT é o executável fixo; CMD são os argumentos padrão (ou o comando, se não houver ENTRYPOINT)
"Alpine é sempre a melhor base"	É pequena, mas usa musl libc; binários glibc e algumas bibliotecas nativas falham ou ficam mais lentos
"--cpu-shares limita a CPU"	É um peso relativo que só vale em disputa; para limitar use --cpus
"Dois managers dão alta disponibilidade ao Swarm"	Dois managers não toleram nenhuma falha; use 3 ou 5
"O Docker inventou os contêineres"	chroot, Jails, Zones, OpenVZ e LXC vieram antes; a inovação do Docker foi tornar a tecnologia acessível
"O contêiner emula um sistema operacional"	Ele roda direto no kernel do host, isolado; quem emula hardware e roda outro kernel é a VM

Afirmção falsa	O que é verdade
"Um contêiner roda em qualquer sistema"	Precisa de kernel compatível (Linux, na prática) e da mesma arquitetura de CPU; no macOS e no Windows, roda numa VM Linux
" <code>exit</code> sai do contêiner e o deixa rodando"	Em <code>docker run -it ... bash</code> , <code>exit</code> encerra o PID 1 e para o contêiner; use <code>Ctrl+P Ctrl+Q</code>
"Só o manager <code>Leader</code> aceita comandos do Swarm"	Qualquer manager aceita e encaminha ao líder
"O Swarm coloca a tarefa no nó com menor carga"	Ele distribui por quantidade de tarefas e recursos reservados (spread), não pela carga medida
"Docker secrets só existem no Swarm"	O Compose também tem <code>secrets</code> (arquivos montados, sem a criptografia do Raft)

13.2 Informações desatualizadas em materiais antigos

Livros, cursos e posts de 2015 a 2020 continuam circulando. Ao estudar por eles, atualize estes pontos:

Se o material diz...	Hoje (set/2026)
Instalar com <code>apt-key adv</code> e o repositório <code>apt.dockerproject.org</code>	<code>apt-key</code> está descontinuado e o repositório foi desativado. Use <code>download.docker.com</code> com a chave em <code>/etc/apt/keyrings</code> (capítulo 1)
<code>service docker start / /etc/init.d/docker status</code>	<code>systemctl start docker / systemctl status docker</code>
Kernel 3.8 ou superior	A documentação lista as versões de distribuição suportadas; qualquer distribuição mantida atende
"Docker só roda em Linux 64 bits"	O daemon roda em Linux (<code>amd64</code> , <code>arm64</code> e outras arquiteturas de 64 bits); no macOS e no Windows, via VM. Há também contêineres Windows nativos
Storage drivers AUFS, devicemapper e <code>overlay</code>	Removidos (Engine 24 e 25). <code>overlay2</code> no store clássico ou containerd snapshots, padrão no Engine 29
<code>--storage-opt dm.basesize, dm.thinpooldev</code>	Opções do devicemapper, que não existe mais
<code>docker daemon ...</code> ou <code>docker -d</code>	<code>dockerd</code>
Docker Machine, boot2docker, Docker Toolbox	Descontinuados. Docker Desktop (ou alternativas) localmente; Terraform/cloud-init + <code>docker context</code> para hosts remotos
<code>dockerd -H tcp://0.0.0.0:2375</code>	Nunca sem TLS; use SSH (<code>docker context</code>) ou TLS mútuo na porta 2376
<code>--link</code> para ligar contêineres	Redes definidas pelo usuário com DNS embutido
Data-only container com <code>--volumes-from</code>	Volumes nomeados (<code>docker volume create</code>)
<code>docker-compose</code> (com hífen) e <code>version: "3"</code>	<code>docker compose</code> (plugin, v5) e arquivo <code>compose.yaml</code> sem <code>version</code>
"Para usar Compose, use <code>docker stack</code> "	<code>docker compose</code> para um host; <code>docker stack deploy</code> para Swarm
<code>registry:2</code> e <code>github.com/docker/distribution</code>	<code>registry:3</code> , projeto <code>distribution/distribution</code> na CNCF
Criar conta com <code>docker login</code>	Conta criada no site do Docker Hub; <code>docker login</code> só autentica
Site ImageLayers para ver camadas	Página da tag no Docker Hub, <code>docker image history</code> , <code>buildx imagetools inspect</code> , <code>diver</code>
Saída de build com <code>Step 1/2</code> e <code>Removing intermediate container</code>	Saída do BuildKit (<code>[+] Building</code> , <code>[1/2]</code> , <code>CACHED</code>)
MAINTAINER no Dockerfile	<code>LABEL org.opencontainers.image.authors=...</code>
Imagens <code>centos:7</code> , <code>debian:8</code> , <code>mysql:5.7</code> , <code>postgres:9.4</code> , <code>mongo:3.2</code> , <code>alpine:3.1</code>	Todas fora de suporte. Use versões mantidas e fixe a tag (e o digest)
<code>KernelMemory / --kernel-memory</code>	Descontinuado; sem efeito com cgroup v2
Docker Content Trust (<code>DOCKER_CONTENT_TRUST=1</code>)	Aposentado. Use cosign ou Notation

Se o material diz...	Hoje (set/2026)
Swarm com 2 managers	3 (ou 5) managers; 2 não toleram nenhuma falha
"Volume de um serviço Swarm é compartilhado entre os nós"	Com o driver <code>local</code> , cada nó tem o seu volume; compartilhar exige driver de rede
Dois bancos de dados usando o mesmo volume de dados	Nunca: corrompe os dados. Um volume por instância e replicação do banco

13.3 Pares que mais se confundem

Par	Diferença
<code>docker run</code> vs. <code>docker start</code>	<code>run</code> cria um contêiner novo ; <code>start</code> inicia um contêiner existente
<code>docker exec</code> vs. <code>docker attach</code>	<code>exec</code> inicia um processo novo no contêiner; <code>attach</code> conecta ao STDIN/STDOUT do PID 1 (sair com Ctrl+C pode parar o contêiner; use Ctrl+P Ctrl+Q para desanexar)
<code>docker stop</code> vs. <code>docker kill</code>	<code>stop</code> envia <code>SIGTERM</code> e espera; <code>kill</code> envia <code>SIGKILL</code> (ou outro sinal) imediatamente
<code>docker save</code> vs. <code>docker export</code>	<code>save</code> exporta imagens com camadas e metadados; <code>export</code> exporta o sistema de arquivos de um contêiner
<code>COPY</code> vs. <code>ADD</code>	<code>ADD</code> também baixa URLs, clona Git e extrai tarballs locais; prefira <code>COPY</code>
<code>CMD</code> vs. <code>ENTRYPOINT</code>	Argumentos do <code>docker run</code> substituem o <code>CMD</code> ; o <code>ENTRYPOINT</code> só muda com <code>--entrypoint</code>
<code>ARG</code> vs. <code>ENV</code>	<code>ARG</code> só existe no build; <code>ENV</code> persiste na imagem e na execução
Volume vs. bind mount	Volume é gerenciado pelo Docker e pré-populado; bind mount é um caminho do host e esconde o conteúdo da imagem
Bridge padrão vs. definida pelo usuário	Só a definida pelo usuário tem DNS por nome e isolamento por rede
<code>-p 8080:80</code> vs. <code>-p 127.0.0.1:8080:80</code>	O primeiro expõe em todas as interfaces; o segundo só localmente
<code>always</code> vs. <code>unless-stopped</code>	Após reiniciar o daemon, <code>always</code> volta mesmo se tiver sido parado manualmente; <code>unless-stopped</code> não
<code>docker compose run</code> vs. <code>exec</code>	<code>run</code> cria um contêiner avulso novo; <code>exec</code> roda um processo em um contêiner já em execução
Imagem dangling vs. não usada	Dangling não tem tag (<code><none></code>); não usada pode ter tag, mas nenhum contêiner a usa
Tag vs. digest	A tag é móvel; o digest é imutável e identifica o conteúdo
Rootless vs. <code>users-remap</code>	No rootless, o daemon também roda sem root; no <code>users-remap</code> , só o root do contêiner é mapeado
<code>mode=min</code> vs. <code>mode=max</code> no cache	<code>max</code> exporta também as camadas de estágios intermediários
Replicated vs. global (Swarm)	Número fixo de réplicas vs. uma tarefa por nó
<code>docker compose</code> vs. <code>docker stack deploy</code>	O mesmo formato de arquivo: <code>compose</code> roda em um host; <code>stack</code> implanta como serviços de um Swarm
<code>dockerd</code> vs. <code>docker</code>	<code>dockerd</code> é o daemon (servidor); <code>docker</code> é a CLI (cliente) que fala com a API dele
<code>bip</code> vs. <code>default-address-pools</code>	<code>bip</code> define a sub-rede da bridge padrão <code>docker0</code> ; <code>default-address-pools</code> , as faixas das redes criadas pelo usuário e pelo Compose
<code>docker secret</code> vs. <code>docker config</code>	Os dois são arquivos entregues aos serviços; o <code>secret</code> é cifrado e seu conteúdo nunca aparece no <code>inspect</code>
<code>docker node promote</code> vs. <code>docker swarm join-token manager</code>	<code>promote</code> transforma um worker existente em manager; o token de manager faz um nó novo entrar já como manager

13.4 Palavras-chave e respostas

Se o problema diz...	Pense em...
"demora 10 segundos para parar"	PID 1 não trata SIGTERM; forma exec, <code>exec "\$@"</code> , <code>--init</code>
"saiu com 137"	OOM kill ou SIGKILL
" <code>exec format error</code> "	Arquitetura errada; <code>--platform</code> , imagem multiplataforma
"o nome do outro contêiner não resolve"	Rede bridge definida pelo usuário
"funciona dentro, mas não pela porta publicada"	Aplicação escutando em <code>127.0.0.1</code>
"o build reinstala tudo a cada commit"	Ordem das instruções e <code>.dockerignore</code>
"token no build"	<code>RUN --mount=type=secret</code>
"imagem enorme"	Multi-stage, base mínima, limpar na mesma camada
"disco cheio"	<code>docker system df</code> , prune, rotação de logs
"precisa rodar em Mac M-series e servidor x86"	<code>buildx --platform linux/amd64,linux/arm64</code>
"sem perder dados ao recriar o contêiner"	Volume nomeado
"a API sobe antes do banco"	<code>depends_on</code> com <code>service_healthy</code>
"cluster tolerante a falha de um manager"	3 managers
"garantir que o que foi testado é o que vai para produção"	Promover o digest

14. Referência Rápida de Comandos

Como usar: consulta de bancada. Os comandos seguem a forma agrupada (`docker container ...`, `docker image ...`); as formas curtas (`docker ps`, `docker rmi`) continuam válidas.

14.1 Contêineres

Comando	Para que serve
<code>docker run -d --name app -p 8080:80 imagem</code>	Cria e inicia em segundo plano
<code>docker run --rm -it imagem sh</code>	Sessão interativa descartável
<code>docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'</code>	Lista com colunas escolhidas
<code>docker logs -f --since 10m app</code>	Acompanha os logs
<code>docker exec -it app sh</code>	Shell no contêiner em execução
<code>docker stop app / docker start app / docker restart app</code>	Controle de execução
<code>docker rm -f app</code>	Remove (forçando a parada)
<code>docker inspect -f '{{.State.ExitCode}}' app</code>	Campo específico do estado
<code>docker stats --no-stream</code>	Consumo de recursos
<code>docker cp app:/etc/config.yaml .</code>	Copia arquivos do contêiner
<code>docker update --memory 1g app</code>	Ajusta limites em execução

14.2 Imagens e build

Comando	Para que serve
<code>docker build -t org/app:1.0 .</code>	Build local
<code>docker build --target test .</code>	Build até um estágio
<code>docker build --check .</code>	Build checks sem construir
<code>docker build --secret id=npmc,src=\$HOME/.npmrc .</code>	Build com segredo
<code>docker buildx build --platform linux/amd64,linux/arm64 -t org/app:1.0 --push .</code>	Build multiplataforma
<code>docker buildx bake --push</code>	Vários builds a partir do arquivo Bake
<code>docker image history org/app:1.0</code>	Camadas e tamanhos
<code>docker buildx imagetools inspect org/app:1.0</code>	Plataformas e digests no registry
<code>docker tag org/app:1.0 ghcr.io/org/app:1.0</code>	Novo nome para a mesma imagem
<code>docker push ghcr.io/org/app:1.0</code>	Envio ao registry
<code>docker save -o app.tar org/app:1.0 / docker load -i app.tar</code>	Transporte sem registry
<code>docker scout quickview org/app:1.0</code>	Resumo de vulnerabilidades

14.3 Redes e volumes

Comando	Para que serve
<code>docker network create app</code>	Rede bridge definida pelo usuário
<code>docker network create --internal dados</code>	Rede sem saída externa
<code>docker network connect app contêiner</code>	Conecta um contêiner em execução
<code>docker network inspect app</code>	Contêineres e IPs da rede
<code>docker volume create dados</code>	Cria um volume nomeado
<code>docker volume inspect dados</code>	Driver e ponto de montagem
<code>docker run --rm -v dados:/d -v "\$(pwd)":/b alpine tar czf /b/dados.tgz -C /d .</code>	Backup de volume

14.4 Compose

Comando	Para que serve
<code>docker compose up -d --build --wait</code>	Sobe, reconstruindo e esperando ficar saudável
<code>docker compose config</code>	Modelo resolvido
<code>docker compose ps / logs -f</code>	Estado e logs
<code>docker compose exec web sh</code>	Shell em um serviço
<code>docker compose run --rm web npm test</code>	Tarefa avulsa
<code>docker compose watch</code>	Sincronização em desenvolvimento
<code>docker compose --profile debug up -d</code>	Inclui serviços de um perfil
<code>docker compose down / down -v</code>	Remove tudo / e também os volumes

14.5 Sistema, contextos e Swarm

Comando	Para que serve
<code>docker version / docker info</code>	Versões e configuração do daemon
<code>docker system df -v</code>	Uso de disco
<code>docker system prune -af --filter until=24h</code>	Limpeza agressiva de itens antigos
<code>docker events --filter event=oom</code>	Eventos de OOM
<code>docker context create prod --docker host=ssh://user@host</code>	Contexto remoto
<code>docker context use prod</code>	Troca o daemon alvo
<code>docker swarm init --advertise-addr <ip></code>	Cria um Swarm
<code>docker node ls / docker node update --availability drain <nó></code>	Nós e manutenção
<code>docker stack deploy -c compose.yaml loja</code>	Implanta uma stack
<code>docker service ls / ps / logs / rollback</code>	Operação de serviços

15. Autoteste — Flashcards de Revisão

Cada card abaixo esconde a resposta: clique para expandir só depois de tentar responder mentalmente. O objetivo é forçar a lembrança ativa, não a releitura. Uma rodada de 20 cards por dia cobre todo o banco em menos de uma semana.

15.1 Fundamentos e ciclo de vida

Quais mecanismos do kernel Linux tornam um contêiner possível?

RESPOSTA

Namespaces (isolam o que o processo enxerga), **cgroups** (limitam o que ele consome) e um **sistema de arquivos em camadas** (union filesystem/snapshotter). Capabilities, seccomp e AppArmor/SELinux complementam a segurança.

Qual é a principal diferença de isolamento entre contêiner e VM?

RESPOSTA

O contêiner **compartilha o kernel** do host; a VM tem **kernel próprio** sobre um hypervisor. Por isso a VM oferece uma fronteira de segurança mais forte, e o contêiner é mais leve e rápido.

Qual é o caminho de um `docker run` até o processo?

RESPOSTA

CLI → API do `dockerd` → `containerd` → `containerd-shim` → `runc`, que cria namespaces e cgroups, executa o processo e sai. O shim continua como pai do processo.

O que o Engine 29 mudou no armazenamento de imagens?

RESPOSTA

O **containerd image store** passou a ser o padrão em instalações novas, no lugar dos storage drivers clássicos como `overlay2`. Ele suporta imagens multiplataforma e atestados nativamente.

Por que estar no grupo `docker` equivale a ter root?

RESPOSTA

Porque quem controla o daemon pode executar um contêiner privilegiado montando `/` do host e alterar qualquer arquivo.

Qual é a forma recomendada de acessar um daemon Docker remoto?

RESPOSTA

`docker context` com **SSH** (`host=ssh://usuario@servidor`), ou TCP com TLS mútuo na porta 2376. Nunca a porta 2375 sem TLS.

O que acontece, passo a passo, em um `docker stop` ?

RESPOSTA

O Docker envia `SIGTERM` (ou o `STOPSIGNAL`) ao PID 1, espera **10 segundos** e, se o processo não sair, envia `SIGKILL`.

Por que uma aplicação iniciada com `CMD node server.js` demora para parar?

RESPOSTA

Na forma shell, o PID 1 é `/bin/sh`, que não repassa o `SIGTERM` ao Node. O Docker espera 10 s e mata com `SIGKILL`. A correção é a forma exec: `CMD ["node", "server.js"]`.

O que significam os códigos de saída **125**, **126**, **127** e **137**?

RESPOSTA

125: o próprio `docker run` falhou. **126**: comando não executável. **127**: comando não encontrado. **137**: morto por `SIGKILL` (`128 + 9`), muitas vezes por OOM.

Qual é a diferença entre as políticas `always` e `unless-stopped` ?

RESPOSTA

Ambas reiniciam o contêiner quando ele sai. Depois de reiniciar o daemon, `always` sobe o contêiner mesmo que ele tenha sido parado manualmente; `unless-stopped` respeita a parada manual.

Para que serve `--init` ?

RESPOSTA

Coloca um init mínimo (`tini`) como PID 1, que repassa sinais à aplicação e recolhe processos zumbis.

Como investigar um contêiner cuja imagem não tem shell?

RESPOSTA

`docker debug <contêiner>` (Docker Desktop) ou um contêiner auxiliar compartilhando namespaces: `docker run -it --rm --pid container:app --network container:app nicolaka/netshoot`.

O que `docker system prune` remove por padrão?

RESPOSTA

Contêineres parados, redes não usadas, imagens dangling e cache de build. **Não** remove volumes nem imagens com tag não usadas (isso exige `--volumes` e `-a`).

O Docker inventou os contêineres? Qual foi a inovação dele?

RESPOSTA

Não. `chroot` (1979), FreeBSD Jails, Solaris Zones, OpenVZ e LXC vieram antes. A inovação do Docker foi de **experiência**: Dockerfile, imagem em camadas, registry público (Docker Hub) e uma CLI simples, que tornaram a tecnologia acessível a qualquer desenvolvedor.

Qual era a relação entre o Docker e o LXC?

RESPOSTA

As primeiras versões do Docker (2013) eram um wrapper do LXC com AUFS. No Docker 0.9 (2014), o LXC foi trocado pela **libcontainer**, que depois deu origem ao **runc**.

Qual namespace isola hostname e qual isola a árvore de processos?

RESPOSTA

UTS isola hostname e domínio; **PID** isola a árvore de processos (a aplicação é o PID 1 dentro do contêiner e tem outro PID no host).

Como a interface `eth0` de um contêiner se liga à rede do host?

RESPOSTA

Por um **par veth**: uma ponta fica no namespace de rede do contêiner (`eth0`) e a outra no host (`vethxxxx`), ligada à `bridge docker0`, que é o gateway dos contêineres.

Que regras de netfilter o Docker cria para `-p 8080:80`?

RESPOSTA

Um **DNAT** na chain `DOCKER` da tabela `nat` (porta 8080 do host → IP do contêiner:80), um **MASQUERADE** para a saída dos contêineres e um **ACCEPT** na tabela `filter` para o tráfego encaminhado.

Qual é a forma recomendada de instalar o Docker Engine em um servidor Debian ou Ubuntu?

RESPOSTA

Pelo repositório oficial `download.docker.com`, com a chave em `/etc/apt/keyrings` e os pacotes `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin` e `docker-compose-plugin`. O script `get.docker.com` é só para teste e desenvolvimento.

O que substituiu o Docker Machine?

RESPOSTA

Localmente, o Docker Desktop (ou Colima, OrbStack, Rancher Desktop). Para hosts remotos, provisionamento com Terraform/cloud-init, instalação pelo repositório oficial e acesso com `docker context` via SSH.

Para que servem as opções `bip` e `fixed-cidr` do daemon?

RESPOSTA

`bip` define o IP e a sub-rede da bridge `docker0`; `fixed-cidr` restringe a faixa de IPs entregue aos contêineres dentro dela. Para as redes criadas pelo usuário, use `default-address-pools`.

O que o `-H fd://` na unit do `systemd` significa?

RESPOSTA

Socket activation: o `systemd` abre o socket e o entrega ao `dockerd`. Por isso, definir `hosts` também no `daemon.json` conflita e impede o daemon de subir.

Como sair de um `docker run -it ubuntu bash` sem parar o contêiner?

RESPOSTA

Com **Ctrl+P seguido de Ctrl+Q**. `exit` encerra o `bash`, que é o PID 1, e para o contêiner. Para voltar, `docker attach` (ao PID 1) ou, melhor, `docker exec -it <c> bash`.

Qual é a diferença entre `docker create` e `docker run`?

RESPOSTA

`create` só cria o contêiner (status `Created`); `run` cria e inicia. Um contêiner criado é iniciado com `docker start` (`-ai` para anexar).

15.2 Imagens e Dockerfile

Qual é a diferença entre tag e digest?

RESPOSTA

A **tag** é um rótulo móvel que pode apontar para outra imagem amanhã. O **digest** (`sha256:...`) é o hash do manifest: imutável e identifica exatamente o conteúdo.

Para que serve um image index (manifest list)?

RESPOSTA

Agrupa um manifest por plataforma (`linux/amd64`, `linux/arm64` ...) sob a mesma tag. O cliente baixa automaticamente a variante da sua arquitetura.

Qual é a referência completa de `nginx`?

RESPOSTA

`docker.io/library/nginx:latest`.

Quais são os limites de pull do Docker Hub (set/2026)?

RESPOSTA

Janela de 6 horas: **100** pulls para usuários não autenticados (por IPv4 ou /64 IPv6), **200** para contas Personal autenticadas e ilimitado para Pro, Team e Business.

Qual é a diferença entre `docker save` e `docker export`?

RESPOSTA

`save` exporta **imagens** com camadas, tags e metadados (volta com `load`). `export` exporta o sistema de arquivos achatado de um **contêiner**, sem histórico, CMD nem ENV (volta com `import`).

Por que se deve copiar `package.json` antes do restante do código?

RESPOSTA

Para que a camada de instalação de dependências fique em cache e só seja refeita quando os manifestos mudarem, e não a cada alteração no código.

Por que `RUN apt-get update` sozinho em uma linha é um problema?

RESPOSTA

O cache de `RUN` depende só do texto do comando; a camada fica em cache para sempre e um `apt-get install` posterior pode usar índices desatualizados. Junte `update` e `install` no mesmo `RUN`.

Qual é o resultado de `ENTRYPOINT ["app"]` com `CMD ["--port", "80"]` e `docker run img --port 90`?

RESPOSTA

`app --port 90`. Os argumentos do `docker run` substituem o `CMD` e são passados ao `ENTRYPOINT`.

Quando usar `ADD` em vez de `COPY`?

RESPOSTA

Só para baixar uma URL (de preferência com `--checksum`), clonar um repositório Git ou extrair automaticamente um tarball local. Nos outros casos, `COPY`.

O que o `.dockerignore` evita?

RESPOSTA

Enviar arquivos desnecessários ao builder (build lento), invalidar o cache sem motivo e copiar segredos (`.env`, chaves) e `.git` para a imagem com `COPY`...

O que é um multi-stage build e qual o principal ganho?

RESPOSTA

Um Dockerfile com vários `FROM`: os estágios iniciais compilam e o estágio final copia só o artefato. A imagem final fica sem compiladores, código-fonte nem cache.

Quais são os padrões do `HEALTHCHECK`?

RESPOSTA

Intervalo de 30 s, timeout de 30 s, `start-period` de 0 s, `start-interval` de 5 s e 3 tentativas até `unhealthy`.

Qual o risco de usar Alpine como base para qualquer aplicação?

RESPOSTA

Alpine usa **musl libc**; binários e bibliotecas nativas compiladas para glibc podem falhar ou ter desempenho pior.

O que são as Docker Hardened Images?

RESPOSTA

Imagens base mínimas e endurecidas mantidas pela Docker, sem shell na variante de runtime, rodando como não root, com SBOM e provenance assinados. Gratuitas e Apache 2.0 desde dezembro de 2025.

O que acontece com um arquivo da imagem quando o contêiner o altera pela primeira vez?

RESPOSTA

É feito o **copy-up**: o arquivo inteiro é copiado para a camada gravável e alterado lá. Apagar um arquivo da imagem cria um **whiteout** na camada superior, sem liberar espaço na imagem.

O que acontece com `docker build .` sem `-t`?

RESPOSTA

A imagem é criada sem nome, como `<none>:<none>`. Dê o nome com `-t repositório:tag` no build ou depois com `docker image tag`.

O `.` em `docker build -t app .` é o caminho do Dockerfile?

RESPOSTA

Não: é o **contexto de build** (um diretório). O builder procura o `Dockerfile` na raiz do contexto; para outro arquivo, use `-f`.

Por que o serviço deve rodar em primeiro plano no contêiner?

RESPOSTA

Porque o contêiner vive enquanto o PID 1 vive. Um serviço que vai para segundo plano faz o contêiner terminar, e iniciar o serviço à mão com `docker exec` não é reproduzível. Exemplo: `ENTRYPOINT ["apache2"] + CMD ["-D", "FOREGROUND"]`.

Por que evitar `docker commit` para criar imagens de produção?

RESPOSTA

Não há receita reproduzível: a imagem carrega a sujeira da sessão, não tem revisão nem cache de build, e refazê-la com uma base nova exige repetir tudo à mão. O Dockerfile resolve todos esses pontos.

Qual é o formato de nome necessário para enviar uma imagem ao Docker Hub?

RESPOSTA

`usuário-ou-organização/repositório:tag`, por exemplo `minhaconta/apache:1.0`. Sem namespace, o push tenta o namespace `library`, reservado às imagens oficiais.

Como subir um registry local e enviar uma imagem a ele?

RESPOSTA

`docker run -d -p 5000:5000 -v registry-dados:/var/lib/registry registry:3`, depois `docker tag app:1.0 localhost:5000/app:1.0` e `docker push localhost:5000/app:1.0`.

15.3 BuildKit e Buildx

Como passar um token a um `RUN` sem deixá-lo na imagem?

RESPOSTA

`RUN --mount=type=secret,id=token ...` no Dockerfile e `docker build --secret id=token,src=arquivo .` (ou `env=VAR`). O segredo fica disponível só durante aquele `RUN`, em `/run/secrets/token`.

Para que serve `RUN --mount=type=cache`?

RESPOSTA

Mantém o cache do gerenciador de pacotes (npm, pip, Go, Maven, apt) entre builds sem gravá-lo na imagem.

Qual é a diferença entre `mode=min` e `mode=max` no cache exportado?

RESPOSTA

`min` exporta só as camadas da imagem final; `max` exporta também as camadas dos estágios intermediários, o que acelera builds multi-stage no CI.

Quais são as três estratégias de build multiplataforma?

RESPOSTA

Emulação **QEMU** (simples e lenta), **nós nativos** de cada arquitetura (rápida, exige infraestrutura) e **compilação cruzada** com `$BUILDPLATFORM` e `$TARGETARCH` (rápida, depende da linguagem).

O que causa `exec format error`?

RESPOSTA

Executar uma imagem (ou binário) de outra arquitetura de CPU, por exemplo `arm64` em um host `amd64` sem emulação.

Com o driver `docker-container`, por que a imagem não aparece em `docker image ls`?

RESPOSTA

Porque o resultado fica no cache do builder. É preciso `--load` (carregar no image store local) ou `--push` (enviar ao registry).

O que o Compose v5 mudou no build?

RESPOSTA

Removeu o builder interno e passou a delegar os builds ao **Docker Bake**.

O que são SBOM e provenance?

RESPOSTA

SBOM é o inventário de pacotes da imagem. **Provenance** registra como, onde e a partir de qual código a imagem foi construída (SLSA). Ambos são anexados com `--sbom=true` e `--provenance=mode=max`.

15.4 Redes e armazenamento

Por que usar uma rede bridge definida pelo usuário em vez da bridge padrão?

RESPOSTA

Porque ela tem **DNS embutido** (contêineres se encontram pelo nome), isola os contêineres de outras redes e permite conectar e desconectar em execução.

Qual é o endereço do DNS embutido do Docker dentro do contêiner?

RESPOSTA

`127.0.0.11`.

Um contêiner responde internamente, mas não pela porta publicada. Qual a causa mais comum?

RESPOSTA

A aplicação está escutando em `127.0.0.1` dentro do contêiner. Ela precisa escutar em `0.0.0.0`.

Por que uma porta publicada fica acessível mesmo com o UFW bloqueando?

RESPOSTA

As regras de iptables do Docker são avaliadas antes das regras do UFW. Restrinja com bind em `127.0.0.1` ou regras na chain `DOCKER-USER`.

Como um contêiner no Linux acessa um serviço do host?

RESPOSTA

Com `--add-host=host.docker.internal:host-gateway` e o endereço `host.docker.internal` (no Docker Desktop o nome já existe). `localhost` aponta para o próprio contêiner.

Quais portas precisam estar abertas entre os nós de um Swarm?

RESPOSTA

2377/tcp (gerenciamento), **7946/tcp e udp** (descoberta entre nós) e **4789/udp** (tráfego da overlay, VXLAN).

Qual é a diferença entre volume nomeado e bind mount?

RESPOSTA

O volume é gerenciado pelo Docker, fica na área de dados do daemon e é pré-populado com o conteúdo da imagem. O bind mount aponta para um caminho qualquer do host e esconde o conteúdo da imagem naquele diretório.

Por que `-v ./config.yaml:/app/config.yaml` pode criar um diretório no host?

RESPOSTA

Com `-v`, se o caminho de origem não existir, o Docker cria um **diretório** vazio. Com `--mount`, o comando falha com erro claro.

O que acontece com volumes anônimos em um `docker run --rm`?

RESPOSTA

São removidos junto com o contêiner. Volumes nomeados são preservados.

Como fazer backup de um volume?

RESPOSTA

Com um contêiner temporário que monta o volume e um diretório do host: `docker run --rm -v dados:/d -v "$(pwd)":/b alpine tar czf /b/dados.tgz -C /d ..` Para bancos, prefira a ferramenta de backup do próprio banco.

Quais storage drivers foram removidos do Docker e qual é o padrão hoje?

RESPOSTA

AUFS e overlay (Engine 24) e devicemapper (Engine 25). O padrão é overlay2 no store clássico e os **containerd** **snapshotters** em instalações novas do Engine 29.

O que foi o data-only container e o que o substituiu?

RESPOSTA

Um contêiner que nunca rodava, só para guardar volumes, montados em outros com `--volumes-from`. Foi substituído pelos **volumes nomeados** (`docker volume create`).

Duas instâncias de PostgreSQL podem usar o mesmo volume de dados?

RESPOSTA

Não. Cada instância supõe acesso exclusivo aos arquivos; isso impede a inicialização ou corrompe os dados. Use volumes separados e a replicação do banco.

Como descobrir onde um volume fica no host?

RESPOSTA

`docker volume inspect --format '{{ .Mountpoint }}' <volume>`. No Linux, `/var/lib/docker/volumes/<volume>/_data`; no Docker Desktop, o caminho fica dentro da VM.

15.5 Compose

Qual é a ordem de precedência do nome do projeto no Compose?

RESPOSTA

Flag `-p`, depois `COMPOSE_PROJECT_NAME`, depois o atributo `name`: do arquivo e, por fim, o nome do diretório.

Quais são as condições do `depends_on`?

RESPOSTA

`service_started` (padrão), `service_healthy` (espera o healthcheck) e `service_completed_successfully` (espera sair com código 0).

Qual é a diferença entre o arquivo `.env` e o atributo `env_file`?

RESPOSTA

O `.env` alimenta a **interpolação** de `${VAR}` no arquivo Compose. O `env_file` define variáveis **dentro do contêiner**.

Qual é a precedência de variáveis de ambiente dentro de um contêiner do Compose?

RESPOSTA

`docker compose run -e` > `environment` > `env_file` > ENV da imagem.

O que `docker compose down -v` faz a mais que `down` ?

RESPOSTA

Remove também os volumes nomeados declarados no projeto e os anônimos, apagando os dados.

Onde os `secrets` do Compose aparecem no contêiner?

RESPOSTA

Como arquivos em `/run/secrets/<nome>`.

Para que servem os `profiles` ?

RESPOSTA

Serviços com perfil só sobem quando o perfil é ativado (`--profile` ou `COMPOSE_PROFILES`); serviços sem perfil sobem sempre.

Quais são as ações do `develop.watch` ?

RESPOSTA

`sync` (copia arquivos), `rebuild` (reconstrói e recria), `sync+restart` (copia e reinicia) e `sync+exec` (copia e executa um comando).

Qual comando mostra o Compose resolvido, com variáveis e arquivos mesclados?

RESPOSTA

`docker compose config`.

O campo `version:` ainda é necessário no `compose.yaml` ?

RESPOSTA

Não. É obsoleto, só gera aviso; o Compose sempre valida com o schema mais recente.

15.6 Segurança e operação

Quantas capabilities o Docker concede por padrão e como reduzi-las?

RESPOSTA

14 (como `CHOWN`, `NET_BIND_SERVICE`, `NET_RAW`, `SETUID`). Reduza com `--cap-drop ALL` e adicione só o necessário com `--cap-add`.

O que `--privileged` faz?

RESPOSTA

Concede todas as capabilities, acesso a todos os dispositivos do host e desliga seccomp e AppArmor. Na prática, remove o isolamento.

Qual é a diferença entre o modo `rootless` e o `usersns-remap`?

RESPOSTA

No **rootless**, o daemon e os contêineres rodam como usuário comum. No `usersns-remap`, o daemon continua root, mas o root do contêiner é mapeado para um UID sem privilégio no host.

Por que variáveis de ambiente não são o melhor lugar para segredos?

RESPOSTA

Aparecem em `docker inspect`, em `/proc/<pid>/environ`, em dumps e logs de erro, e são herdadas por processos filhos. Segredos em arquivo (`/run/secrets`) ou em cofre são mais seguros.

O que substituiu o Docker Content Trust para assinar imagens?

RESPOSTA

Sigstore/**cosign** (inclusive assinatura keyless com OIDC) e **Notation**. O DCT (Notary v1) foi aposentado.

O que acontece quando um contêiner ultrapassa `--memory`?

RESPOSTA

O kernel mata o processo por OOM; o contêiner sai com código **137** e `OOMKilled: true`.

Qual é a diferença entre `--cpus` e `--cpu-shares`?

RESPOSTA

`--cpus` é um **limite** de tempo de CPU. `--cpu-shares` é um **peso relativo** (padrão 1024) que só vale quando há disputa.

Qual driver de log faz rotação por padrão?

RESPOSTA

O `local` (20 MB × 5 arquivos). O `json-file`, padrão do Docker, **não** faz rotação a menos que `max-size` e `max-file` sejam definidos.

Qual é o tamanho padrão do `/dev/shm` e quando aumentá-lo?

RESPOSTA

64 MB. Aumente com `--shm-size` para navegadores headless e alguns bancos de dados.

15.7 Orquestração e produção

Quantas falhas um Swarm com 5 managers tolera?

RESPOSTA

2. A fórmula é $(N - 1) / 2$.

O que acontece com um Swarm que perde o quórum?

RESPOSTA

As tarefas existentes continuam rodando, mas não é possível alterar o cluster. A recuperação é `docker swarm init --force-new-cluster` em um manager sobrevivente.

Qual é a diferença entre serviços `replicated` e `global` no Swarm?

RESPOSTA

Replicated mantém um número fixo de réplicas; **global** roda exatamente uma tarefa em cada nó elegível.

O que é o routing mesh?

RESPOSTA

A rede `ingress` do Swarm que publica a porta de um serviço em **todos os nós** e encaminha para uma tarefa saudável, mesmo em nós sem réplica.

Como colocar um nó do Swarm em manutenção?

RESPOSTA

`docker node update --availability drain <nó>`: as tarefas são movidas para outros nós e o nó deixa de receber novas.

O Kubernetes usa o `HEALTHCHECK` da imagem?

RESPOSTA

Não. Ele usa `livenessProbe`, `readinessProbe` e `startupProbe` definidos no manifest.

Por que promover o mesmo digest entre ambientes em vez de reconstruir?

RESPOSTA

Porque garante que o artefato testado é exatamente o que vai para produção; um rebuild pode trazer dependências ou bases diferentes.

Qual é o risco de montar o `docker.sock` em um job de CI?

RESPOSTA

O job ganha controle total do daemon, e portanto root no host, além de ver os contêineres de outros jobs.

Quando o Docker Desktop exige assinatura paga?

RESPOSTA

Para uso profissional em empresas com 250 funcionários ou mais, **ou** com receita anual de US\$ 10 milhões ou mais, e para órgãos de governo.

Em quais nós é possível executar comandos de administração do Swarm?

RESPOSTA

Em **qualquer manager** (`Leader` ou `Reachable`); o pedido é encaminhado ao líder. Workers não aceitam comandos de cluster.

Como remover um manager do Swarm com segurança?

RESPOSTA

Rebaixá-lo com `docker node demote`, depois `docker swarm leave` no próprio nó e `docker node rm` em um manager. Um manager saindo sem ser rebaixado exige `--force` e pode quebrar o quórum.

Um volume montado em um serviço com 5 réplicas é compartilhado entre os nós?

RESPOSTA

Não, com o driver `local`: cada nó cria o próprio volume. Réplicas em nós diferentes veem dados diferentes. Para compartilhar, use um driver de rede (NFS, plugin) ou um serviço de dados externo.

O que é o VIP de um serviço do Swarm?

RESPOSTA

Um IP virtual na rede do serviço; conexões a ele (ou ao nome do serviço) são balanceadas entre as tarefas. Aparece em `Endpoint.VirtualIPs` no `docker service inspect`.

Como rotacionar um secret usado por um serviço?

RESPOSTA

Criar um secret novo, `docker service update --secret-rm antigo --secret-add source=novo,target=<mesmo-nome> <serviço>` e depois `docker secret rm antigo`. Secrets são imutáveis.

Qual é o tamanho máximo de um Docker secret e onde ele aparece no contêiner?

RESPOSTA

500 KB. Aparece como arquivo em `/run/secrets/<nome>` (ou no `target` definido), montado em `tmpfs`.

Qual é a diferença entre `docker compose up` e `docker stack deploy`?

RESPOSTA

Os dois usam arquivos Compose. `docker compose` roda os serviços em **um host**; `docker stack deploy` os cria como **serviços do Swarm** no cluster, usando a seção `deploy` e ignorando `build` e `depends_on`.

Como atualizar um stack já implantado?

RESPOSTA

Rodando de novo `docker stack deploy -c arquivo.yaml <stack>`. O Swarm aplica rolling update só no que mudou; `--prune` remove serviços que saíram do arquivo.

Por que `node-exporter` e `cAdvisor` rodam em modo `global`?

RESPOSTA

Porque precisam coletar métricas de **todos os nós**, inclusive dos que entrarem depois; o modo `global` garante uma tarefa por nó.

Síntese de revisão

Este guia organiza o estudo de Docker em cinco blocos, do funcionamento do contêiner no kernel até a imagem assinada em produção, e reúne formatos de consulta rápida e prática ativa.

- Um contêiner é um processo isolado por namespaces e limitado por cgroups; ele vive enquanto o PID 1 vive. Forma exec, parada graciosa e códigos de saída explicam boa parte dos problemas de execução (capítulos 1 e 2).
- Imagens são camadas endereçadas por conteúdo: tags mudam, digests não. Ordem das instruções, multi-stage, build secrets e cache exportado definem imagens pequenas, rápidas de construir e sem segredos (capítulos 3 a 5).
- Redes definidas pelo usuário, portas publicadas só quando necessário, volumes para dados persistentes e Compose com healthchecks formam a base de qualquer aplicação multi-contêiner (capítulos 6 a 8).
- Segurança e operação dependem de menos privilégios, limites de recursos, logs com rotação e uma cadeia de suprimentos verificável; a imagem é construída uma vez e promovida pelo digest (capítulos 9 a 12).
- As tabelas de decisão, as pegadinhas e a referência de comandos (capítulos 13 e 14) servem de revisão rápida; o autoteste (capítulo 15) oferece 104 perguntas para revisão ativa. Versões e limites foram conferidos em setembro de 2026.